UNIVERSITY OF NORTH TEXAS

Software Engineering

5430

Spring 2023

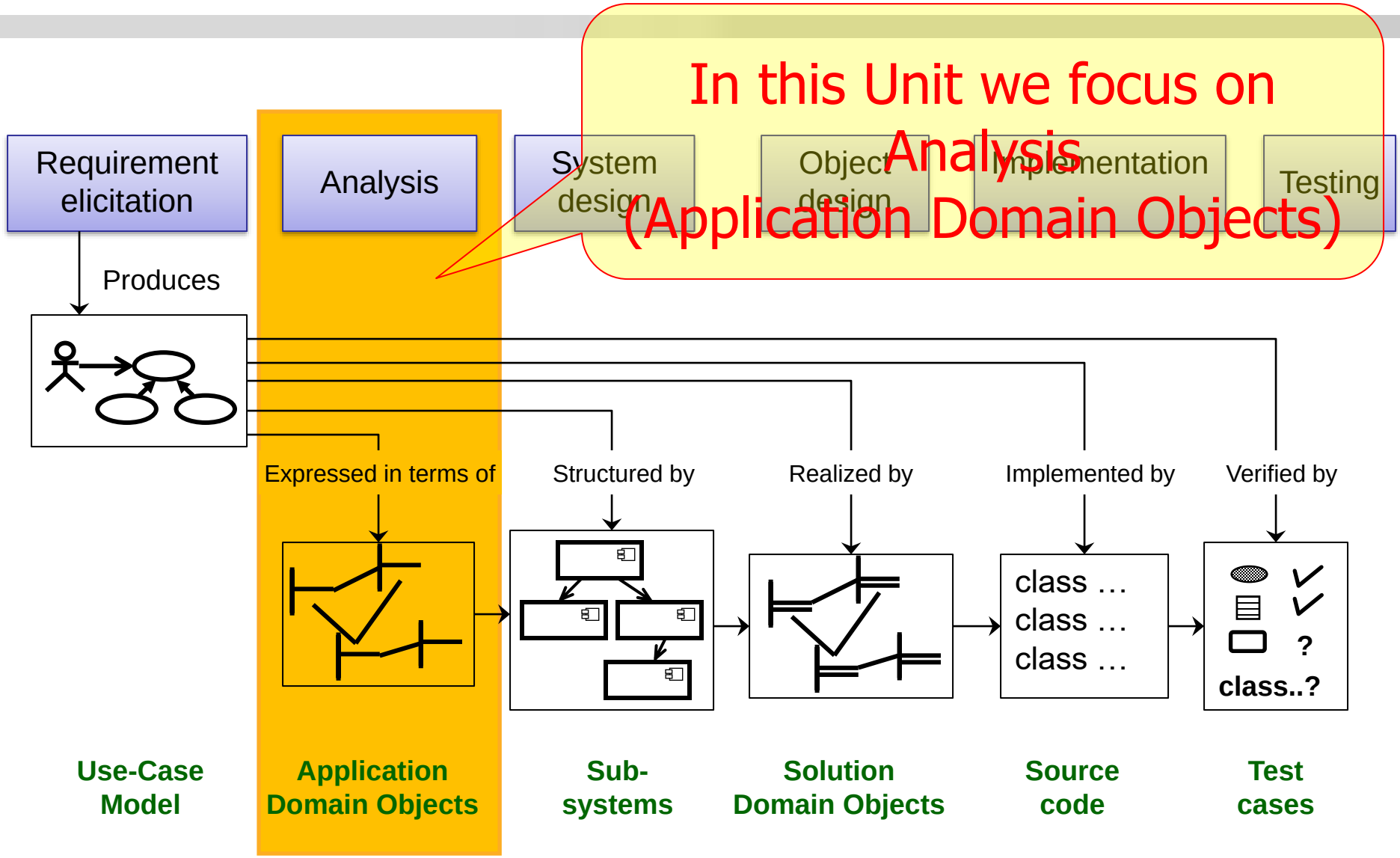
by:
Abdelnasser Ouda
Computer Science Department
Email: abdelnasser . ouda @ unt.edu
Discovery Park F203

Unit 3: Analysis

- An Overview of Analysis
- Analysis Concepts
- Classes and Class Diagrams
- Analysis Activities: From Use Cases to Objects
- Analysis Activities: Identifying Associations, Aggregations, and Attributes
- Modeling Ordered Interactions: Sequence Diagrams
- Mapping Use Cases to Objects with Sequence Diagrams
- Modeling System Workflows: Activity Diagrams
- Modeling State-Dependent Behavior of Individual Objects

An Overview of Analysis

Analysis and SDLC



Identifying Software Development Activities

Recall

- Requirements Elicitation
- Analysis
- System Design

- What is the problem?
- Is the problem fully described?
- What is the solution?

Application Domain

- Object Design
- Program Implementation
- Testing
- Delivery
- Maintenance

- What are the best mechanisms to implement the solution?
- How is the solution constructed?
- Is the problem solved?
- Can the customer use the solution?
- Are enhancements needed?

Solution Domain

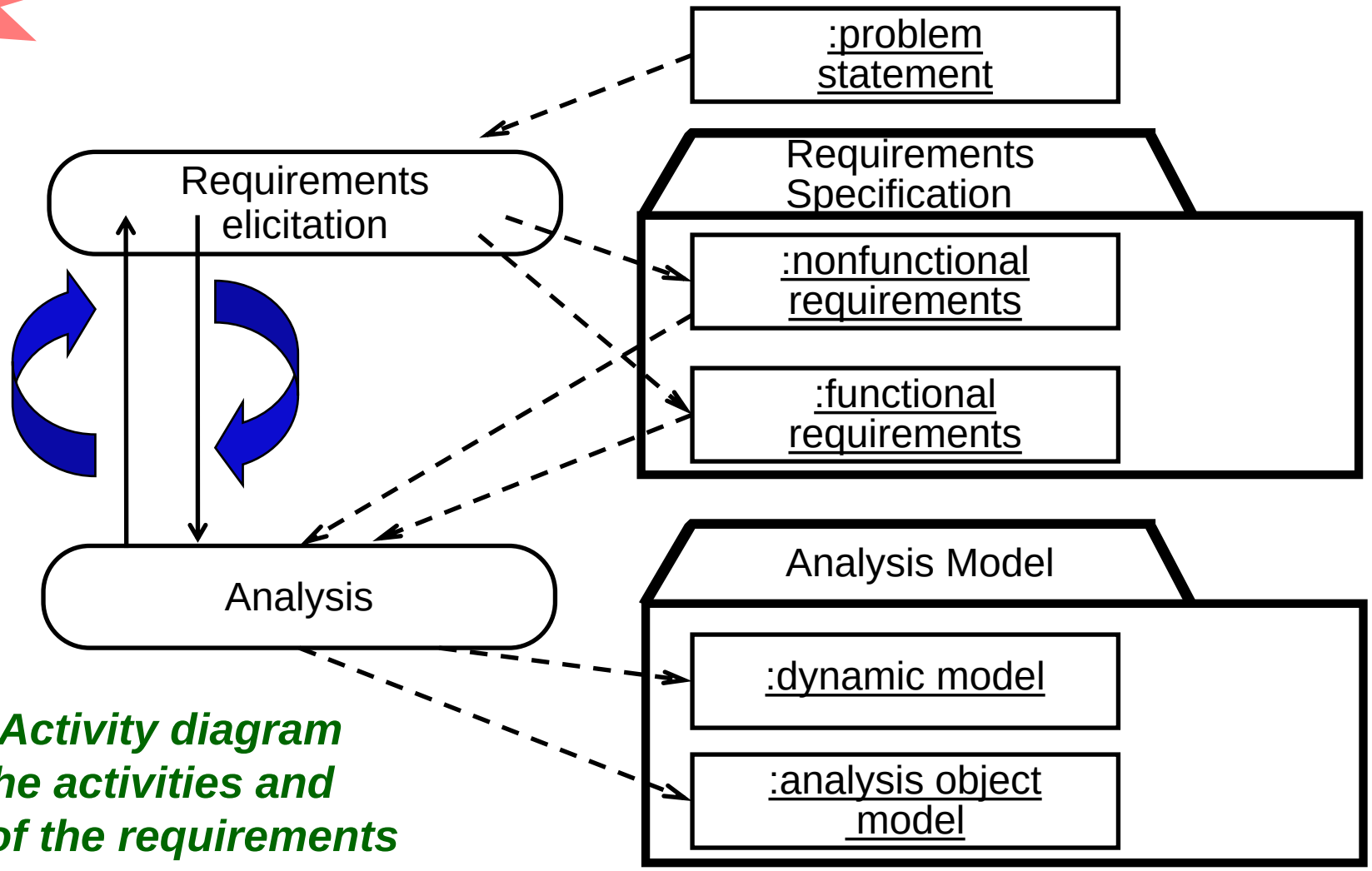
Analysis model

- Analysis results in **a model of the system** that aims to be
  - **correct,**
  - **complete,**
  - **consistent, and**
  - **unambiguous.**

- In object-oriented analysis, software engineers **build a model** (called analysis model) to describe the application domain.
- To build this model, we will focus on
  - **Objects identification,**
  - **Objects behavior,**
  - **Objects relationships, and their classifications.**

From Requirements Specification to Analysis Model

Recall

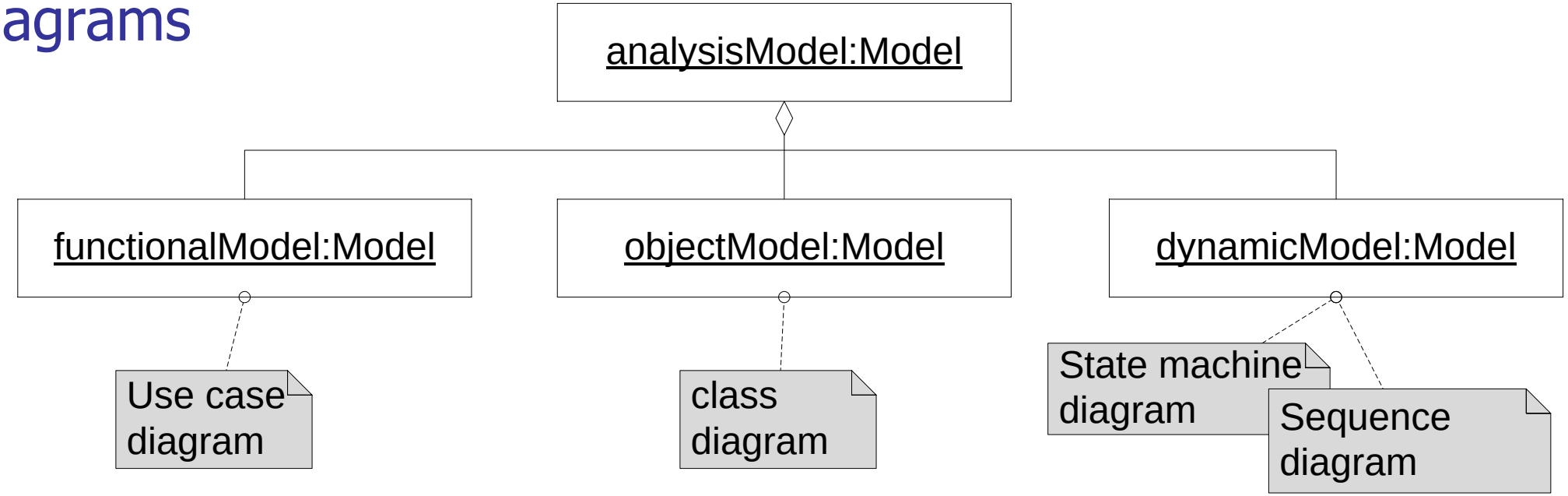


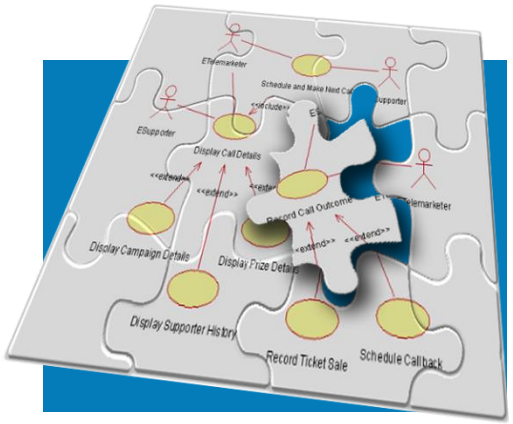
This is an Activity diagram showing the activities and products of the requirements engineering process

The requirements specification is structured and formalized during analysis to produce an analysis model

Analysis model ...

- The analysis model is composed of three individual models:
  - the functional model, represented by use cases and scenarios,
  - the analysis object model, represented by class and object diagrams, and
  - the dynamic model, represented by state machine and sequence diagrams



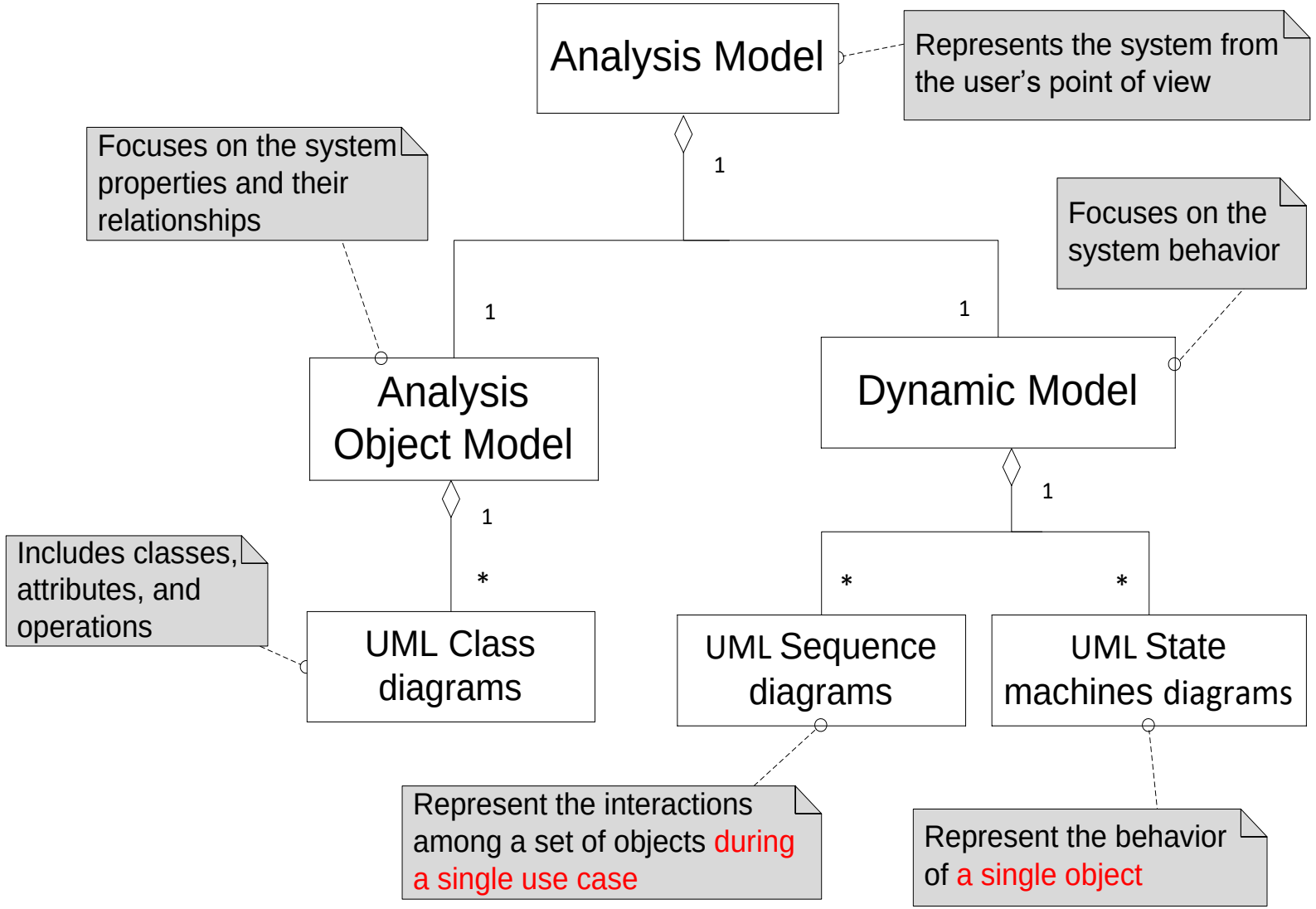


Analysis Concepts

Analysis Object Models and Dynamic Models (1)

Note that these models represent user- level concepts, not actual software classes or components.

For example, classes such as Database or Network should not appear in the analysis model as the user is completely isolated from those concepts.



There are different types of Objects

- **Entity Objects**
  - Represent the persistent information tracked by the system (Application domain objects, also called "Business objects")
- **Boundary Objects**
  - Represent the interaction between the user and the system
- **Control Objects**
  - Represent the control tasks performed by the system.

ATM example

The Customer inserts his bank card into the ATM.

The ATM welcomes the Customer and opens the main menu in the ATM main screen.

The Customer selects the “CheckAccountBalance” function by pressing the appropriate button.

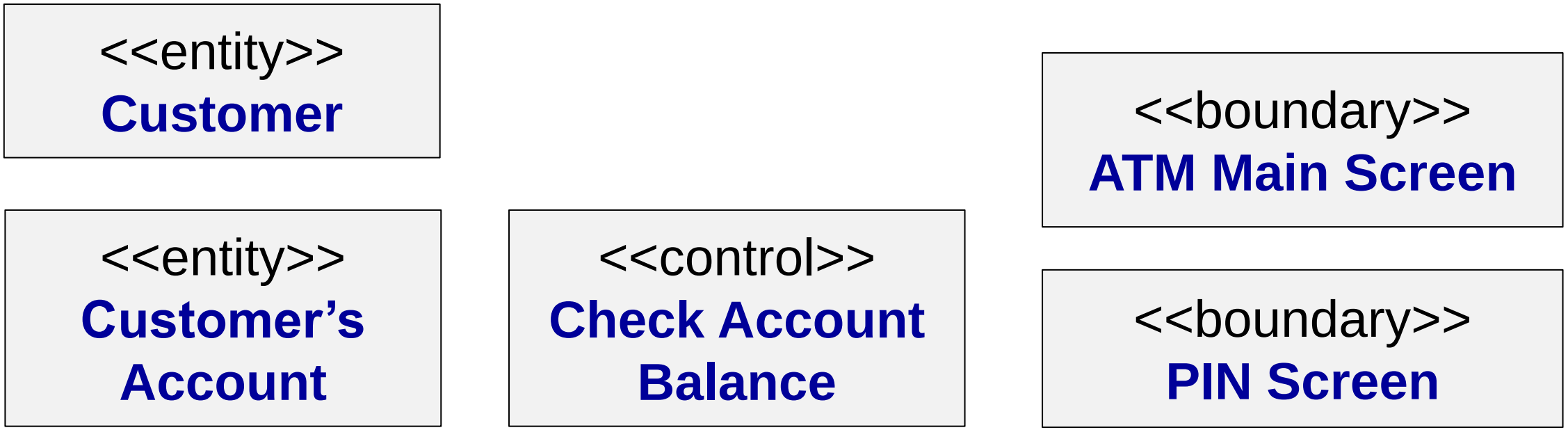
The ATM system shows the enter PIN screen to the Customer.

The Customer enters his correct PIN and confirms it.

The ATM system shows the actual balance of the Customer’s account.

Naming Object Types in UML

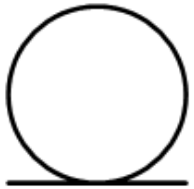
To distinguish different object types in a model we can use the UML Stereotype mechanism



Entity Objects **Control Object** **Boundary Objects**

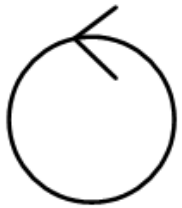
UML is an Extensible Language

- Stereotypes can be represented with icons and graphics:
  - This can increase the readability of UML diagrams.
- One can use icons to identify a stereotype
  - When the stereotype is applied to a UML model element, the icon is displayed beside or above the name



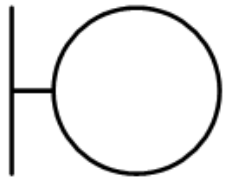
Customer

Entity Objects



Check Account Balance

Control Object

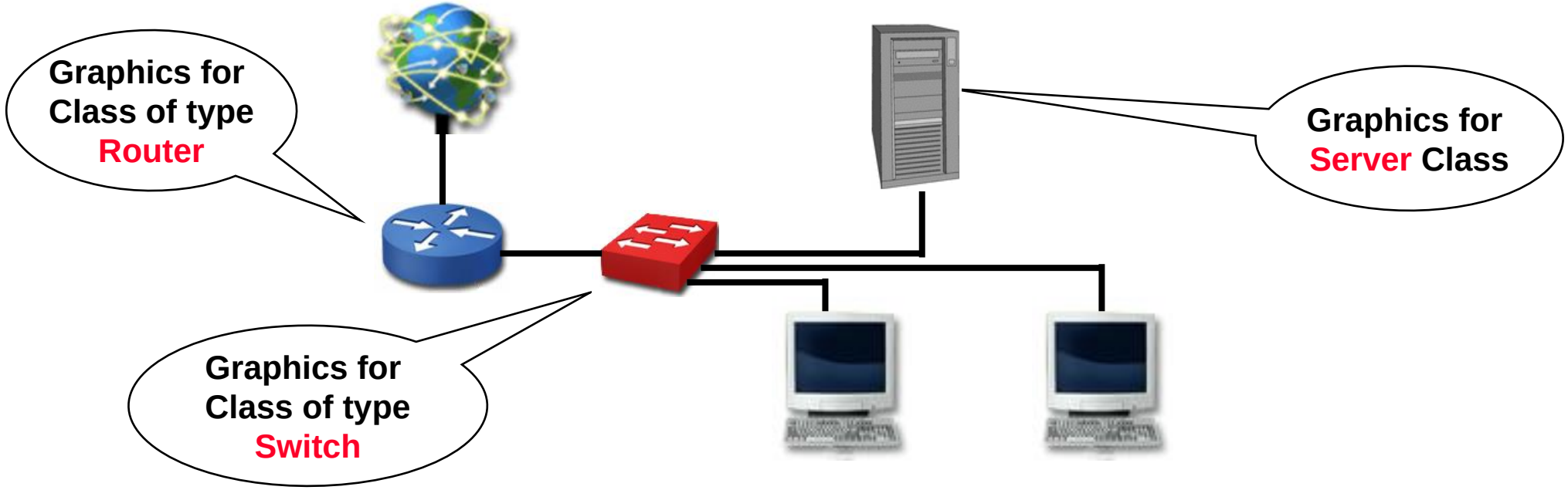


ATM Main Screen

Boundary Objects

Graphics for Stereotypes

- One can also use graphical symbols to identify a stereotype
  - When the stereotype is applied to a UML model element, the graphic replaces the default graphic for the diagram element.
- Example: When modeling a network, define graphics for representing classes of type Switch, Server, Router,etc.



Object Types allow us to deal with Change

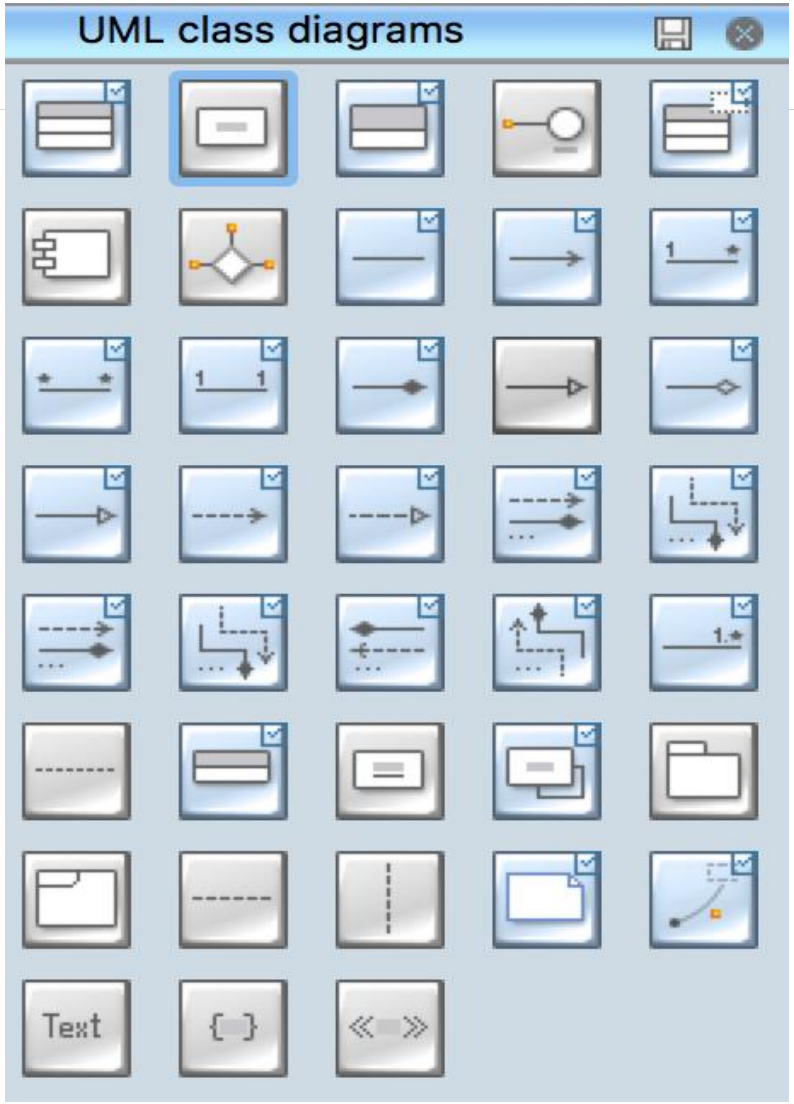
- Having three types of object leads to models that are more resilient to change
  - The interface of a system changes more likely than the control
  - The way the system is controlled changes more likely than entities in the application domain
- The most common Software architecture: Model, View, Controller (MVC)

Model <-> Entity Object

View <-> Boundary Object

Controller <-> Control Object

Next topic: Finding objects.



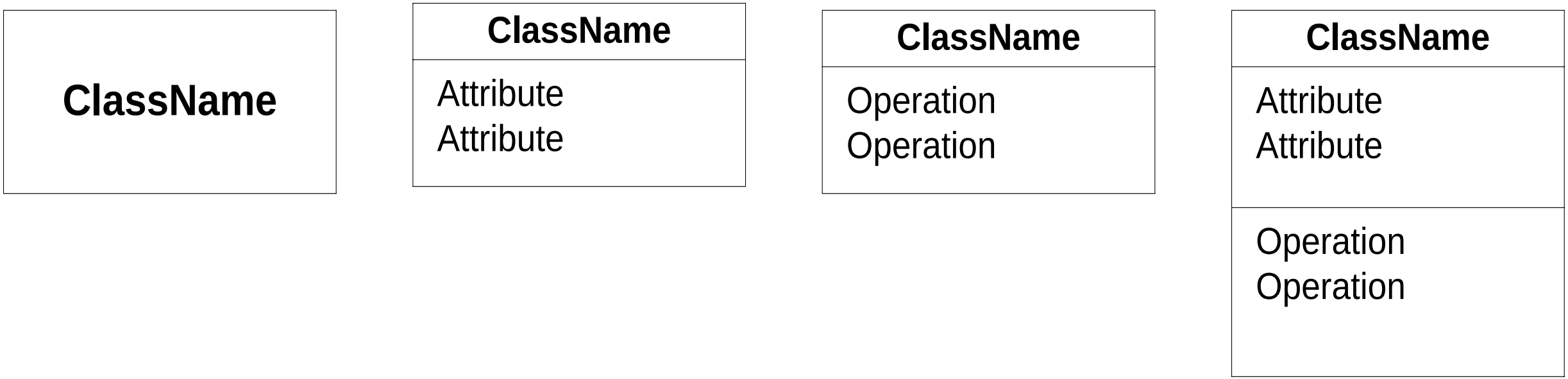
Classes and Class Diagrams

Classes and Objects

- Class diagrams **describe the structure of the system** in terms of classes and objects.
- **Classes** are abstractions that specify the attributes and behavior of a set of objects. **Objects** are entities that encapsulate state and behavior.
- Naming convention: class names start with uppercase letter, multiple words joined; each word starting with a capital letter (e.g. PostalAddress)

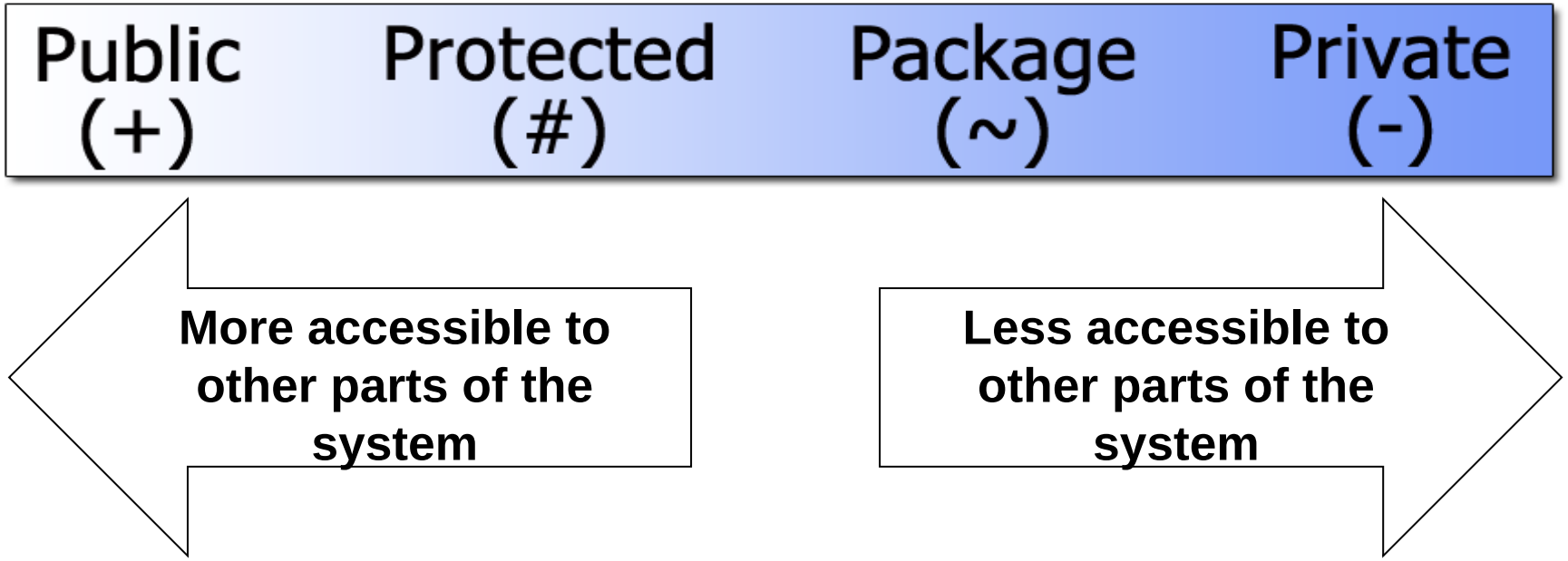
Classes and Objects …

- A class can be represented by four different ways using UML notations.



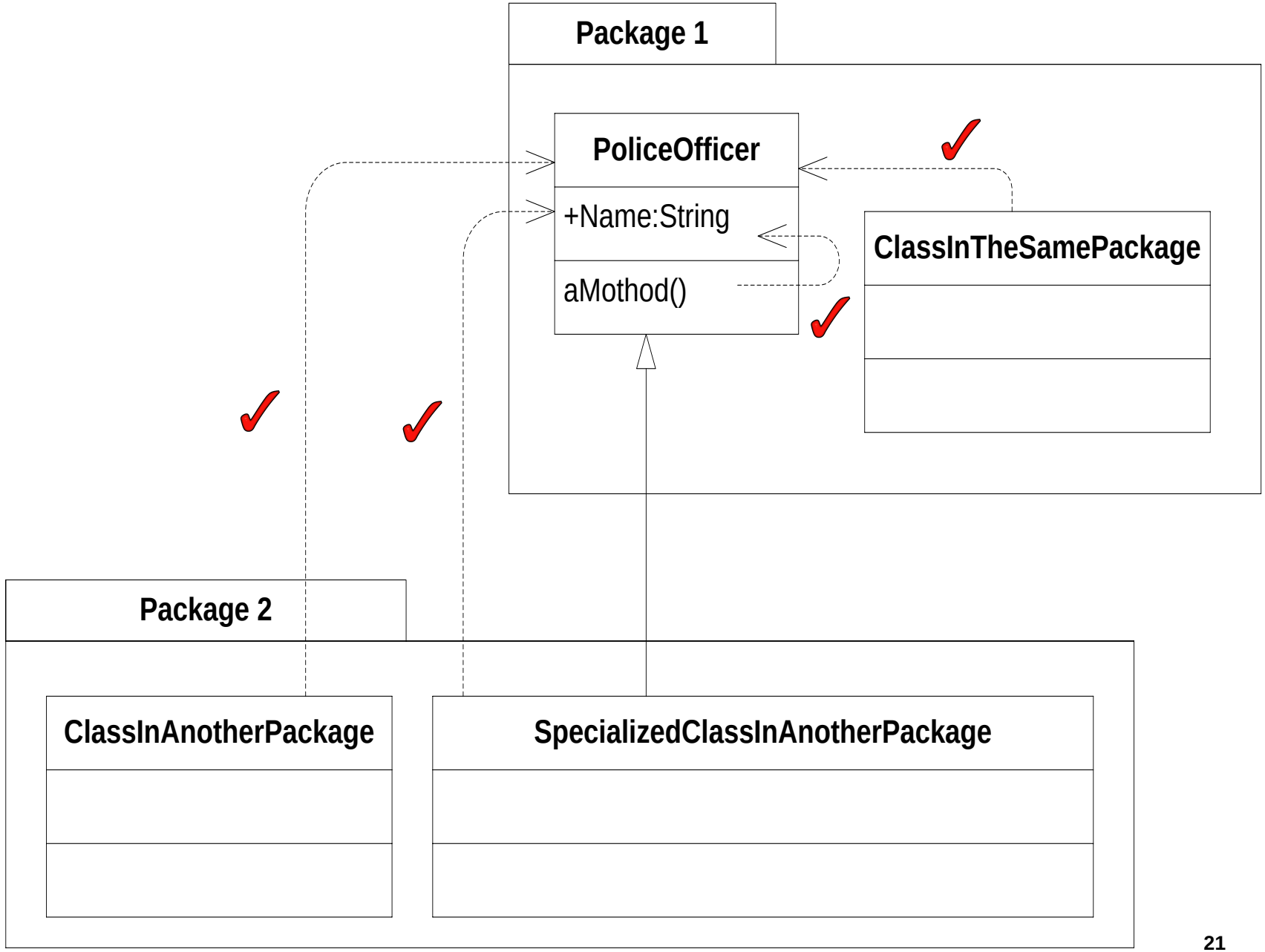
Visibility

- The access to attributes, operations, and even entire classes can be controlled by applying **visibility** characteristics.
- There are **four** different types of visibility that can be applied to the elements of a UML model.



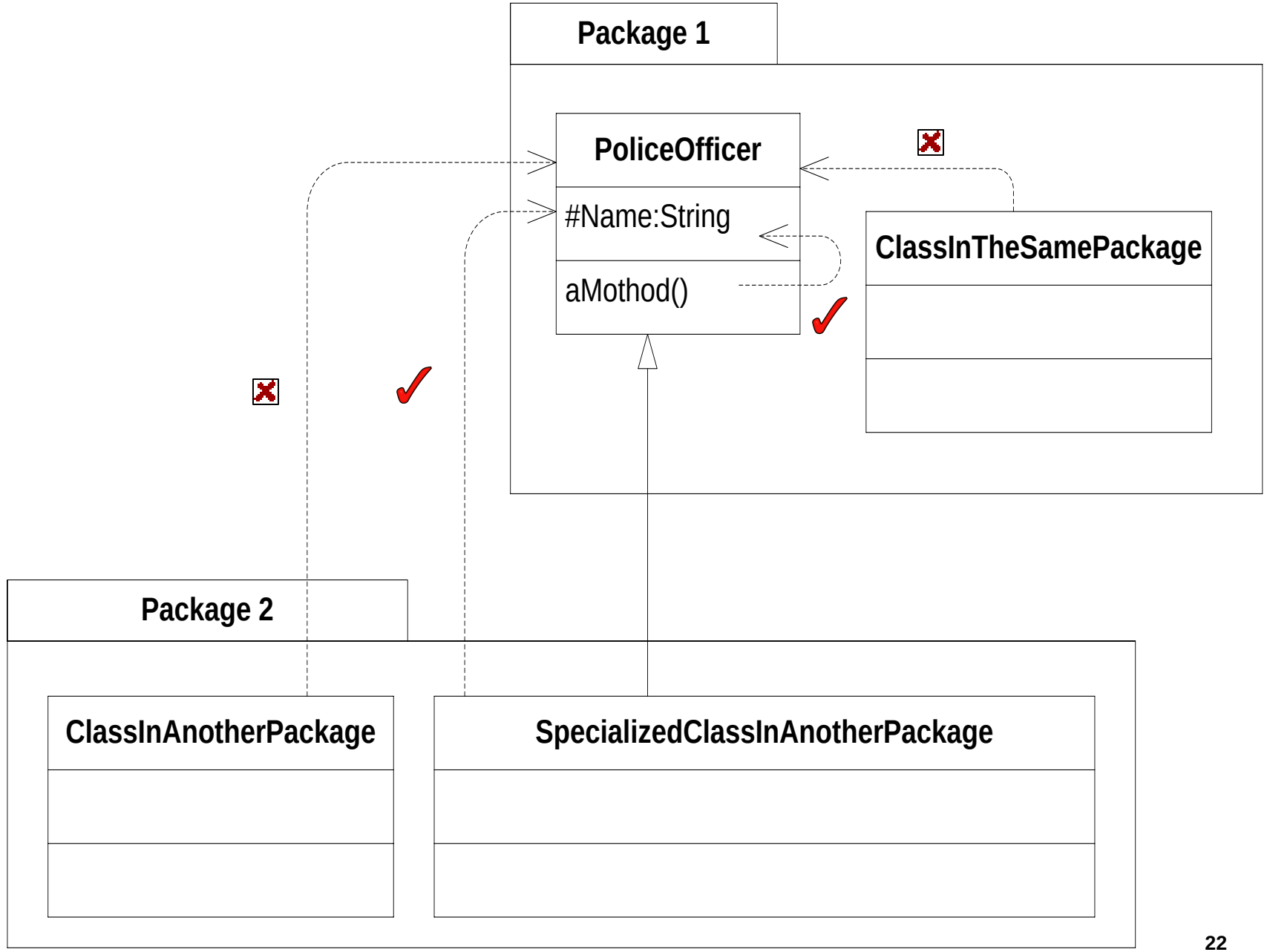
Public Visibility (+)

- Public visibility is specified using the plus (+) symbol before the associated attribute or operation.
- Declare an attribute or operation public if you want it to be accessible directly by any other class.



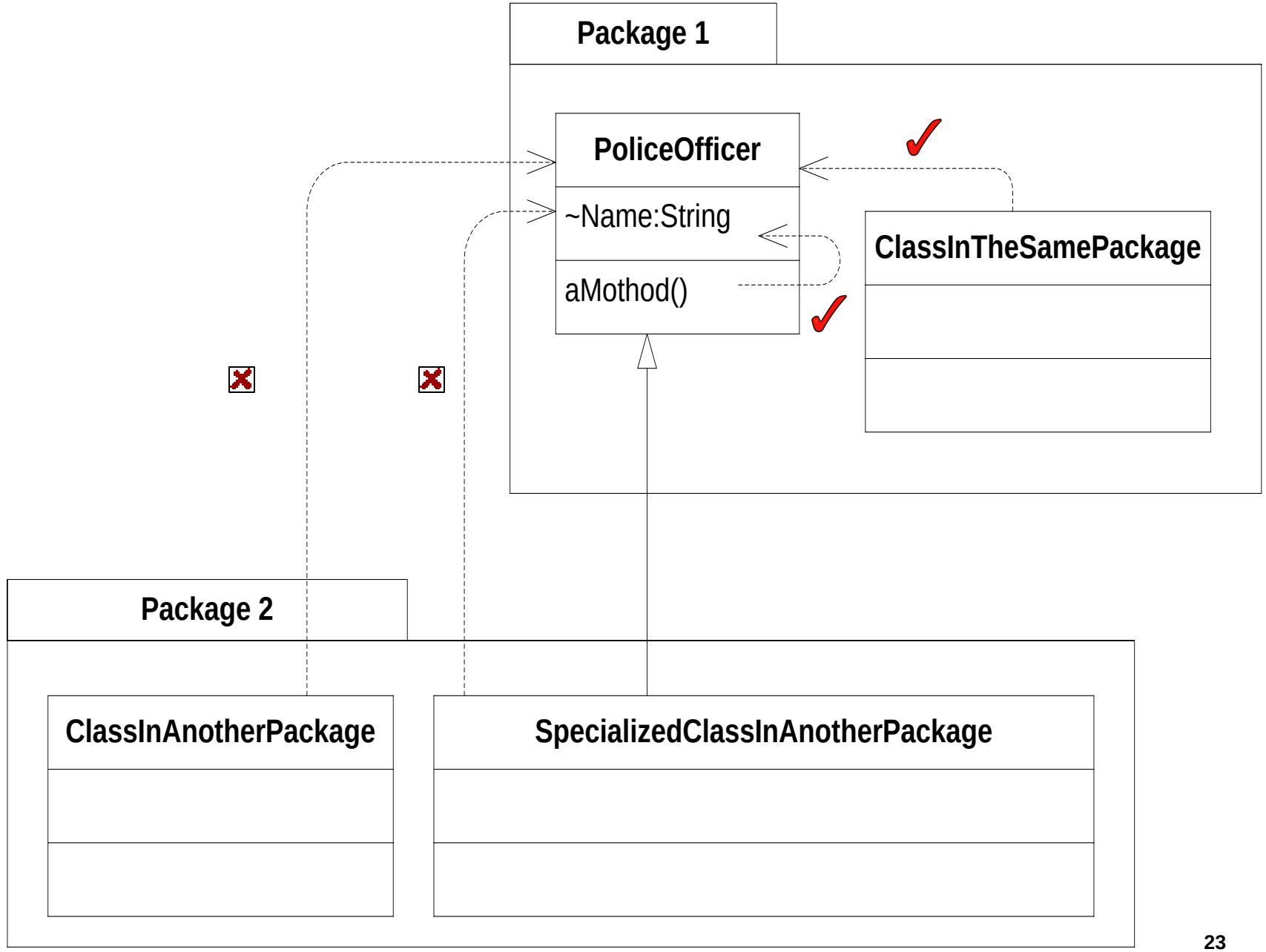
Protected Visibility (#)

- Protected attributes and operations are specified using the hash (#) symbol
- Using protected visibility is like saying, "This attribute or operation is useful inside my class and classes extending my class, but no one else should be using it."



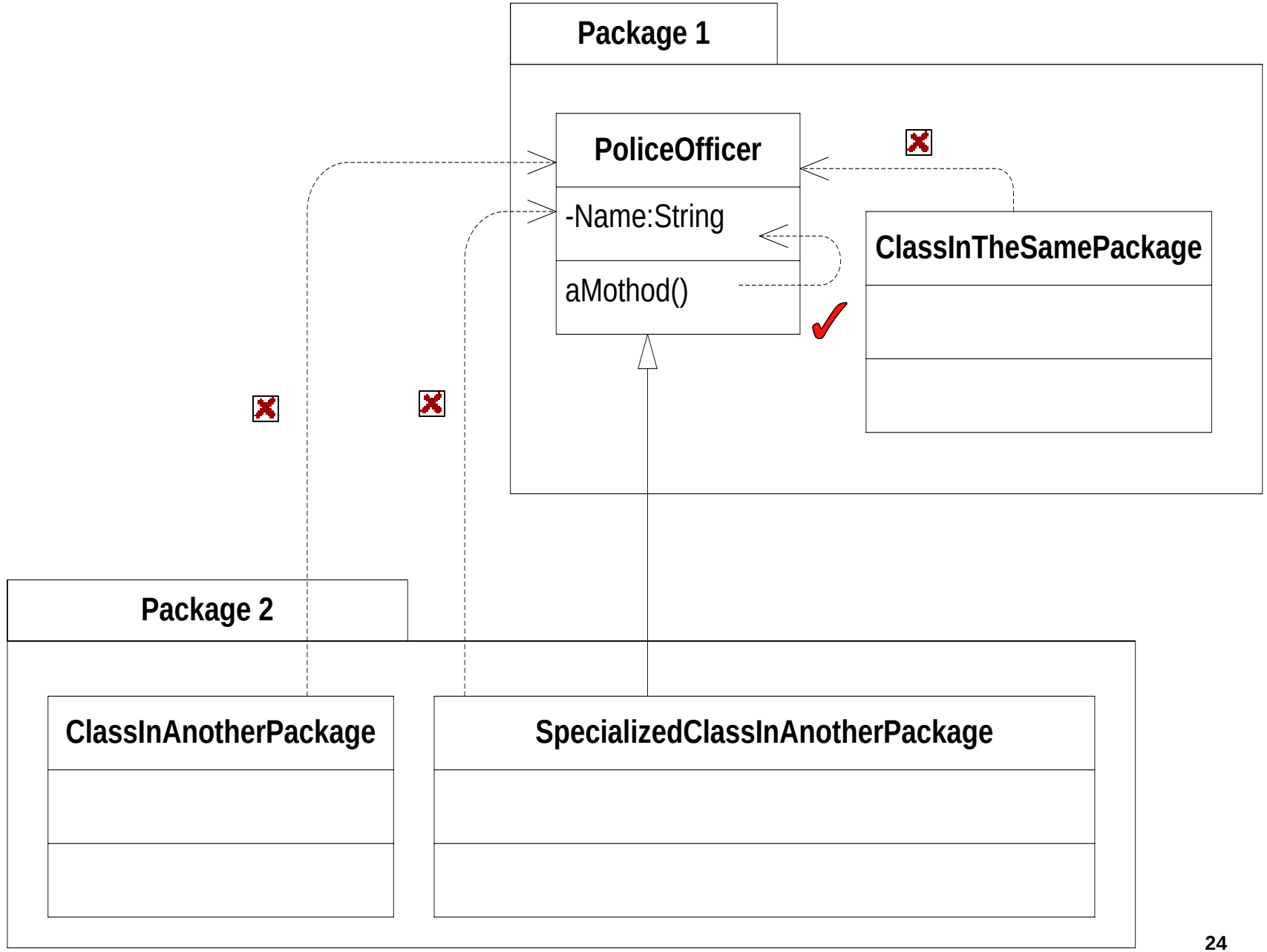
Package Visibility (~)

- Package visibility, specified with a tilde (~), when applied to attributes and operations.



Private Visibility (-)

- Private visibility is the most tightly constrained type of visibility classification, and it is shown by adding a minus (-) symbol before the attribute or operation.



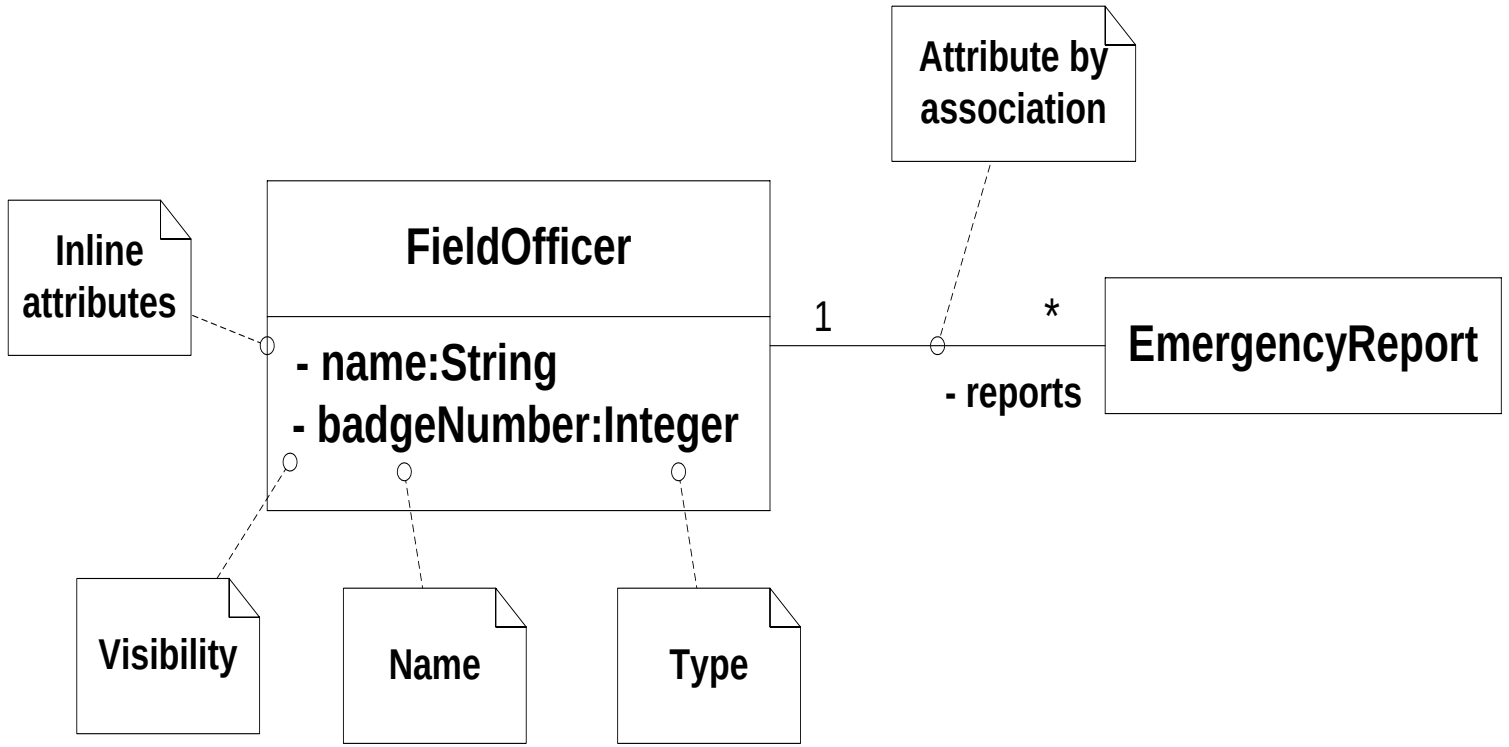
Let us compare

- Let's look at two OO languages in detail, let us compares UML visibility semantics with those of Java and C#.

Visibility	UML semantics	Java semantics	C# semantics
public	Any element that can access the class, can access any of its features with public visibility	Same as UML	Same as UML
protected	Only operations within the class, or within children of the class, can access features with protected visibility	As UML but is also accessible to all classes in the same Java package as the defining class	Same as UML
package	Any element that is in the same package as the class, or in a nested subpackage, can access any of its features with package visibility	The default visibility in Java – nested classes in nested sub-packages don't automatically have access to elements in their parent package	–
private	Only operations within the class can access features with private visibility	Same as UML	Same as UML
internal	–	–	Accessible by any element in the same program
protected internal	–	–	Combines the semantics of protected and internal only applicable to attributes

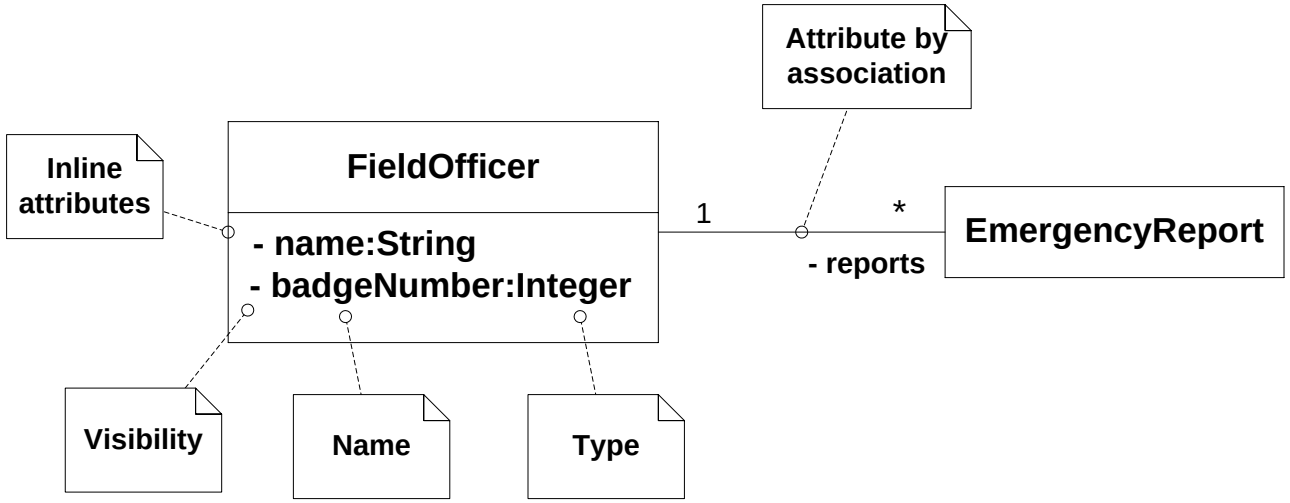
Class State: Attributes

- A class's attributes are the pieces of information that represent the **state** of an object.
- These attributes can be represented inside the class box, known as **inline attributes** or by **association** with another class.



- The only mandatory part of the UML attribute syntax is the attribute **name**.
- Naming convention: attributes start with a small letter; capitalize successive words (e.g., badgeNumber)

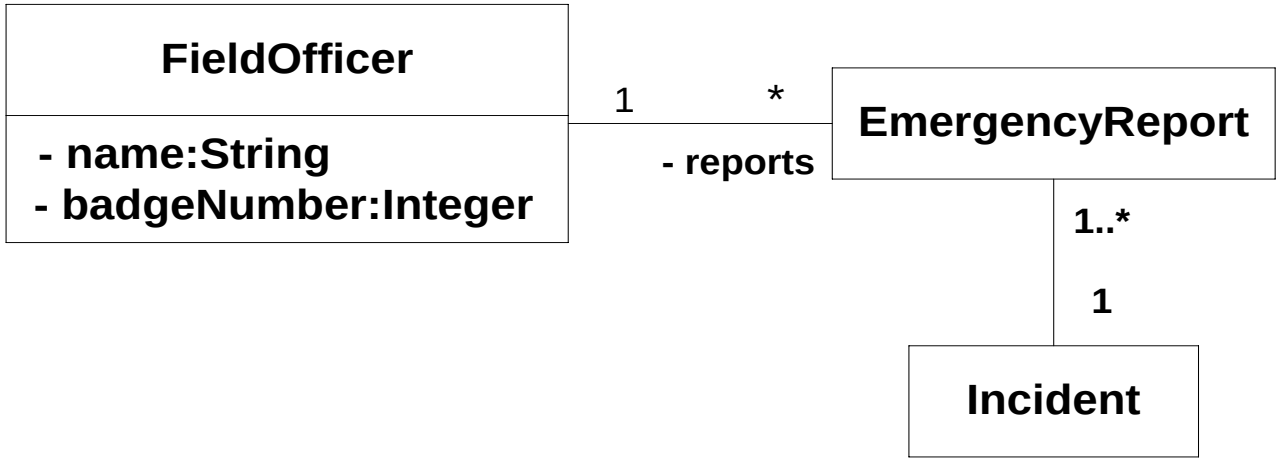
Class State: Attributes




```
public class FieldOfficer
{  // The two inline attributes from Figure.
   private String name;
   private Integer badgeNumebr;
   // The single attribute by association, given the name 'reports'
   private  EmergencyReports [] reports;
   //
   ...
}
```

Multiplicity

- Sometimes an attribute will represent more than one object.
- Multiplicity allows you to specify that an attribute actually represents a collection of objects.
- Multiplicity can be applied to both inline and attributes by association



As per this class diagram, each FieldOfficer has a list of EmergencyReports, and all EmergencyReports are written by exactly one FieldOfficer.
Also any Incident should relate to at least one EmergencyReport.

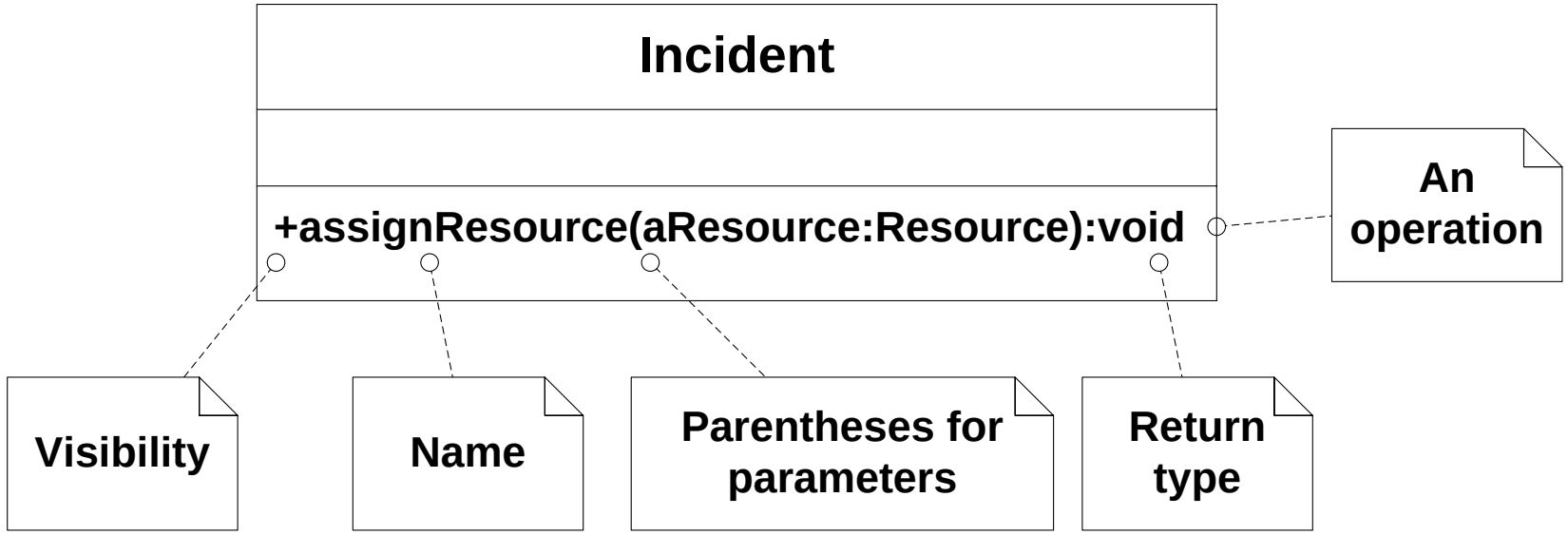
Multiplicity …

- If multiplicity is not explicitly stated, then it is undecided
  - there is no "default" multiplicity in UML.
- It is a common UML modeling **error** to assume that an undecided multiplicity defaults to a multiplicity of 1.
- Consider the following as examples of multiplicity syntax

Multiplicity symbols	Semantics
0..1	Zero or 1
1	Exactly 1
0..*	Zero or more
*	Zero or more
1..*	1 or more
1..6	1 to 6
1..3, 7..10, 15, 19..*	1 to 3 or 7 to 10 or 15 exactly or 19 to many

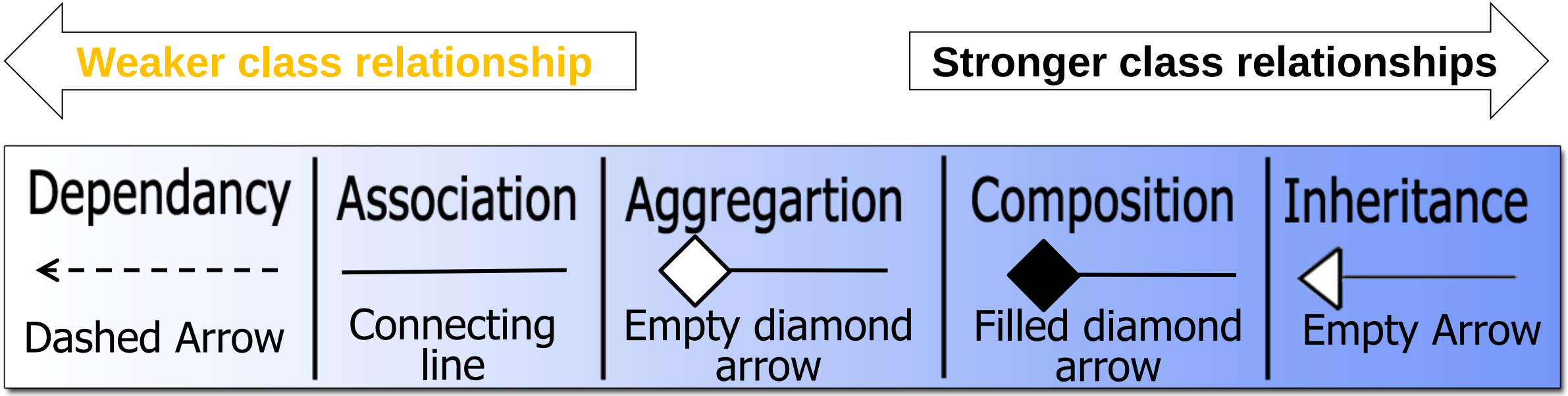
Class Behavior: Operations

- A class's operations describe what a class can do but not necessarily how it is going to do it.
- Operations in UML are specified on a class diagram with a signature that is at minimum made up of a visibility property, a name, a pair of parentheses in which any parameters that are needed for the operation to do its job can be supplied, and a return type, as shown



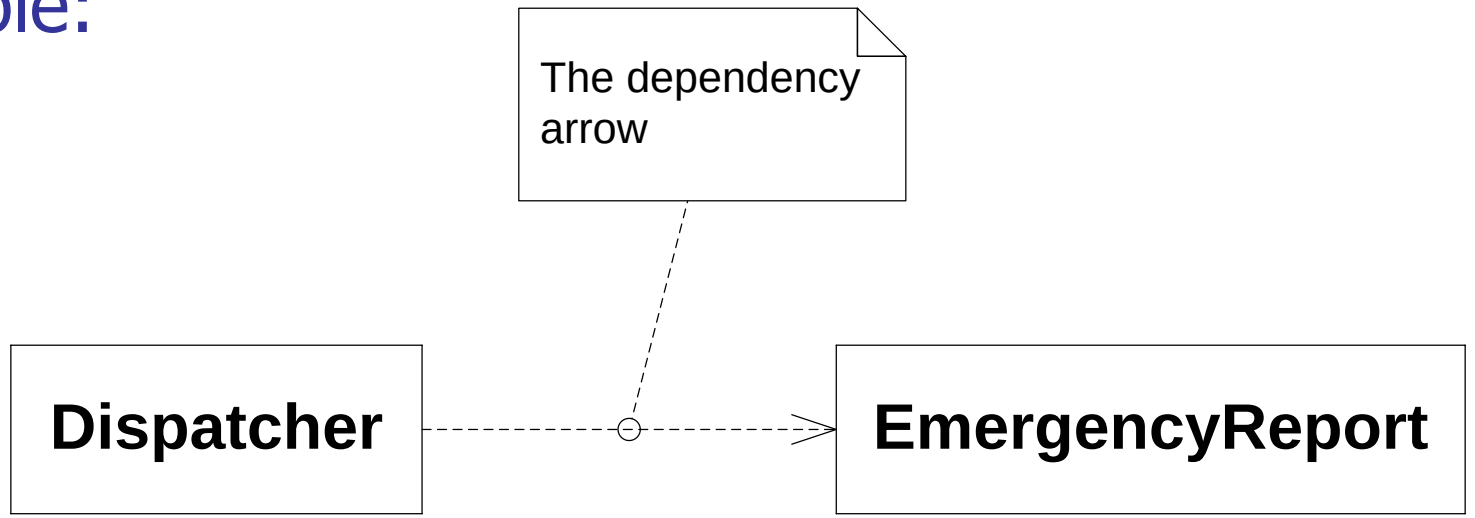
Class Relationships

- Classes work together using different types of relationships.
- Relationships between classes come in different strengths, as shown.



Dependency

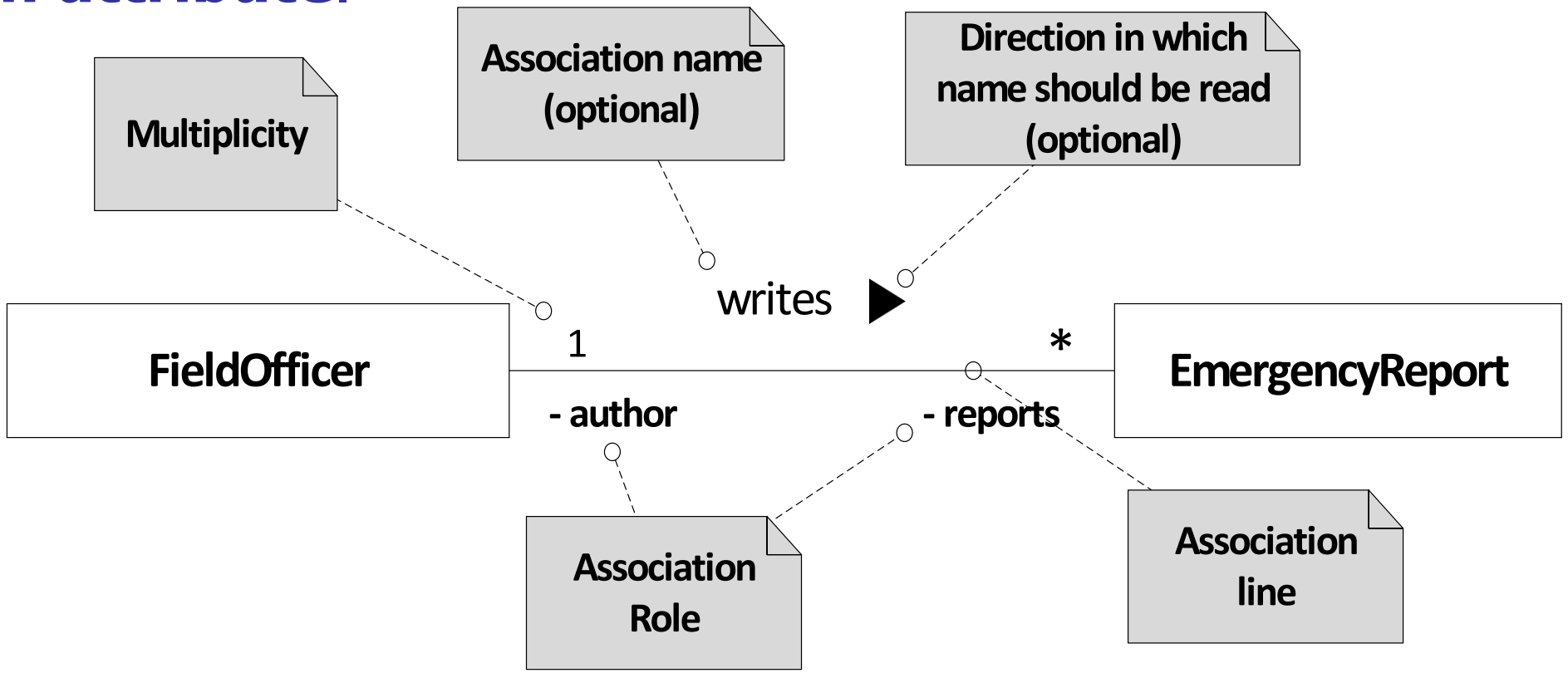
- A dependency between two classes declares that a class needs to know about another class **to use** objects of that class.
- For example:



- The Dispatcher and EmergencyReport classes simply work together at the times when the Dispatcher wants to display the Incident form.

Association

- Although dependency simply allows one class to use objects of another class, association means that a class will actually **contain** a reference to an object, or objects, of the other class **in the form of an attribute**.



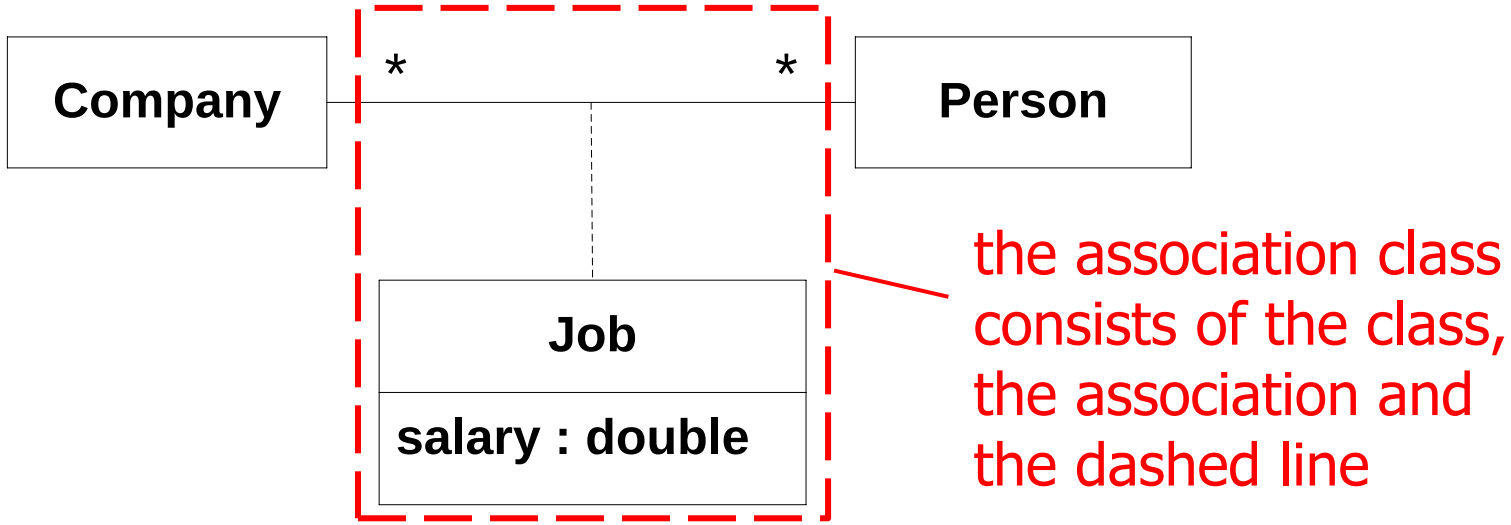
Association classes

- Sometimes association can have attributes and operations. Such association is called **association class.**
- A common problem in OO modeling is that when you have a many-to-many relationship between two classes. For example, consider the following association.

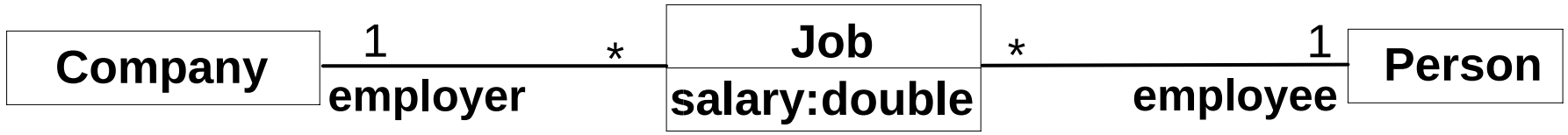
- What will happen if each Person has a salary with each Company they are employed by? Where should the salary be recorded – in the Person class or in the Company class?
- The answer is that the salary is a *property of the association itself.*

Association classes, cont.

- UML allows you to model this situation using an association class as shown:

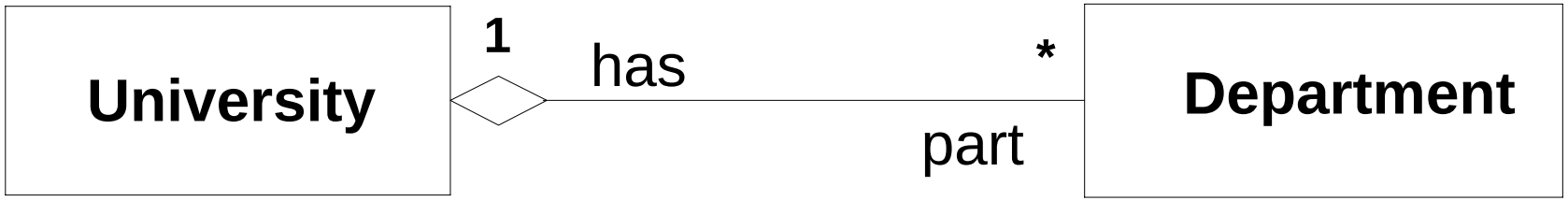


- Any association class can be transformed into a class and simple associations as follows:



Aggregation

- Aggregation means putting objects together to make a bigger object.
- Aggregations usually form a **whole-part hierarchy.**
- In UML, aggregation is treated as a constrained form of association, and is represented by using an empty diamond arrowhead next to the owning class, as shown:



- The parts can sometimes exist independently of the whole.
- It is possible to have shared ownership of the parts by several aggregates.
- Aggregation is transitive and asymmetric.

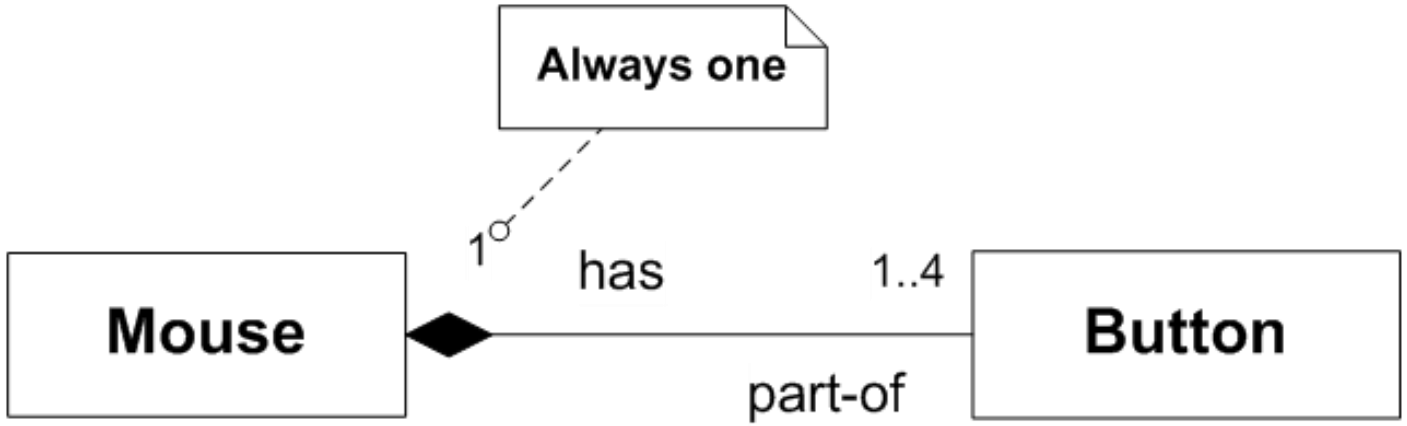
Discovering aggregation

- Discovered in parallel with discovery of associations
- The common test phrases
  - "has"
    - In the top-down explanation, the phrase is "has" (Book "has" Chapter, for example).
  - "is-part-of"
    - In the bottom-up interpretation, the phrase is "is part of" (Chapter "is part of" Book, for instance).

If the relationship is read aloud with these phrases and the sentence does not make sense, then the relationship is not an aggregation.

Composition

- **Composition** is a stronger form of aggregation and has similar (but more constrained) semantics.
- Like aggregation, it is **a whole-part** relationship and is both transitive and asymmetric.
- The key difference between aggregation and composition is that in composition the parts have no independent life outside of the whole.
- Furthermore, in composition each part belongs to one and only one whole.

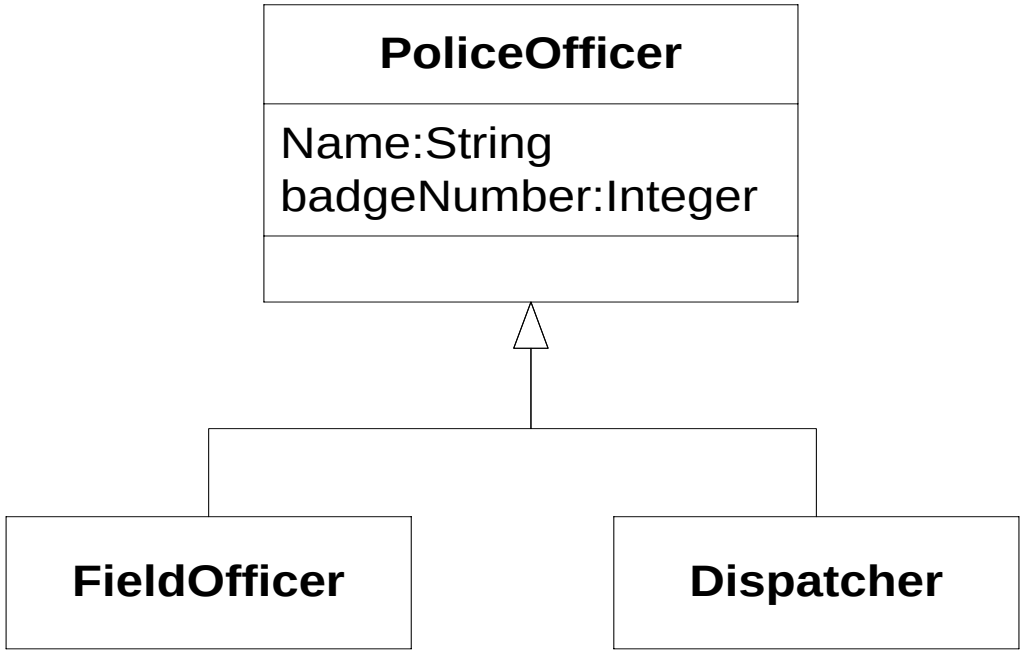


Composition ...

- We can summarize composition semantics as follows:
  - the parts belong to exactly one whole at a time
  - the whole has sole responsibility for the creation and destruction of all its parts
  - if the whole is destroyed, it must either destroy all its parts, or give responsibility for them over to some other object.
  - when the whole is created, all its parts should be created as well

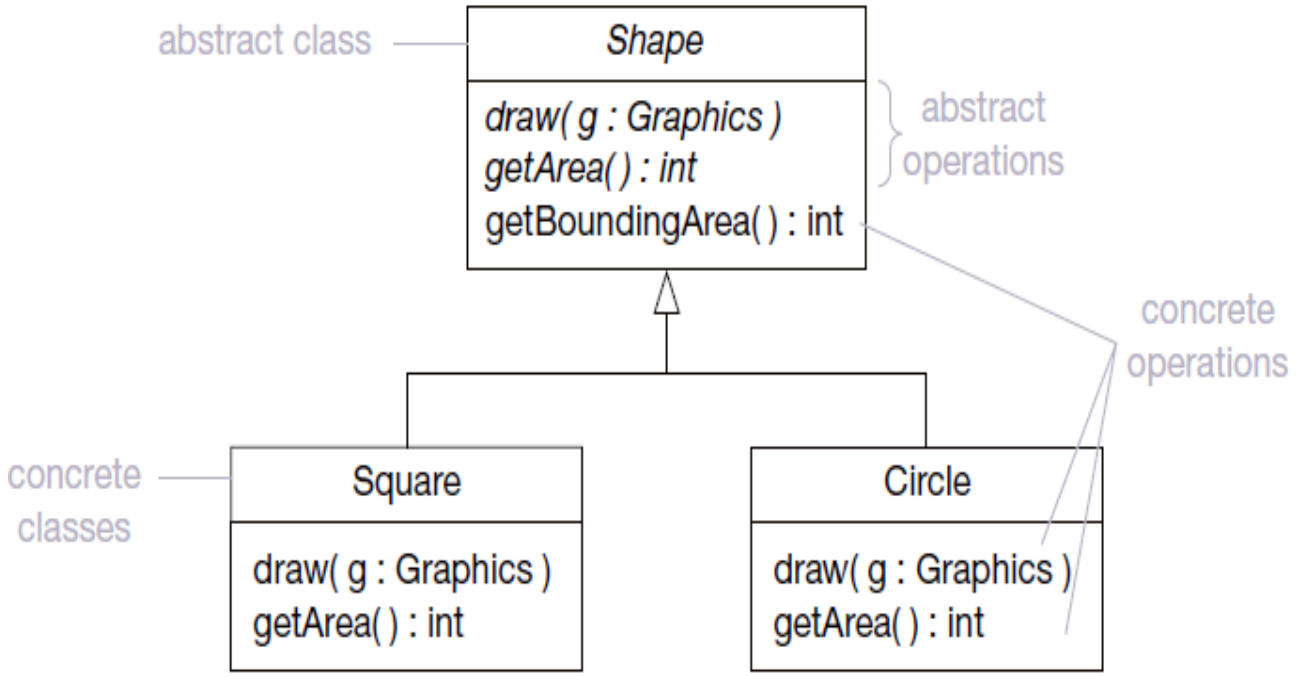
Generalization (aka. Inheritance)

- Generalization and inheritance are used to describe a class that is a type of another class.
- The terms **is a part of** and **is a kind of** are your tools to decide whether a relationship between two classes is **aggregation** or **generalization**.
- In UML, the generalization arrow is used to show that a class is a type of another class, as shown



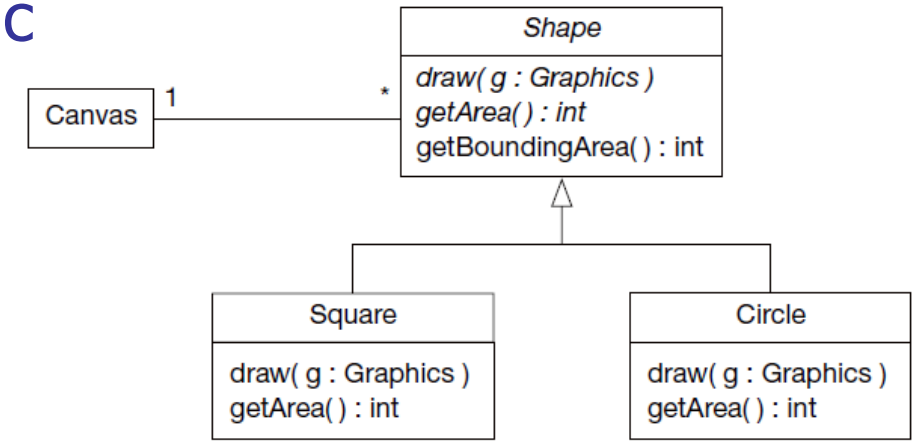
Abstract Classes

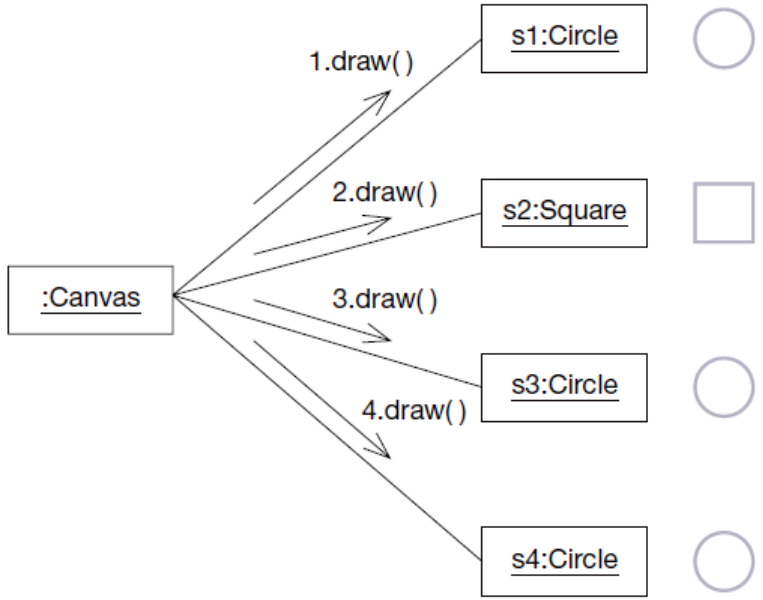
- The abstract class is an important modeling concept that makes parent general enough for reusability.
- An abstract class is a parent where not all its behaviors (methods) are implemented, and that will not have direct instance objects.
- Only subclasses of the abstract parent class can be instantiated.
- In UML, to indicate that the implementation of one method is to be left to subclasses by declaring those operations as abstract, write their signatures in **italics.**
- If any part of a class is declared abstract, then the class itself also needs to be declared as abstract by writing its name in italics, as shown:



Polymorphism

- Polymorphism means "many forms". A polymorphic operation is one that has many implementations.
- Consider the following example:
  - The class Canvas that maintains a collection of Shape objects.
  - This collection has a mix of instance of Square and Circle classes. (not an instance of Shape, why?)
  - From this object model, a :Canvas object holds a collection of four Shape objects s1, s2, s3 and s4 where s1, s3 and s4 are objects of class Circle, and s2 is an object of class Square.
  - What happens when the :Canvas object iterates over this collection, and sends each object the message draw( )?





Overloading Methods

- Multiple methods within the same class can have the same name
- Java distinguishes them by noting the parameters
  - Different numbers of parameters
  - Different types of parameters
- This is called the signature of the method

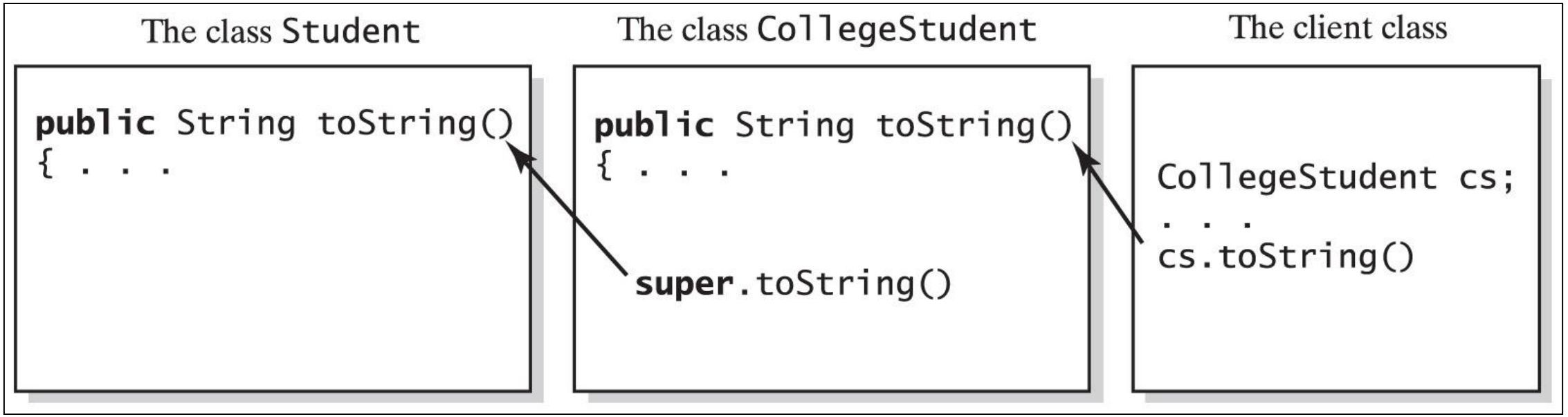
Overloading a Method

- When the derived class method has
  - The same name
  - The same return type … but …
  - Different number or type of parameters
- Then the derived class has available
  - The derived class method … and
  - The base class method with the same name
- Java distinguishes between the two methods due to the different parameters

Overriding Methods

- When a derived class defines a method with the same signature as in base class
  - Same name
  - Same return type
  - Same number, types of parameters
- Objects of the derived class that invoke the method will use the definition from the derived class
- It is possible to use **`super`** in the derived class to call an overridden method of the base class

Overriding Methods



The method **`toString`** in **`CollegeStudent`** overrides the method **`toString`** in **`Student`**

Overriding vs. Overloading

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
  }
}

class B {
  public void p(int i) {
  }
}

class A extends B {
  // This method overrides the method in B
  public void p(int i) {
    System.out.println(i);
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    A a = new A();
    a.p(10);
  }
}

class B {
  public void p(int i) {
  }
}

class A extends B {
  // This method overloads the method in B
  public void p(double i) {
    System.out.println(i);
  }
}
```

Processor.java

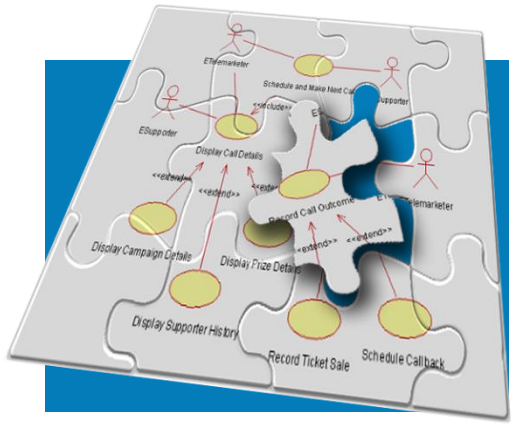
```java
package com.journaldev.examples;

import java.util.Arrays;

public class Processor {

    public void process(int i, int j) {
        System.out.printf("Processing two integers:%d, %d", i, j);
    }

    public void process(int[] ints) {
        System.out.println("Adding integer array:" + Arrays.toString(ints));
    }

    public void process(Object[] objs) {
        System.out.println("Adding integer array:" + Arrays.toString(objs));
    }
}

class MathProcessor extends Processor {

    @Override
    public void process(int i, int j) {
        System.out.println("Sum of integers is " + (i + j));
    }

    @Override
    public void process(int[] ints) {
        int sum = 0;
        for (int i : ints) {
            sum += i;
        }
        System.out.println("Sum of integer array elements is " + sum);
    }

}
```

Overloading: Same Method Name but different parameters in the same class

Overriding: Same Method Signature in both superclass and child class

48

Overriding	Overloading
Implements "runtime polymorphism"	Implements "compile time polymorphism"
The method call is determined at runtime based on the object type	The method call is determined at compile time
Occurs between superclass and subclass	Occurs between the methods in the same class
Have the same signature (name and method arguments)	Have the same name, but the parameters are different
On error, the effect will be visible at runtime	On error, it can be caught at compile time



Analysis Activities: From Use Cases to Objects

Activities during Object Modeling

Main goal: Find the important abstractions

- Steps during object modeling
  1. Class identification
  2. Find the attributes
  3. Find the methods
  4. Find the associations between classes
- Order of steps
  - Goal: get the desired abstractions
  - Order of steps secondary, only a heuristic
- What happens if we find the wrong abstractions?
  - We iterate and revise the model

Class Identification

- **Approaches**
  - **Application domain approach**
    - Ask application domain experts to identify relevant abstractions
  - **Syntactic approach**
    - Start with use cases
    - Analyze the text to identify the objects
    - Extract participating objects from flow of events
  - **Design patterns approach**
    - Use reusable design patterns
  - **Component-based approach**
    - Identify existing solution classes.

Finding Participating Objects in Use Cases

- Pick a use case and look at flow of events
- Do a textual analysis (noun-verb analysis)
  - Nouns are candidates for objects/classes
  - Verbs are candidates for operations
  - This is also called Abbott's Technique
- Abbott's Technique can be used to
  - Identify Entity Object
  - Identify Control Object
  - Identify Boundary Object

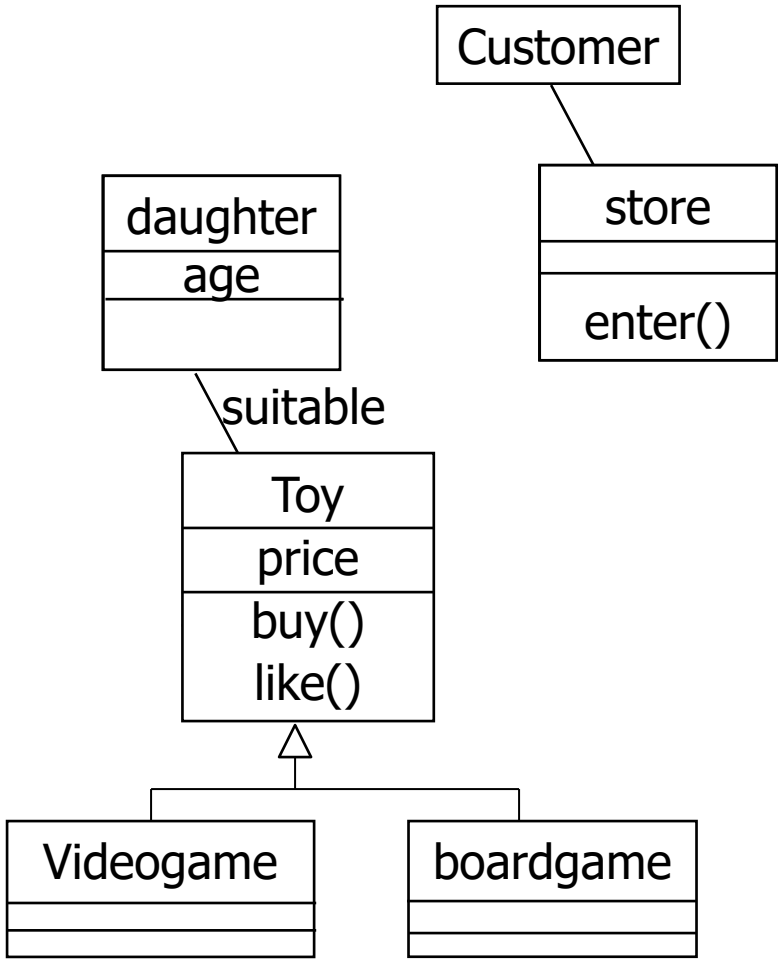
Example for using Abbott's Technique

- **Flow of Events:**
  - The customer enters the store to buy a toy.
  - It has to be a toy that his daughter likes, and it must cost less than $50.
  - He tries a videogame, which uses a data glove and a head-mounted display. He likes it.
  - An assistant helps him.
  - The suitability of the game depends on the age of the child.
  - His daughter is only 3 years old.
  - The assistant recommends another type of toy, namely the board game "Monopoly".

Mapping parts of speech to UML model components (Abbot's Technique)

Example	Part of speech	UML model component
"Monopoly"	proper noun	object
Toy	improper noun	class
Likes, recommend	doing verb	operation
is a	being verb	inheritance
has an	having verb	aggregation
must be	modal verb	constraint
enter	transitive verb	operation
depends on	intransitive verb	constraint, class, association

Generating a Class Diagram from Flow of Events



- The customer enters the store to buy a toy. It has to be a toy that his daughter likes and it must cost less than $50. He tries a videogame, which uses a data glove and a head-mounted display. He likes it.

An assistant helps him. The suitability of the game depends on the age of the child. His daughter is only 3 years old. The assistant recommends another type of toy, namely a boardgame. The customer buy the game and leaves the store

Filtering the candidate classes list

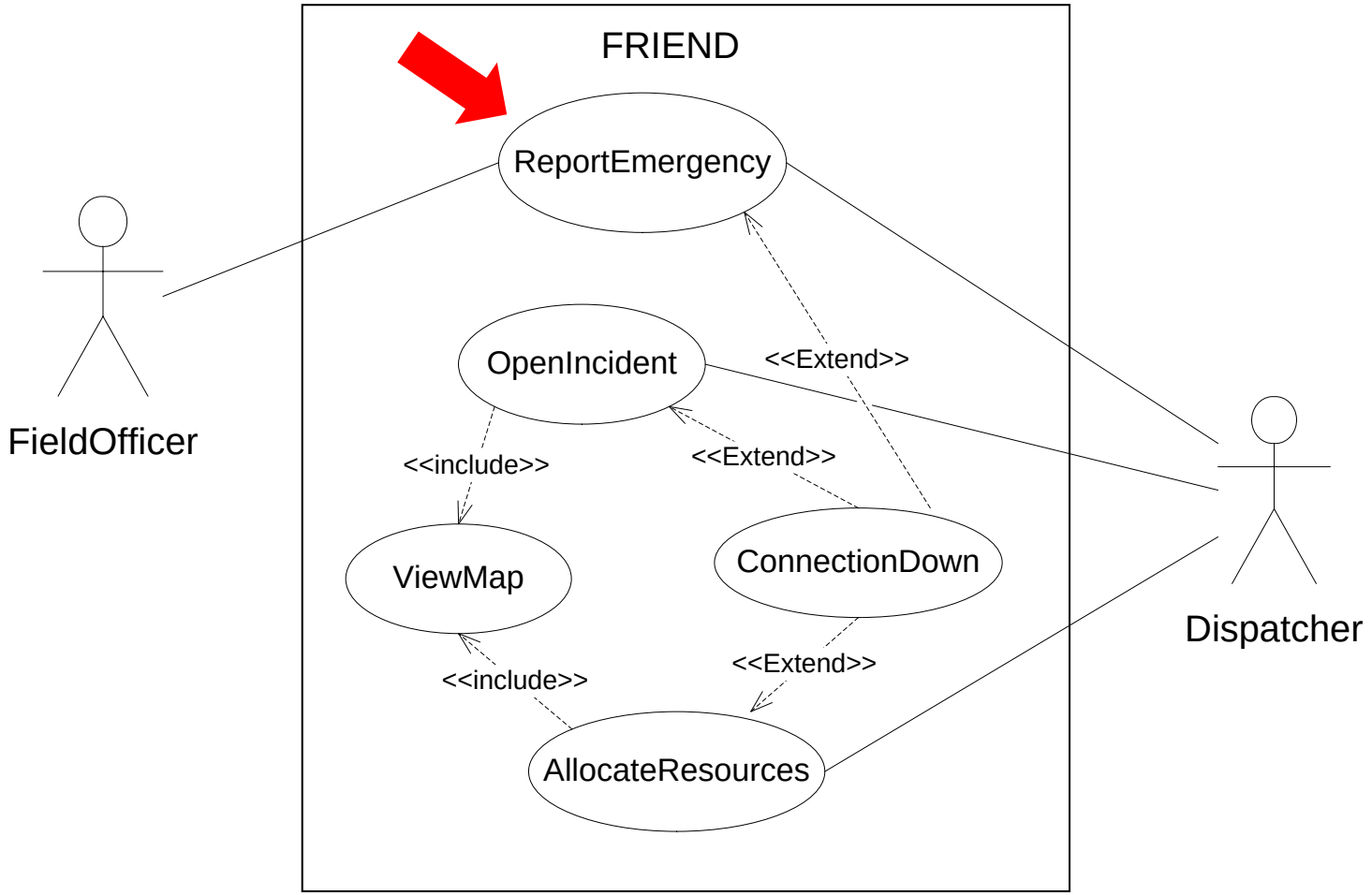
- Abbott's Technique may result many more nouns than relevant classes
  - Many nouns correspond to attributes or synonyms for other nouns.
  - Sorting through all the nouns for a large requirements specification is a time-consuming activity.
- The following heuristics can be used in conjunction with Abbott's Technique to determine whether the identified noun is a candidate class or not:
  - **Is it a container for data?**
  - **Does it have separate attributes that will take on different values?**
  - **Would it have many instance objects?**
  - **Is it in the scope of the application domain?**

Identifying Entity Objects

- All classes we have used so far define 'business objects'
  - Called entity classes, such as, customer, store, videogame, etc.
  - Represent persistent database objects
- Eventually, these are the classes that define the database model for an application domain
- This means there is a map between entity classes in the application domain and the corresponding tables in a relational database

From use case (ReportEmergency) to objects

- Let us focus only on ReportEmergency use case to find the corresponding object classes



ReportEmergency, UC Description

- Let us now examine the **ReportEmergency** use case to identify **the entity classes**

Use case name	ReportEmergency
Entry condition	The FieldOfficer is logged into FRIEND
Flow of events	1. The FieldOfficer activates the "Report Emergency" function of her terminal. 2. FRIEND responds by presenting a form to the FieldOfficer. The form includes an emergency type menu (general emergency, fire, transportation), a location, incident description, resource request, and hazardous material fields. 3. The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form by pressing the " Send Report" button, at which point, the Dispatcher is notified. 4. FRIEND receives the form and notifies the Dispatcher. 5. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response by allocating resources to the incident (with the AllocateResources use case) and acknowledges the FieldOfficer. 6. FRIEND displays the acknowledgement and selected response to the FieldOfficer.
Exit condition	• The FieldOfficer receives an acknowledgment and the selected response.

Entity classes for the ReportEmergency use case.

Class name	Description
Dispatcher	Police officer who manages Incidents, (for example opens, documents, and closes Incidents in response to Emergency Reports) and also communicates with FieldOfficers.
EmergencyReport	Initial report about an Incident from a FieldOfficer to a Dispatcher.
FieldOfficer	Police or fire officer on duty. A FieldOfficer can be allocated to, at most, one Incident at a time. FieldOfficers are identified by badge numbers.
Incident	Situation requiring attention from a FieldOfficer. An Incident is composed of a description, a response, a status (open, closed, documented), a location, and a number of FieldOfficers.

Identifying Boundary Objects

- Boundary objects represent **the system interface** with the actors.
- The boundary object **collects** the information from the actor and translates it into a form that can be used by both entity and control objects.
- The following slide present some heuristics that can be used in conjunction with Abbott's Technique to determine the boundary classes.

Heuristics for Identifying Boundary Classes

- Select a noun that can be used to
  - Identify user interface controls that the user needs to initiate the use case (e.g., ReportEmergencyButton).
  - Identify forms the users needs to enter data into the system (e.g., EmergencyReportForm).
  - Identify notices and messages the system uses to respond to the user (e.g., AcknowledgmentNotice).
  - Always use the end user's terms for describing interfaces; do not use terms from the solution or implementation domains.

ReportEmergency, UC Description

- Let's examine the **ReportEmergency** use case to identify **the boundary classes**

Use case name	ReportEmergency
Entry condition	The FieldOfficer is logged into FRIEND
Flow of events	1. The FieldOfficer activates the "Report Emergency" function of her terminal. 2. FRIEND responds by presenting a form to the FieldOfficer. The form includes an emergency type menu (general emergency, fire, transportation), a location, incident description, resource request, and hazardous material fields. 3. The FieldOfficer fills the form by selecting the emergency level, type, location, and brief description of the situation. The FieldOfficer also describes possible responses to the emergency situation. Once the form is completed, the FieldOfficer submits the form by pressing the " Send Report" button, at which point, the spatcher is notified. 4. FRIEND receives the form and notifies the Dispatcher. 5. The Dispatcher reviews the submitted information and creates an Incident in the database by invoking the OpenIncident use case. The Dispatcher selects a response by allocating resou (with the AllocateResources use case) and acknowledges the FieldOfficer. 6. FRIEND displays the acknowledgement and selected response to the FieldOfficer.
Exit condition	• The FieldOfficer receives an acknowledgment and the selected response.

Dispatcher Station

Using incident form

Boundary classes for the ReportEmergency use case.

Class name	Description
AcknowledgmentNotice	Notice used for displaying the Dispatcher's acknowledgment to the FieldOfficer.
DispatcherStation	Computer used by the Dispatcher.
ReportEmergencyButton	Button used by a FieldOfficer to initiate the ReportEmergency use case.
EmergencyReportForm	This form is presented to the FieldOfficer on the FieldOfficerStation when the " Report Emergency" function is selected.
FieldOfficerStation	Mobile computer used by the FieldOfficer.
IncidentForm	Form used by the Dispatcher to create Incidents.

Identifying Control Objects

- Control objects are responsible for collecting information from the boundary objects and dispatching it to entity objects.
- Control objects usually do not have a concrete counterpart in the real world.
- **Typically, a control object is created at the beginning of a use case and is diminished at its end.**

Identifying Control Objects ...

- **Heuristics for identifying control objects**
  - Identify one control object per use case.
  - Identify one control object per actor in the use case.
  - The life span of a control object should cover the life of all use case events.
  - If it is difficult to identify the beginning and the end of a control object activation, the corresponding use case probably does not have well-defined entry and exit conditions.

Control classes for the ReportEmergency use case

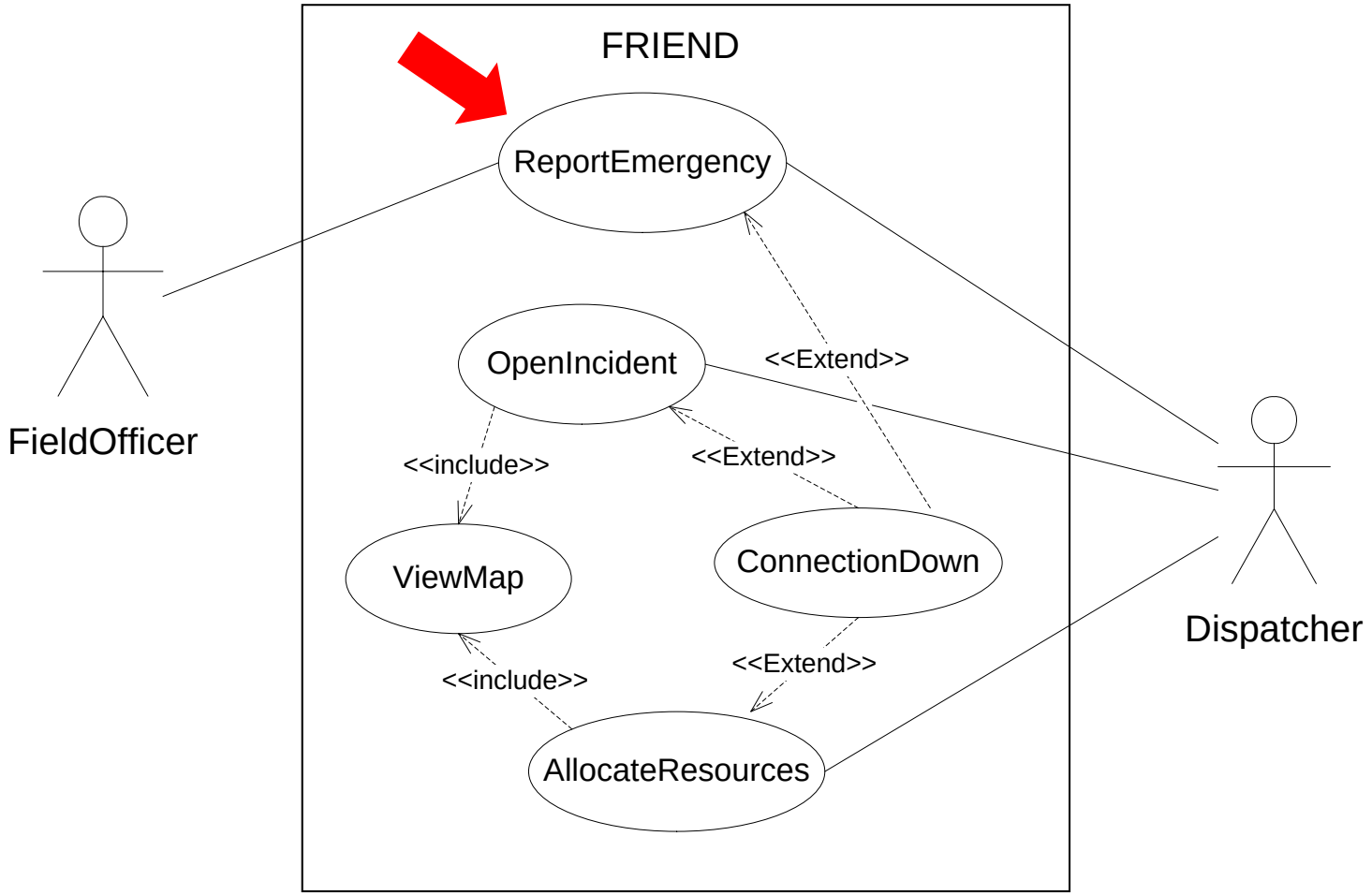
- Initially, we model the control flow of the ReportEmergency use case with a control object for each actor:
  - **ReportEmergencyControl** for the FieldOfficer and
  - **ManageEmergencyControl** for the Dispatcher

Control classes for the ReportEmergency use case ...

Class name	Description
ReportEmergencyControl	Is created when the FieldOfficer initiate the use case. It then creates an EmergencyReportForm and presents it to the FieldOfficer. Collects the information from the form, creates an EmergencyReport, and forwards it to the Dispatcher. When the acknowledgment is received, it creates an AcknowledgmentNotice and displays it to the FieldOfficer.
ManageEmergencyControl	Is created when an EmergencyReport is received. It then creates an IncidentForm and displays it to the Dispatcher. Once the IncidentForm information is filled, it forwards the acknowledgment to the FieldOfficerStation.

From use case (ReportEmergency) to objects

- Let us focus only on ReportEmergency use case to find the corresponding object classes



ReportEmergency Classes

Dispatcher

EmergencyReport

FieldOfficer

Incident

AcknowledgmentNotice

DispatcherStation

ReportEmergencyButton

EmergencyReportForm

FieldOfficerStation

IncidentForm

ReportEmergencyControl

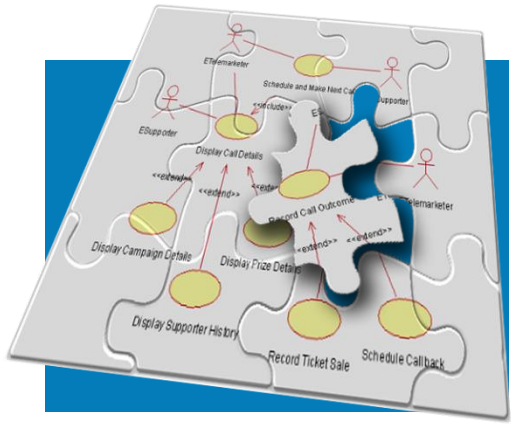
ManageEmergencyControl

Guidelines for Class Discovery

- Each class must have a clear statement of purpose in the system.
- Each class is a template description for a set of objects.
- Each entity class must house a set of attributes.
  - It is a good idea to determine identifying attribute(s) (keys) to assist in reasoning about the class cardinality
- Each class should be distinguished from an attribute.
  - color of a car is normally perceived as an attribute of class car. However, in a paint factory color is definitely a class with its own attributes (brightness, saturation, transparency & so on).
- Each class should house a set of operations (what does class do?)

Steps in Generating Class Diagrams

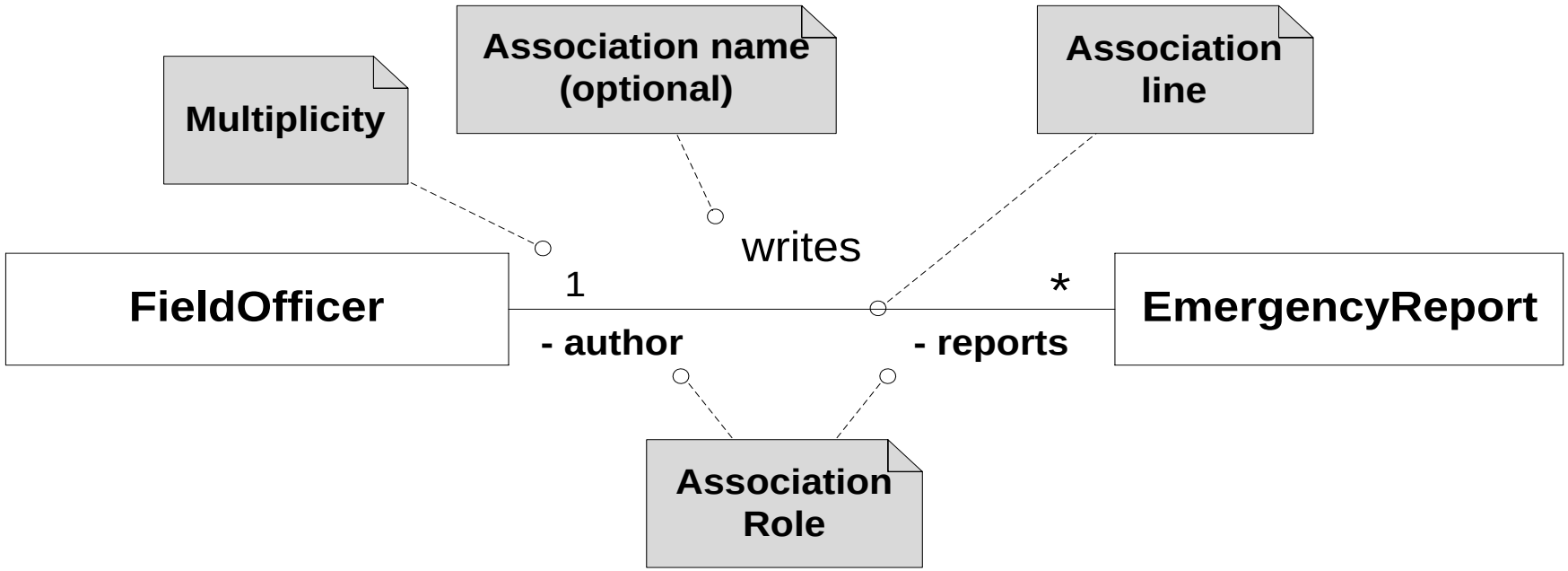
- ✓ Class identification (textual analysis, domain expert)
- ✓ Identification of attributes and operations (sometimes before the classes are found!)
- ✓ Identification of associations between classes
- ✓ Identification of multiplicities
- ✓ Identification of roles
- ✓ Identification of inheritance



Identifying Associations, Aggregations, and Attributes

Identifying Associations between Classes

- Associations have several properties:
  - **A name** to describe the association between the two classes. Association names are optional and need not be unique globally.
  - **A role** at each end, identifying the role of each class with respect to the associations (e. g., FieldOfficer has a collection (an array) called reports of type EmergencyReport) while EmergencyReport has an attribute author of type FieldOfficer.
  - **A multiplicity** at each end, identifying possible number of instances

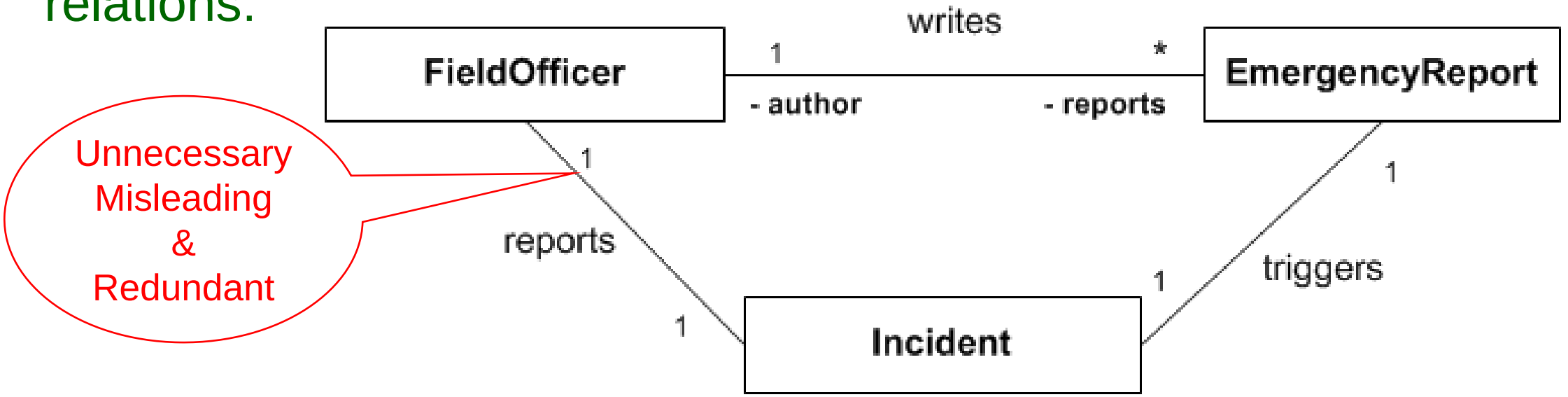


Heuristics for Identifying Associations

- Initially, the associations between **entity** objects are the most important, as they reveal more information about the application domain.
- Associations can be identified by examining **verbs** and **verb phrases** (e.g., has, is part of, manages, reports to, is triggered by, is contained in, talks to, includes).
- In addition, the following heuristics can be used
  - Examine verb phrases.
  - Name associations and roles precisely.
  - Use qualifiers as often as possible to identify namespaces and key attributes.
  - Eliminate any association that can be derived from other associations.
  - Do not worry about multiplicity until the set of associations is stable.
  - Too many associations make a model unreadable

Filtering Verb Phrases for Association

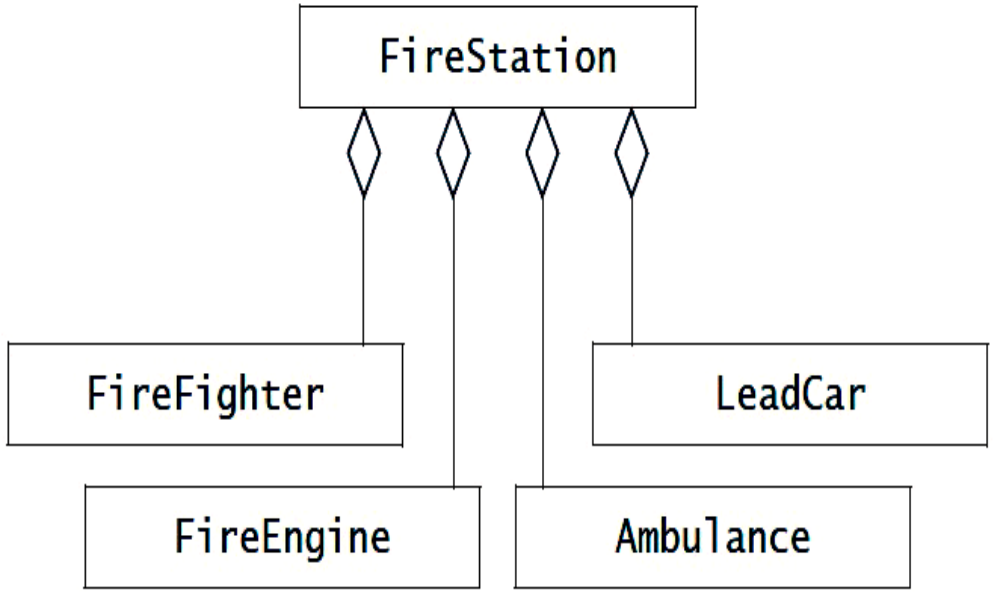
- If we consider all associations identified after examining verb phrases, our model will initially include too many associations.
  - For example, in **ReportEmergency** use case, we identify the following relations:

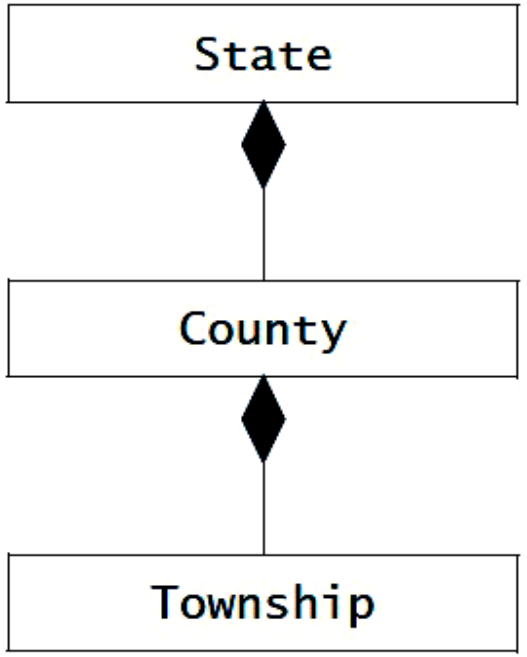


- Given that we already identified an association between the FieldOfficer and the EmergencyReport, the association between Incident and FieldOfficer is not necessary.
- Adding unnecessary associations complicates the model, leading to incomprehensible models and redundant information.

Identifying Aggregates

- Aggregations are special types of associations denoting a whole–part relationship.
- For examples, A FireStation includes FireFighters, FireEngines, Ambulances, and a LeadCar.
- A State is composed of many counties, which in turn is composed of many Townships.
- Aggregations are often used in the user interface to help the user browse through many instances.
  - For example, FRIEND could offer a tree representation for Dispatchers to find Counties within a State or Townships with a specific County.





Identifying Attributes

- **Attributes are properties**. Attribute represents **a single property** of an object. (a partial aspect of the entire object). For examples:
  - The name of an Advertiser is an attribute that identifies an Advertiser. However, it does not include other relevant information like Advertiser account balance, or the type of his/her advertisement.
  - An EmergencyReport, has an emergency type, a location, and a description property. These are good class attribute candidates.
- Most entity objects have an identifying characteristic **used by the actors to access them**. These characteristics are good candidate for class attributes, e.g.
  - FieldOfficers and Dispatchers have a badge number.
  - Incidents and Reports are assigned numbers and are archived by date.
  - Note that, each FieldOfficer has a social security number that is not relevant to the emergency information system.

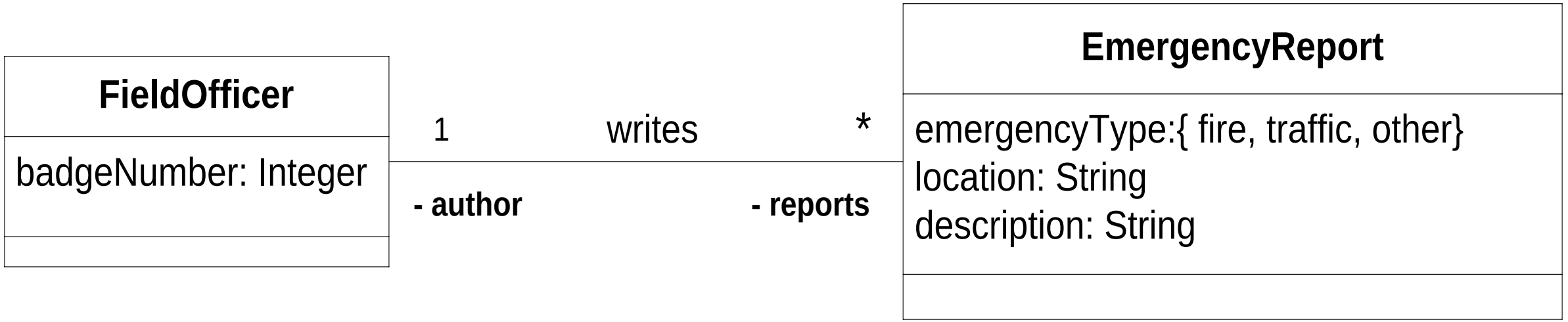
Identifying Attributes ...

- Attributes have:
  - A name identifying them within an object.
  - A brief description.
  - A type describing the legal values it can take

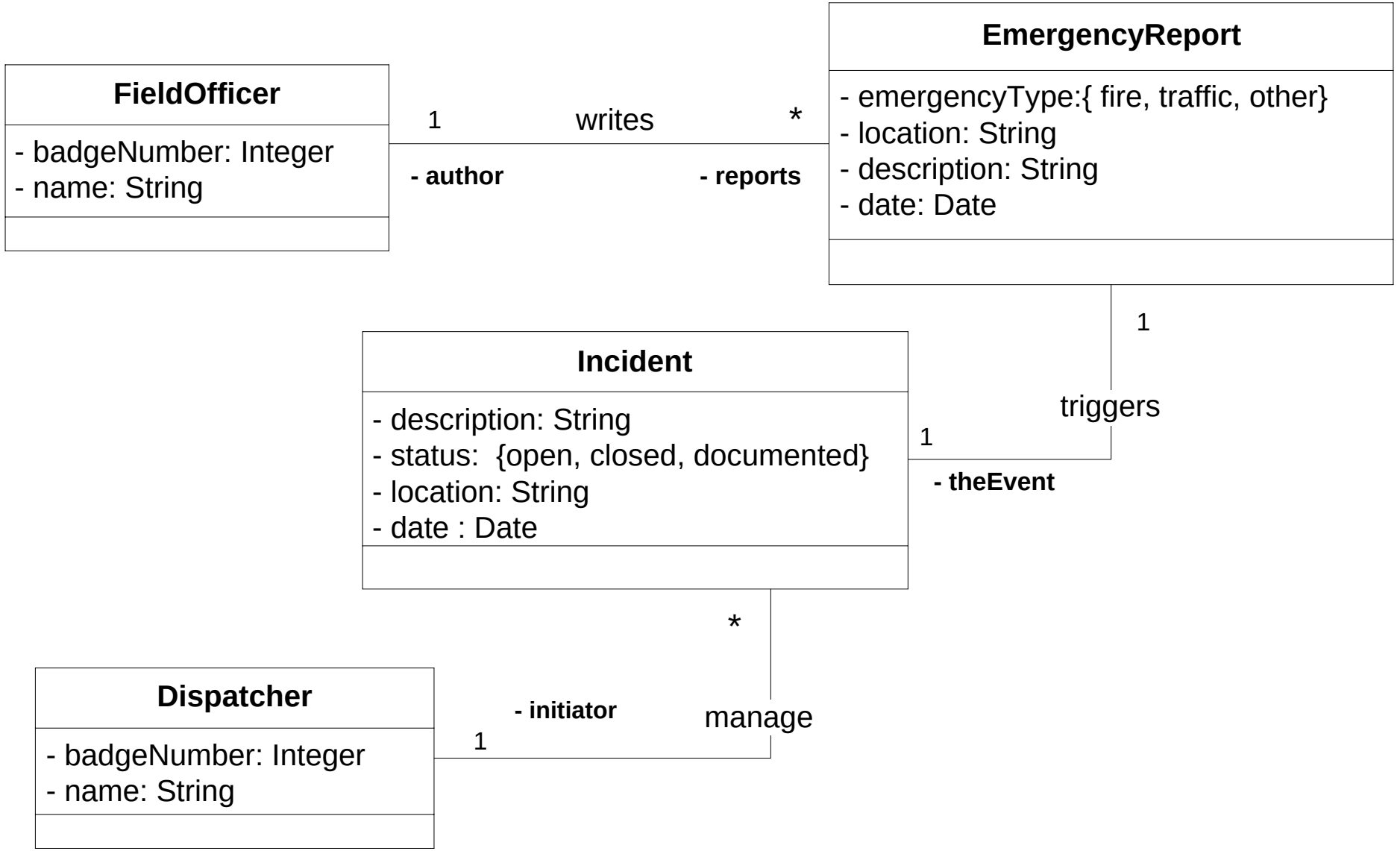
- Heuristics for identifying attributes:
  - Examine possessive phrases.
  - Represent stored state as an attribute of the entity object.
  - Describe each attribute.
  - Do not represent an attribute as an object; use an association instead (see next slide).
  - **Do not waste time describing fine details before the object structure is stable.**

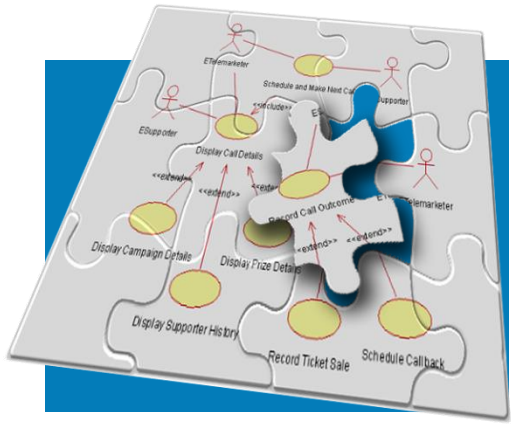
Simple Attributes vs. Associations

- **Attributes have simple types (primitive data types)**. such as
  - A number (e. g., maximum number of Tournaments),
  - A string (e. g., the name of an Advertiser),
  - Dates (e. g., the application start and end date of a Tournament).
- **Non primitive data type are associations**, with single/collection of data.
  - Lists, groups, tables, and sets of non primitive data types are represented by associations.
  - For example, every EmergencyReport has an author that is represented by an association to the FieldOfficer class.



FRIEND – Class diagram





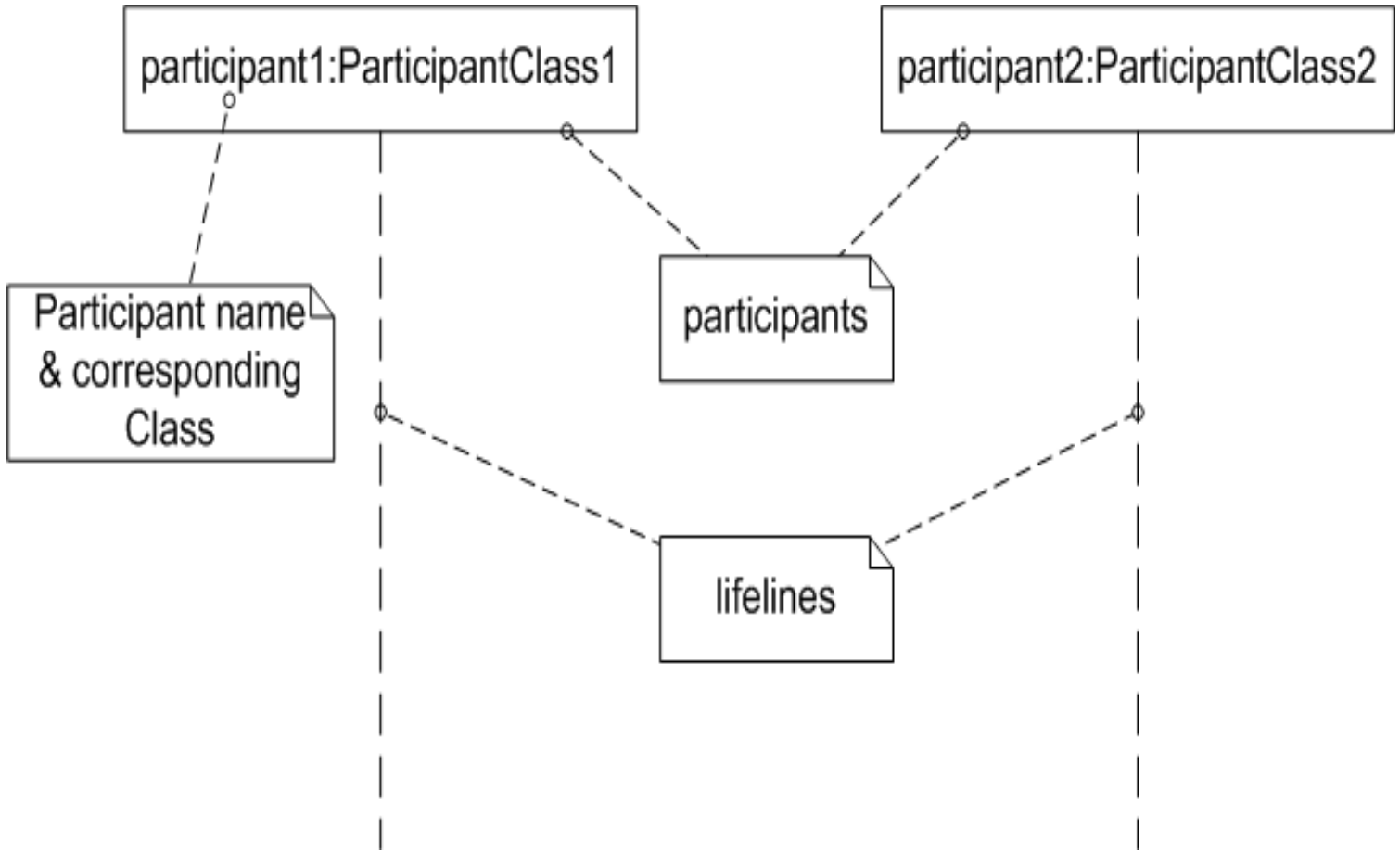
Modeling Ordered Interactions: Sequence Diagrams

Sequence Diagrams

- We use use-case diagrams to describe what the system is able to do.
- We use class diagrams to describe the structure of the system.
- Yet we did not try to describe **how** the system is going to do its job. This is where **sequence diagrams** come into play.
- Sequence diagrams are all about capturing **the order of interactions** between parts of your system, which is represented by use case and class diagrams.

Participants in a Sequence Diagram

- A sequence diagram is made up of a collection of **participants**
  - the parts of your system that interact with each other during a time sequence
- Participants are always arranged horizontally with no two participants overlapping each other
- Each participant has a corresponding lifeline running down the page



Participant Names

- Participants on a sequence diagram can be named in number of different ways, picking elements from the standard format:

name [selector] : class_name ref decomposition

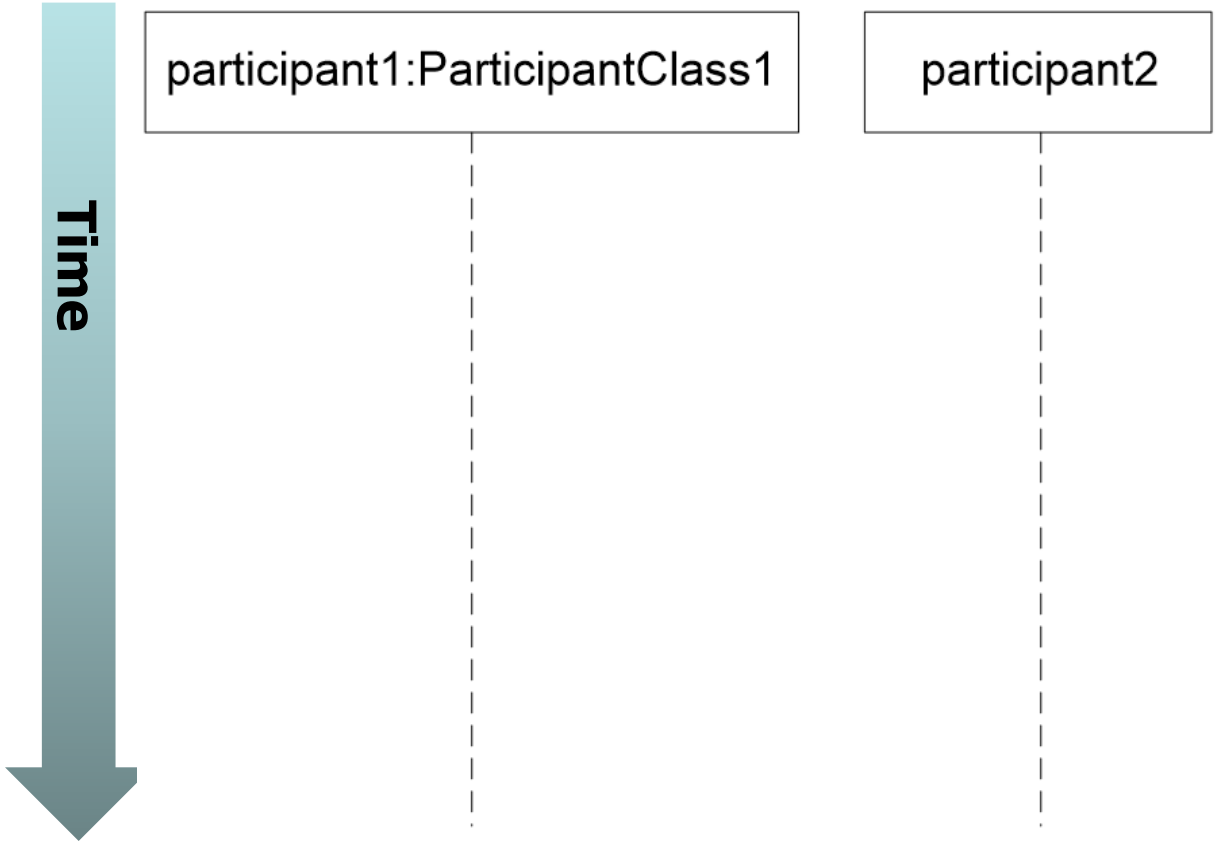
```
Bob: FieldOfficer
reports [2]: EmergencyReports
Bob: FieldOfficer ref policeInteraction
```

Participant Names

Example participant name	Description
Bob	A Participant is named Bob, but at this point in time the participant has not been assigned a class.
: FieldOfficer	The class of the participant is FieldOfficer, but the participant currently does not have its own name.
Bob: FieldOfficer	There is a participant has a name of Bob of the class FieldOfficer.
reports [2] : EmergencyReports	There is a participant that is accessed within an array at element 2, and it is of the class EmergencyReports.
: FieldOfficer ref policeInteraction	The participant is of the class FieldOfficer, and there is another sequence diagram called policeInteraction that shows how the participant works internally. Used when we have Fragment Types "later in this unit".

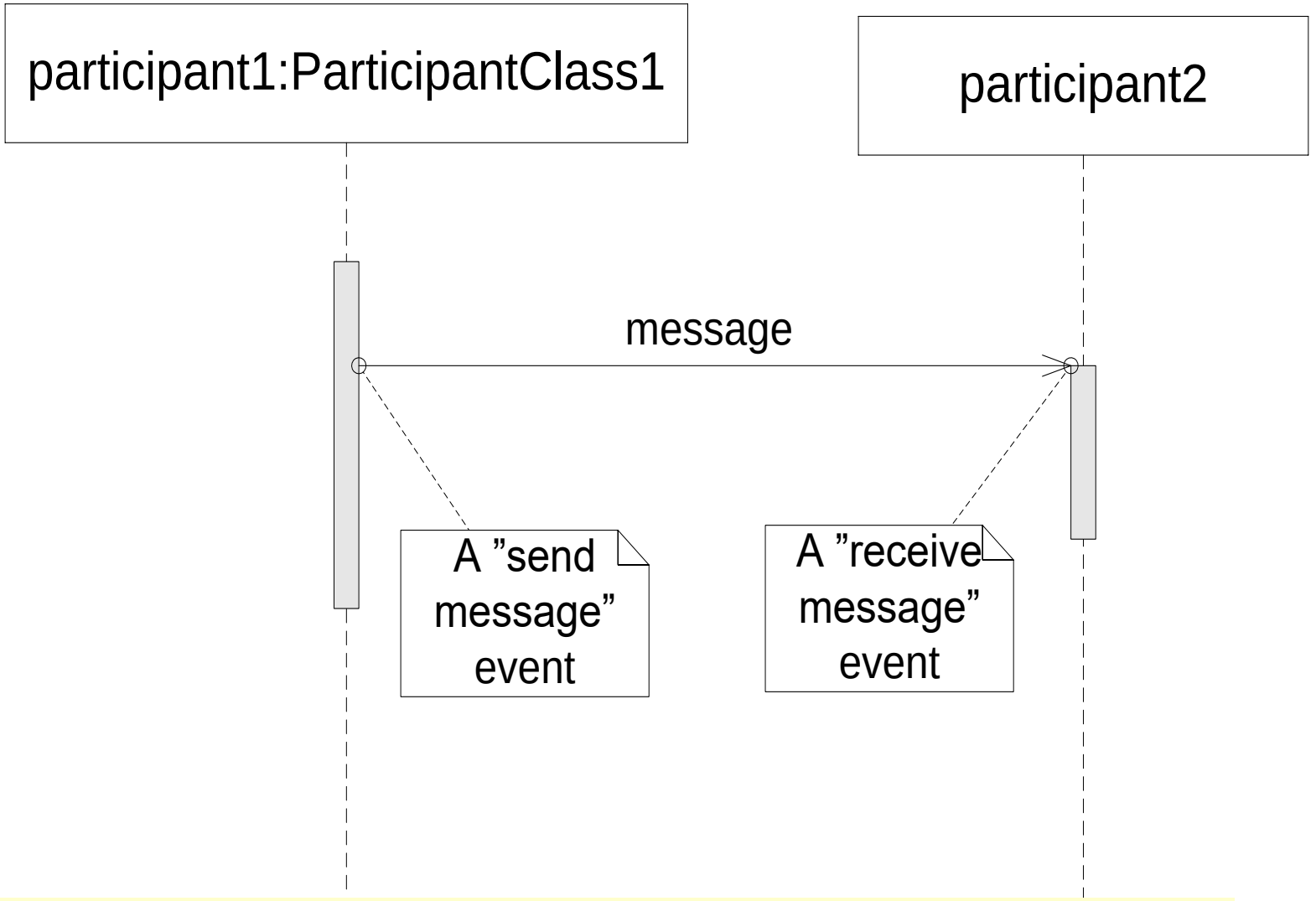
Time

- A sequence diagram describes the order in which the interactions take place, so time is an important factor.
- Picture shows how UML relates time to a sequence diagram.
- Time on a sequence diagram starts at the top of the page and then progresses down the page.
- Time on a sequence diagram is all about ordering, not duration.



Events, Signals, and Messages

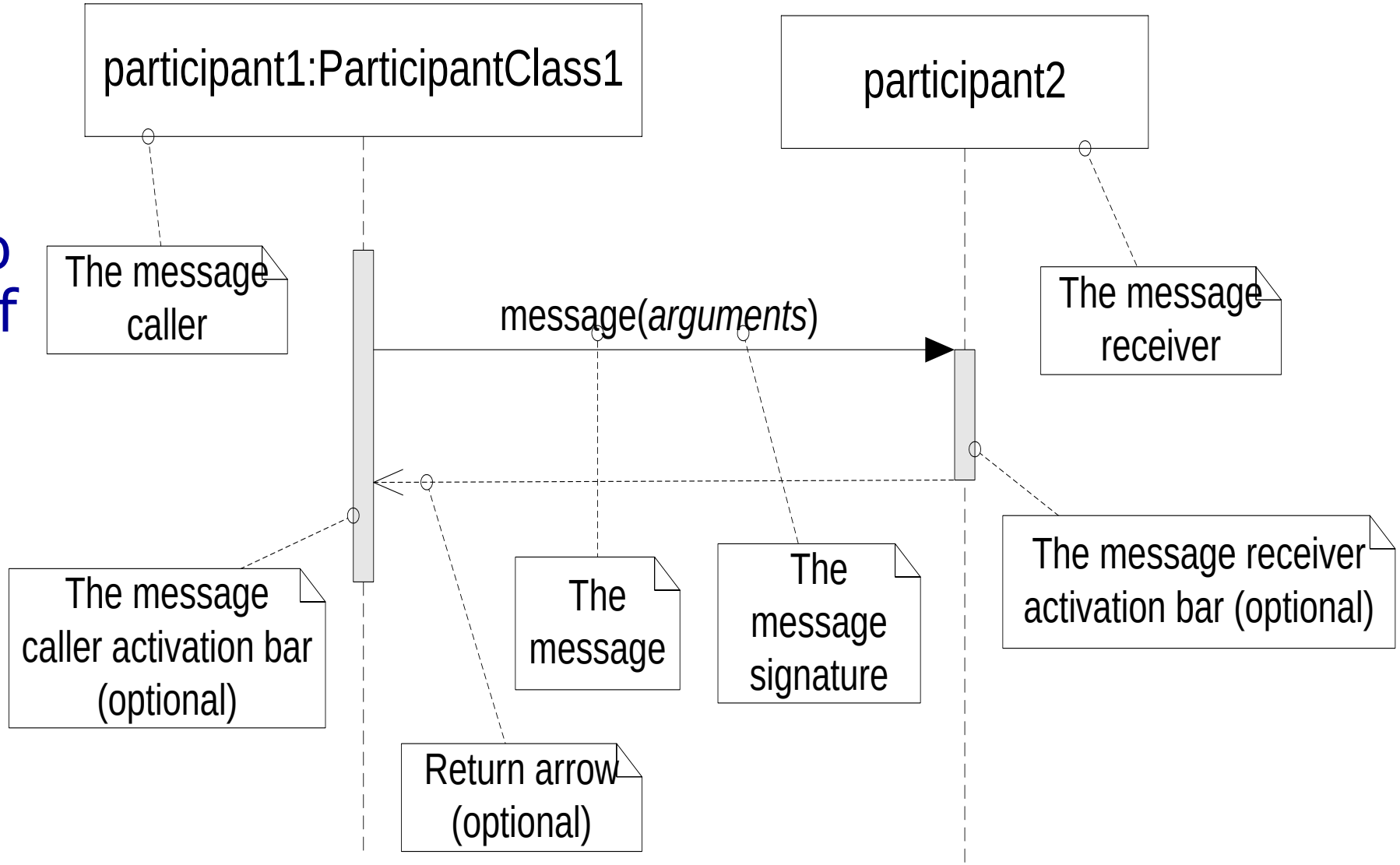
- An event is any point in an interaction where something occurs
  - This something, for example, can be a message or signal when sent or received



An event is defined as a message of notification of a change in system state that is of interest.

Interactions on a sequence diagram

- **An interaction** in a sequence diagram occurs when one participant decides to **send a message** to another participant, as shown.
- Messages can flow **in any direction makes sense**, LtR, RtL, or even back to message Caller itself
- Think of a message as an event that is passed from a Message Caller to get the Message Receiver to do something.



Message Signatures

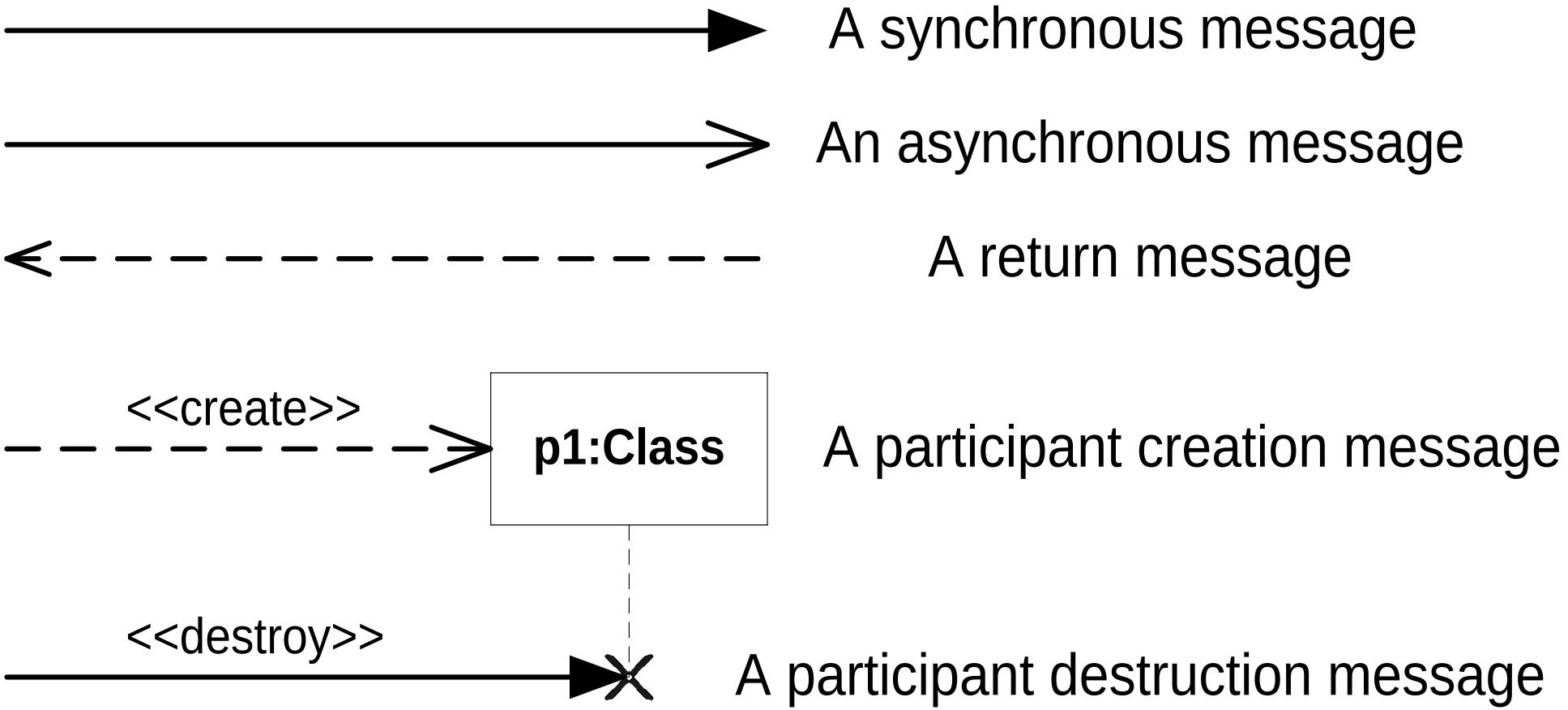
- A message arrow comes with a description, or signature. UML has the format for a message signature.

attribute = signal_or_message_name (arguments) : return_type

Example message signature	Description
doSomething( )	The message's name is doSomething, but no further information is known about it.
doSomething(number1 : Number, number2 : Number)	The message's name is doSomething, and it takes two arguments, number1 and number2, which are both of class Number.
doSomething( ) : ReturnClass	The message's name is doSomething; it takes no arguments and returns an object of class ReturnClass
myVar = doSomething( ) : ReturnClass	The message's name is doSomething; it takes no arguments, and it returns an object of class ReturnClass that is assigned to the myVar attribute of the message caller.

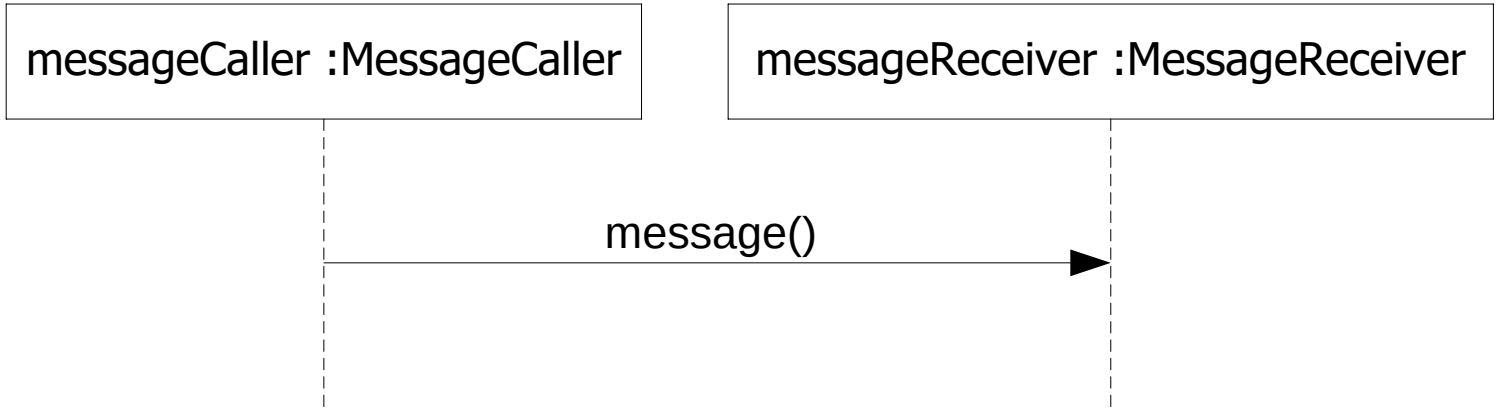
Message Arrows

- The type of arrowhead on a message defines the type of message.
  - For example, to distinguish between **a synchronous message**, and an **asynchronous message**.
- There are 5 main types of message arrow for use on sequence diagram

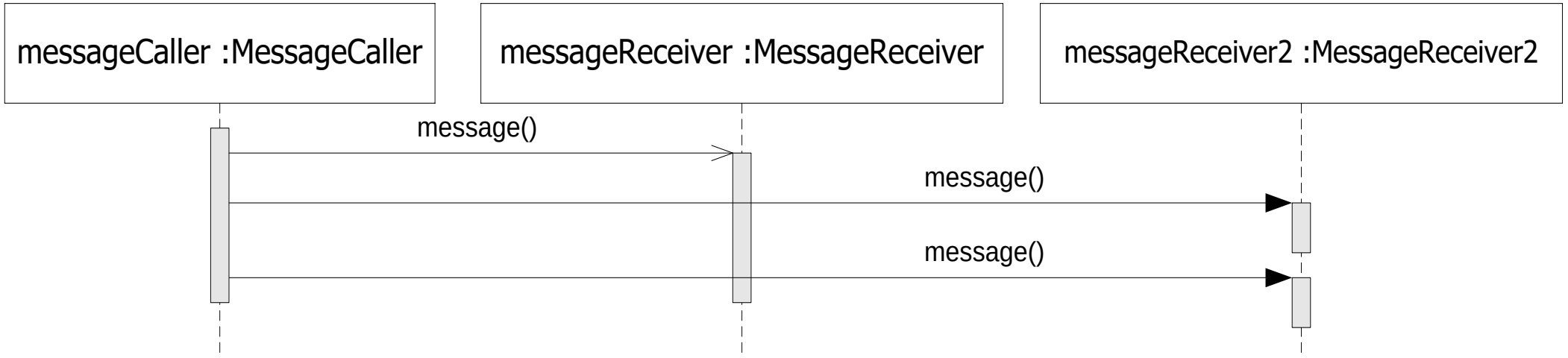


A Synchronous & An asynchronous Messages

- A **synchronous** message is invoked when the messageCaller waits for the messageReceiver to return from the message invocation, as shown:

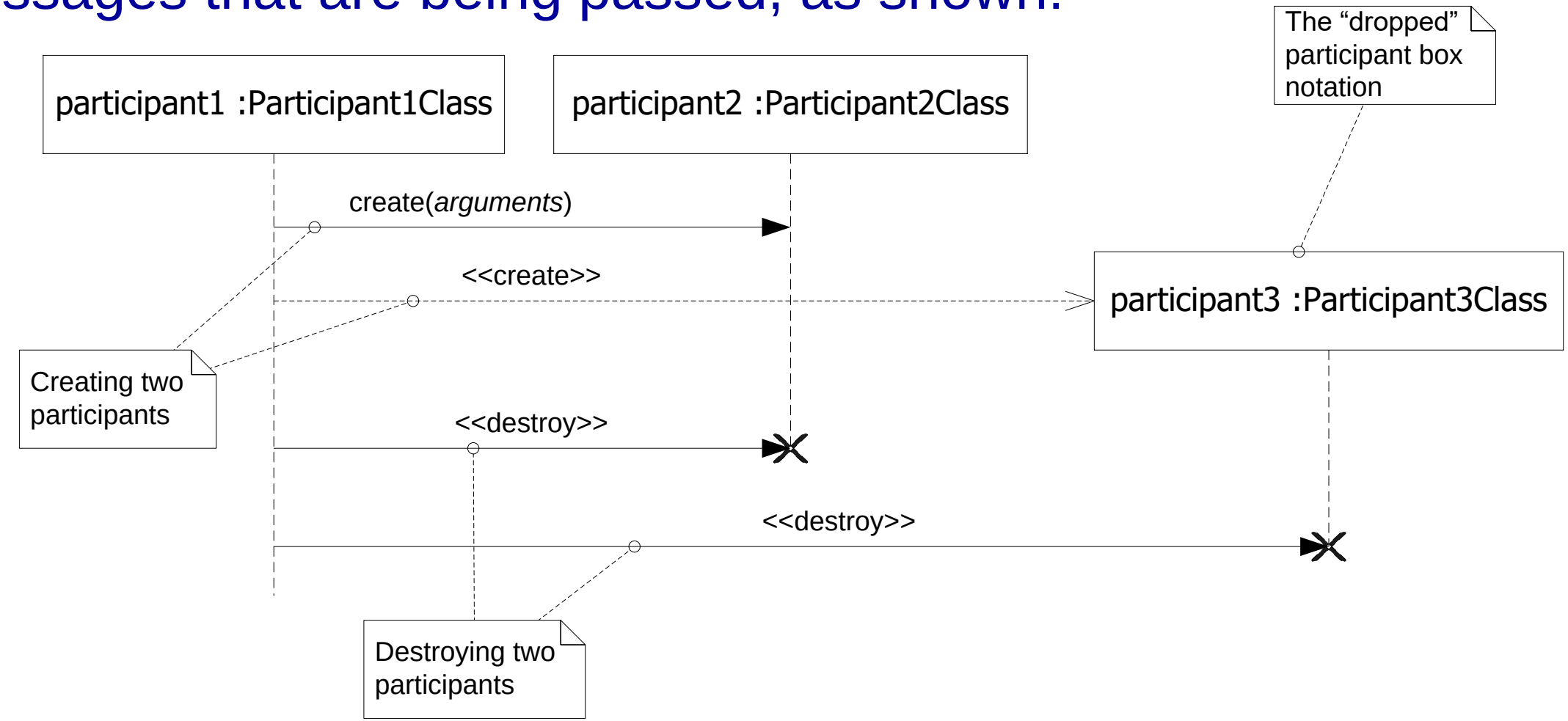


- An **asynchronous** message is invoked by a MessageCaller on a MessageReceiver, but the MessageCaller does not wait for the message invocation to return before carrying on with the rest of the interaction's steps.

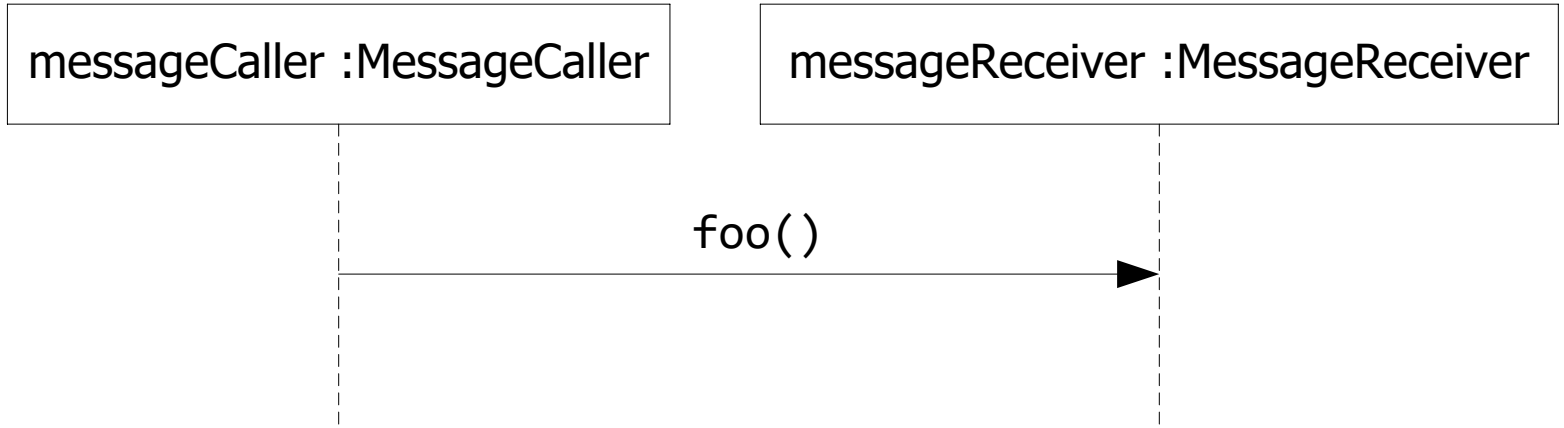


Participant Creation and Destruction Messages

- Participants do not necessarily live for the entire duration of a sequence diagram's interaction.
- Participants can be created and destroyed according to the messages that are being passed, as shown:



A Simple Sequence Diagram to Java



- The messageCaller object makes a regular Java method call to the foo( ) method on the messageReceiver object and then waits for the messageReceiver.foo( ) method to return before carrying on with any further steps in the interaction.

A Simple Sequence Diagram to Java ...

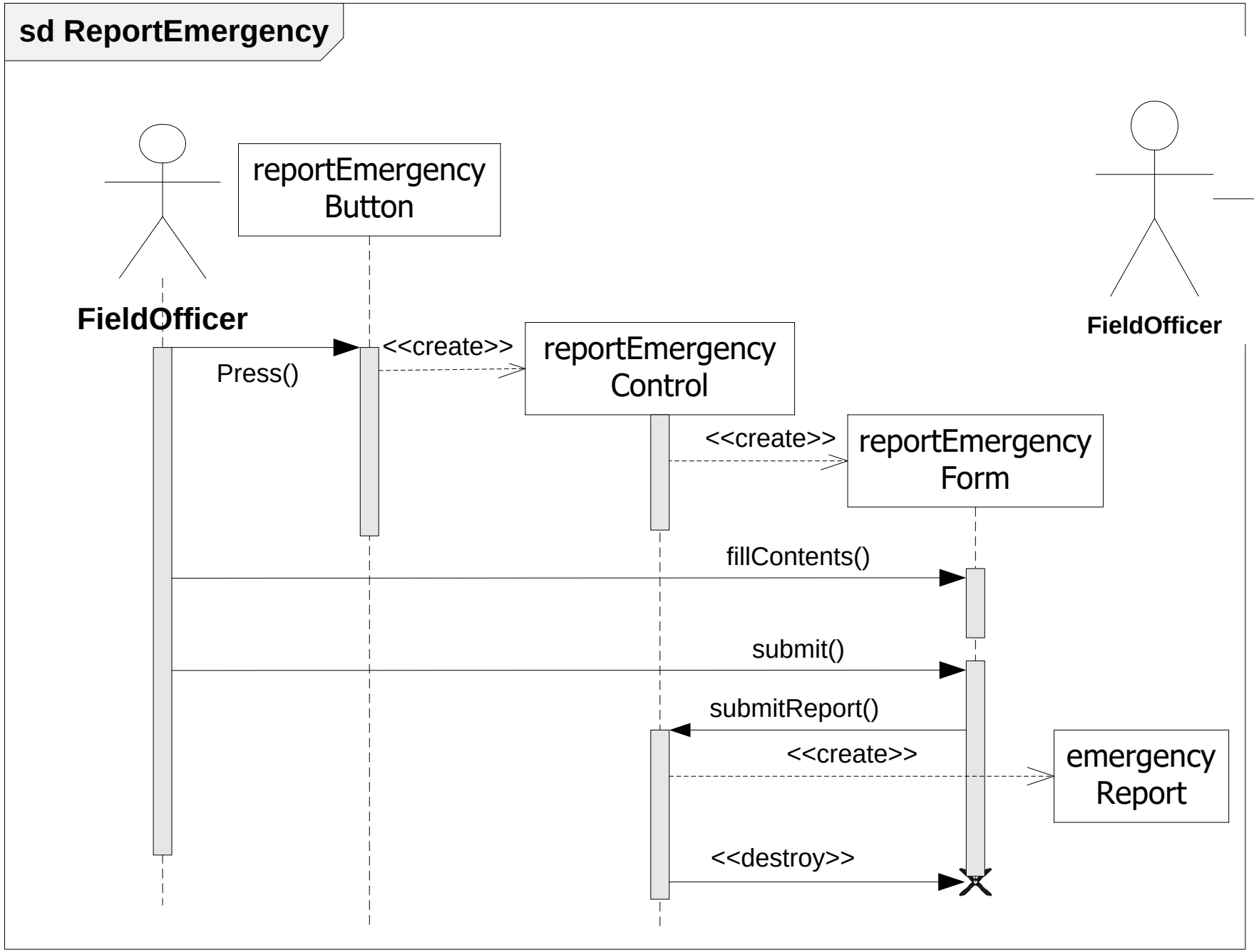
```
public class MessageReceiver {
    public void foo( ) {
       // Do something inside foo.
    }
}

public class MessageCaller {
    private MessageReceiver messageReceiver;
    // Other Methods and Attributes of the class are declared here
    // The messageRecevier attribute is initialized elsewhere in
    // the class.
    public doSomething(String[] args){
        // The MessageCaller invokes the foo( ) method
        this.messageReceiver.foo( ); // then waits for the method to return
        // before carrying on here with the rest of its work
    }
}
```

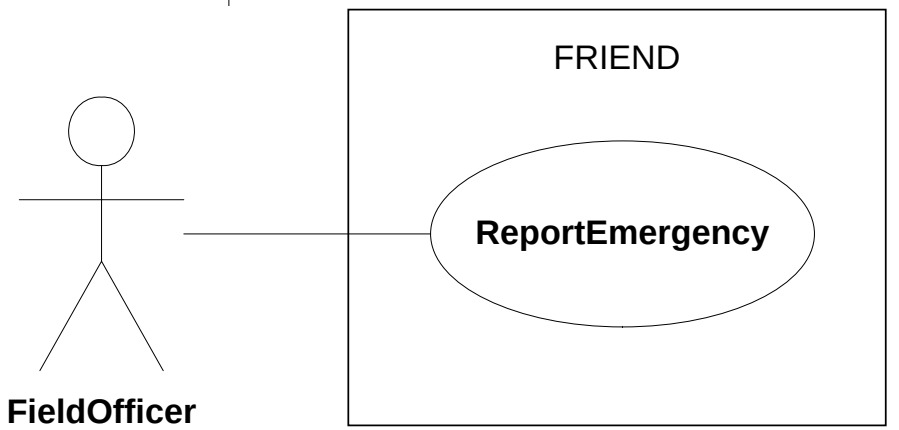
Participant Creation and Destruction Messages

```
public class MessageReceiver {
   // Attributes and Methods of the MessageReceiver class
}

public class MessageCaller {
// Other Methods and Attributes of the class are declared here

   public void doSomething(  ) {
      // The MessageReceiver object is created
      MessageReceiver messageReceiver = new MessageReceiver(  );
   }
}
```

In Java, you will not have an explicit destroy method, so it doesn't need to show one on your sequence diagrams. As shown, the messageReceiver object will be flagged for destruction when the doSomething( ) method completes its execution.

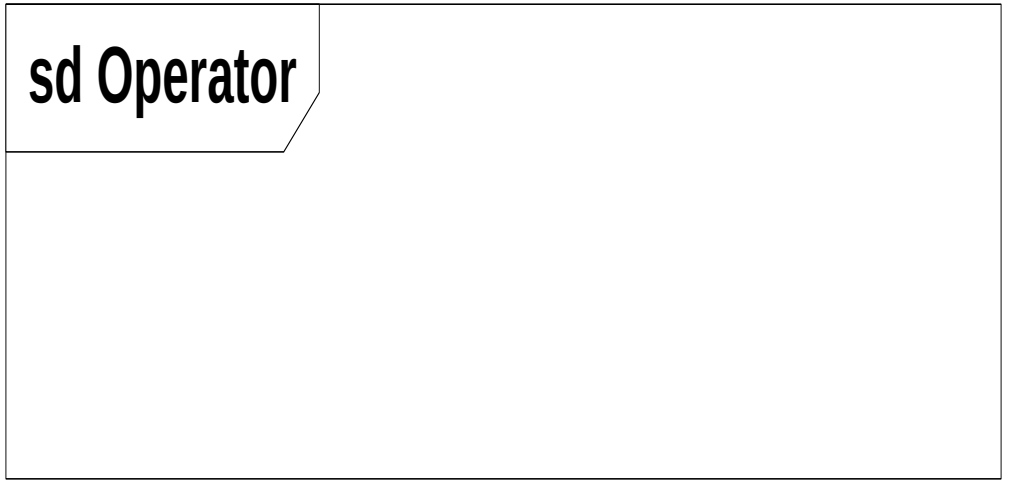
Bringing a Use Case to Life with a Sequence Diagram



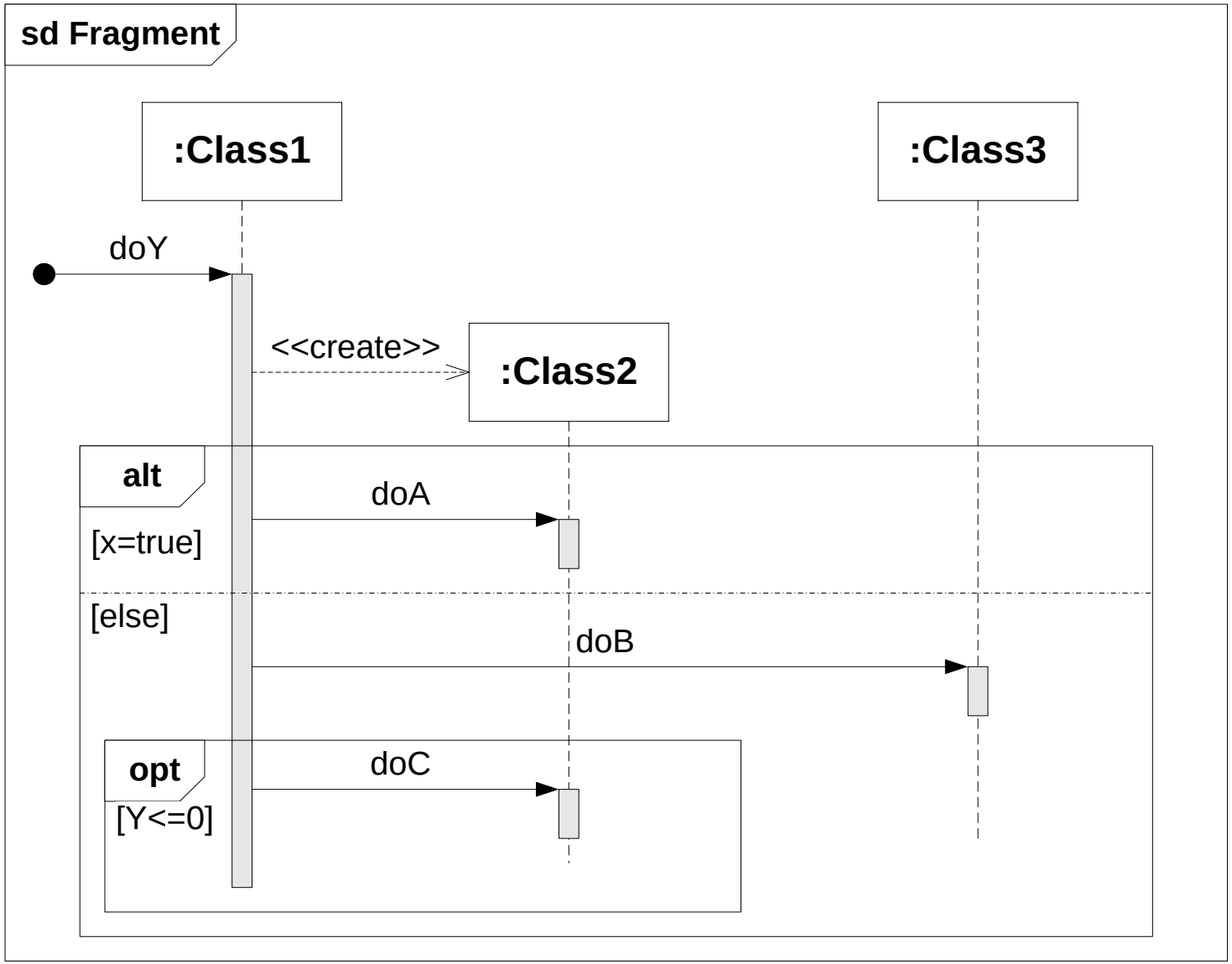


Sequence Fragments

- Suppose that the sequence diagram contains an interaction that we want to **reuse** elsewhere. Or in the sequence diagram we need to show **loops** or **alternative flows**.
  - The answer from the UML designers was the sequence fragment
- A sequence fragment is represented as a box that encloses **a portion** of the interactions within a sequence diagram.
- A fragment box can contain any number of interactions and can contain nested fragments as well.
- The top left corner of the fragment box contains an **operator**. The fragment operator indicates the **type of fragment**.



Sequence Fragments - Notation



Sequence Fragments Types

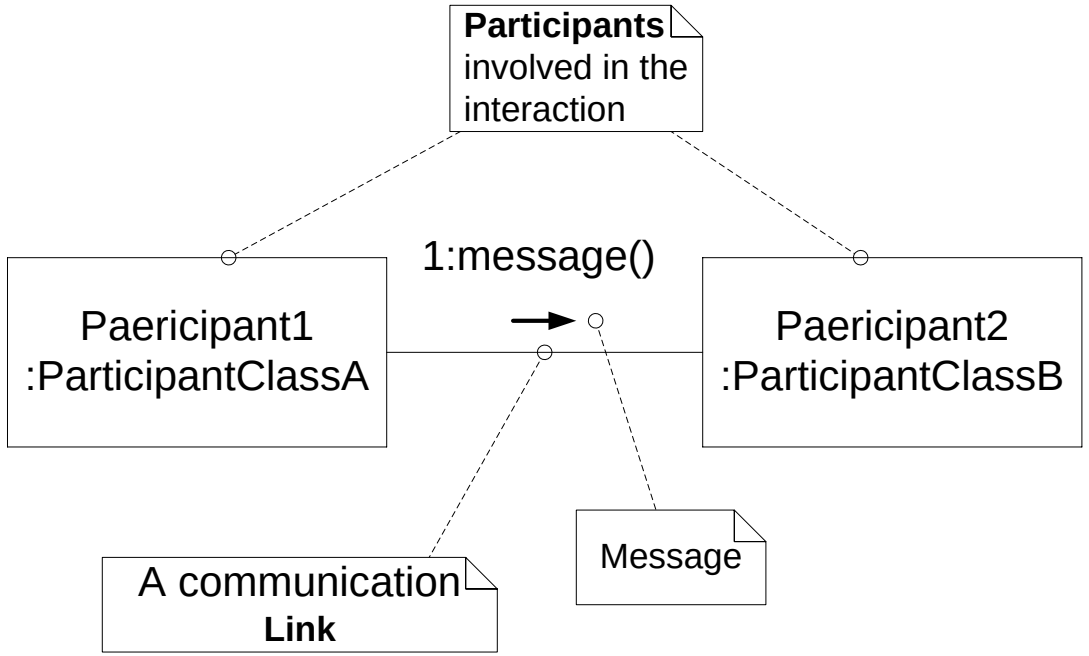
Type	Parameters	Why is it useful?
ref	None	Represents an interaction that is defined elsewhere in the model. Helps you manage a large diagram by splitting, and potentially reusing, a collection of interactions. Similar to the use of <<include>> in use case diagram.
assert	None	Specifies that the interactions contained within the fragment box must occur exactly as they are indicated; otherwise the fragment is declared invalid and an exception should be raised. Works in a similar fashion to the assert statement in Java. Useful when specifying that every step in an interaction must occur successfully, i.e., when modeling a transaction.
loop	min times, max times, [guard condition]	Loops through the interactions contained within the fragment a specified number of times until the guard condition is evaluated to false. Very similar to the Java and C# for(..) loop. Useful when you are trying execute a set of interactions a specific number of times.
break	None	If the interactions contained within the break fragment occur, then any enclosing interaction, most commonly a loop fragment , should be exited. Similar to the break statement in Java and C#.
alt	[guardcondition1] [guardcondition2] ... [else]	Depending on which guard condition evaluates to true first, the corresponding sub-collection of interactions will be executed. Helps you specify that a set of interactions will be executed only under certain conditions. Similar to an if(..) else statement in code.

Communication Diagrams

- The main purpose of sequence diagrams is to show the order of events between the participants of your system that are involved in a particular interaction.
- Communication diagrams add another perspective to an interaction by focusing on the links between the participants.
- To compare:
  - On a sequence diagram, the links between participants are implied by the fact that a message is passed between them.
  - Communication diagrams provide an intuitive way to show the links between participants that are required for the events that make up an interaction.

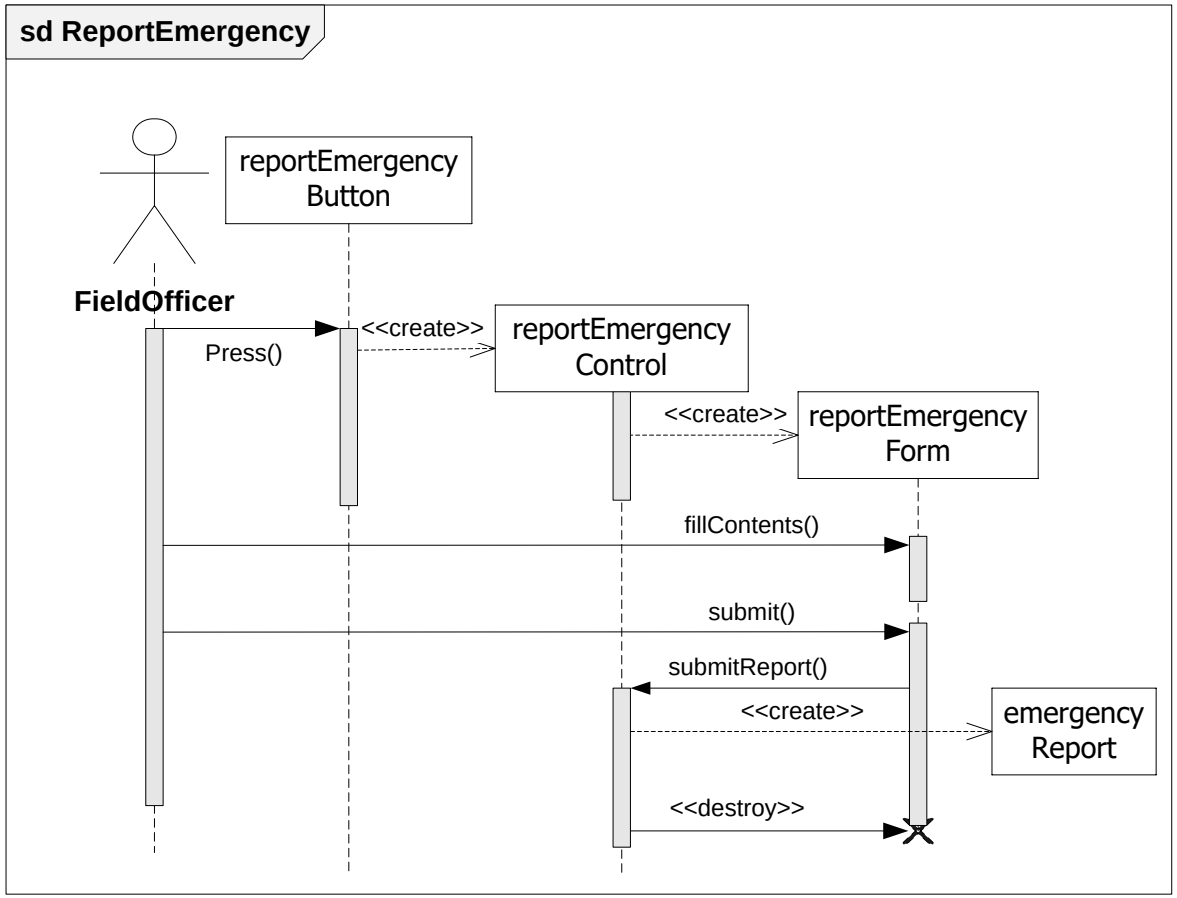
Participants, Links, and Messages

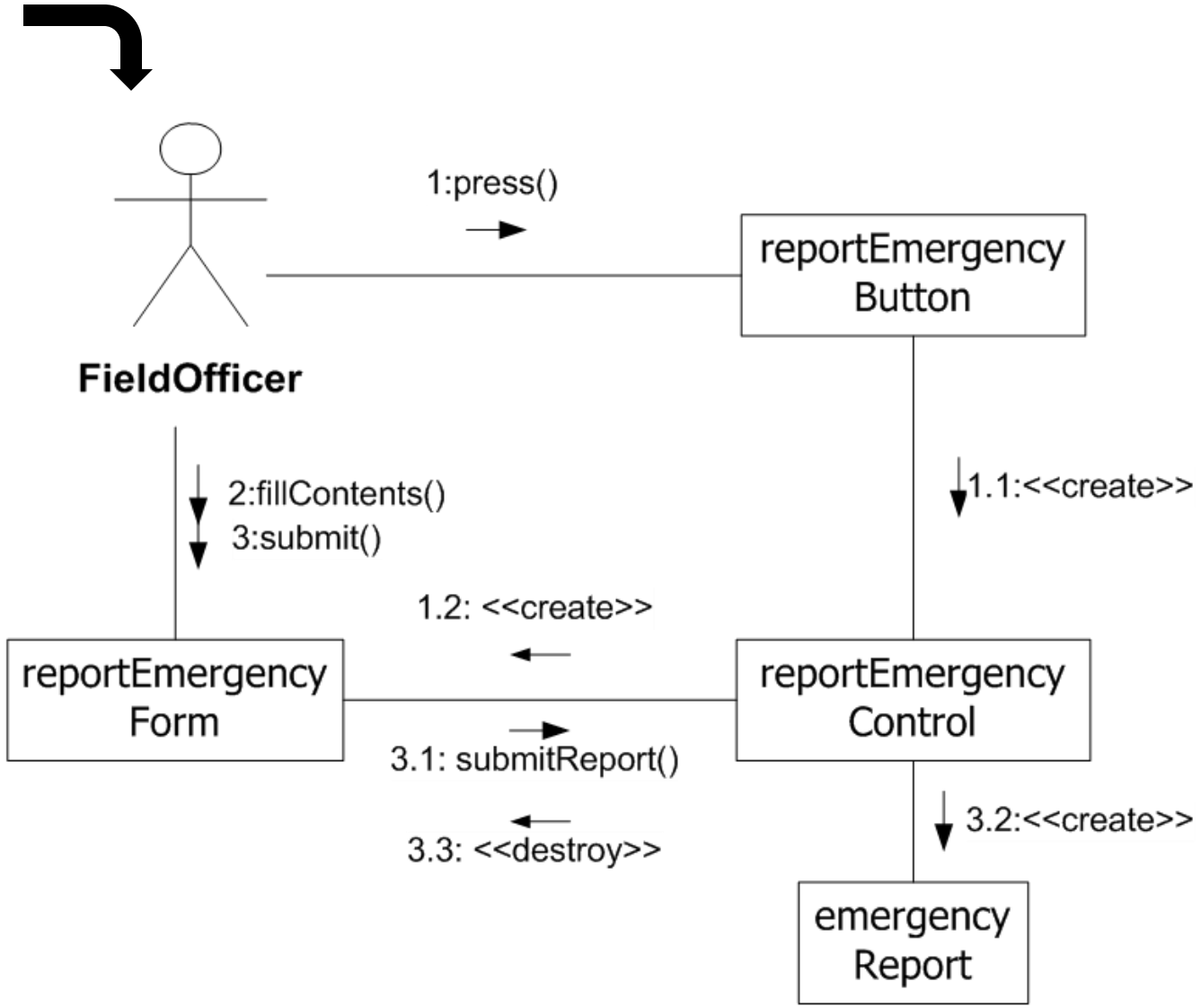
- A communication diagram is made up of three things: participants, the communication links between those participants, and the messages that can be passed along those communication links, as shown

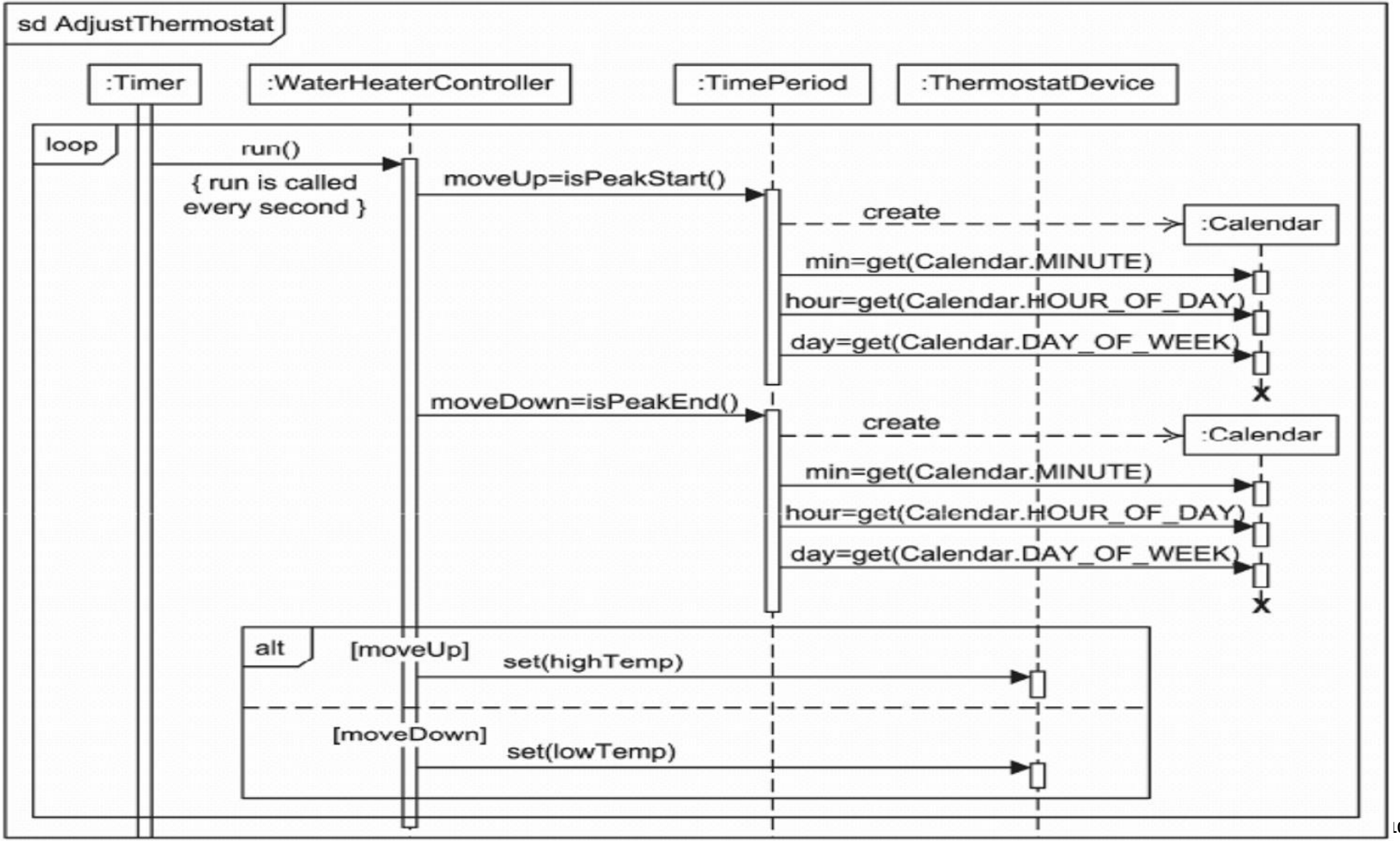


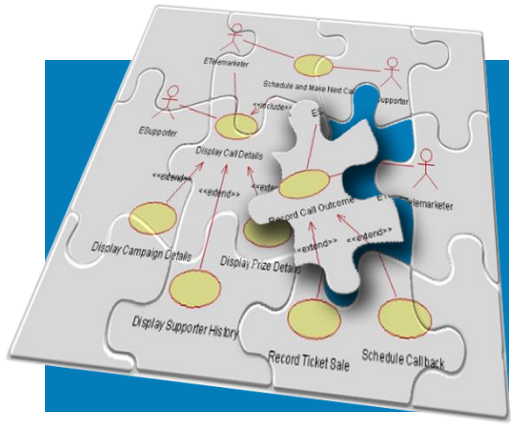
- Message order on a communication diagram is shown using **a number** before each message.
- Numbers start at 1 and increasing until all of the messages on the diagram are involved

Communication Diagram - example



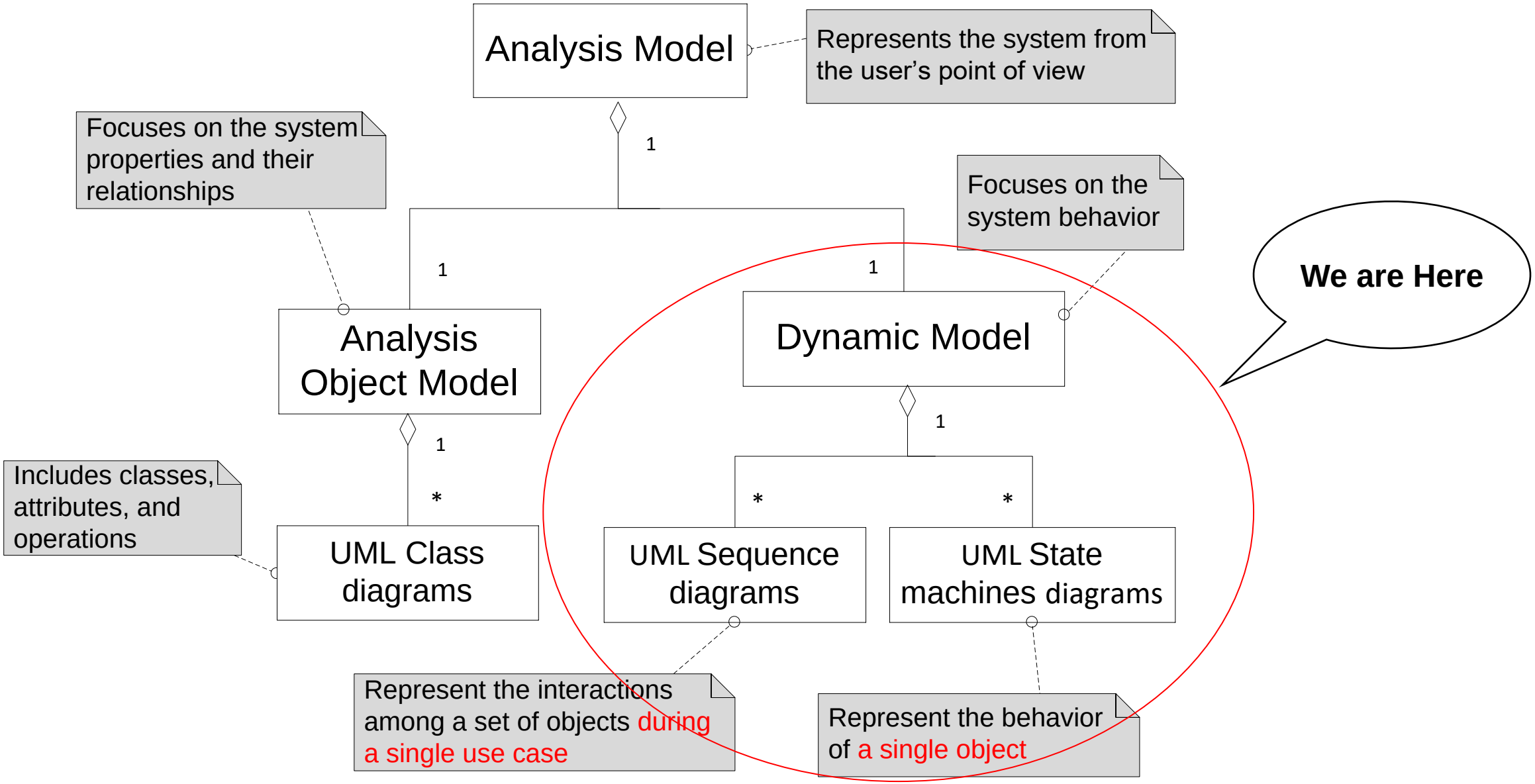



sd AdjustThermostat
:Timer
:WaterHeaterController
:TimePeriod
:ThermostatDevice
loop
run()
{ run is called every second }
moveUp=isPeakStart()
create
:Calendar
min=get(Calendar.MINUTE)
hour=get(Calendar.HOUR_OF_DAY)
day=get(Calendar.DAY_OF_WEEK)
moveDown=isPeakEnd()
create
:Calendar
min=get(Calendar.MINUTE)
hour=get(Calendar.HOUR_OF_DAY)
day=get(Calendar.DAY_OF_WEEK)
alt
[moveUp]
set(highTemp)
[moveDown]
set(lowTemp)



Mapping Use Cases to Objects with Sequence Diagrams

Analysis Object Models and Dynamic Models



- **A sequence diagram ties use cases with objects**.
- It shows **how** the a use case behaves among its participating objects.

UML Interaction Diagrams

Our tools for Dynamic Modeling

- Two types of interaction diagrams:
  - **Sequence Diagram:**
    - Models the interaction between classes and describes the dynamic behavior of several objects over time
    - Good for real-time specifications
  - **Communication Diagram:**
    - Shows the visibility relationship among objects
    - Position of objects is based on the position of the classes in the UML class diagram.
    - Does not show time.

UML State Chart Diagram

Our tools for Dynamic Modeling

- **State Chart Diagram:**
  - A state machine that describes the response of an object of a given interesting class to the receipt of outside actions (Events).
  - Classes without interesting dynamic behavior are not modeled with state diagrams
- **Activity Diagram:**
  - Activity diagrams is used for modeling business processes, in which several object participate.
  - Activity diagrams are "OO flow charts"

Dynamic Modeling

- Purpose:
  - **Detect and supply** operations for the object model.
    - How? - From the flow of events we proceed to **the sequence diagram** to find the operations for the object model.

What is an Event?

- Something that happens at a point in time
- An event sends information from one object to another
- Events can have associations with each other:
  - Causally related:
    - An event happens always before another event
    - An event happens always after another event
  - Causally unrelated:
    - Events that happen concurrently

An event is defined as a message of notification of a change in system state that is of interest.

Sequence Diagram

- Heuristic for finding participating objects in Sequence Diagrams:
  - An event always has a sender and a receiver.
  - The representation of the event is sometimes called a message
  - Find them for each event => These are the objects participating in the sequence diagram

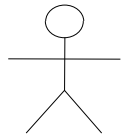
Building Sequence Diagram - an Example

- Flow of **events** for “CheckAccountBalance” use case (ATM system):
  1. The Customer inserts his bank card into the ATM.
  2. The ATM welcomes the Customer and opens the main menu.
  3. The Customer selects the “CheckAccountBalance” function by pressing the appropriate button.
  4. The ATM system shows the enter PIN screen to the Customer.
  5. The Customer enters his correct PIN and confirms it.
  6. ATM system shows the actual balance of the Customer’s account.

- Where are the objects?

Heuristics for Sequence Diagrams

- Layout:
  - 1st column: Should be the actor who initiated the use case
  - 2nd column: Should be a boundary object (that actor used to initiate the use case).
  - 3rd column: Should be the control object that manages the rest of the use case
- Creation of objects:
  - Create control objects at beginning of event flow
  - Control objects are created by boundary objects initiating use cases
  - The control objects create the other boundary objects
- Access of objects:
  - Entity objects can be accessed by control and boundary objects
  - Entity objects recommend not to access boundary or control objects
  - This makes it easier to share entity objects across use cases



Customer

ATMMainForm | **ATMController** | **CustomerAccount**

1. The Customer inserts his bank card.
2. The ATM welcomes the Customer and opens the main menu.
3. The Customer selects "CheckAccountBalance" function.
4. The ATM system shows the enter PIN screen to the Customer.
5. The Customer enters his correct PIN and confirms it.
6. The ATM system shows the actual balance of the Customer's account.

time

Insert card to start a transaction

Display welcome message

Display main menu

CheckAccountBalance button clicked

CheckAccountBalance

Show PIN confirmation field

Enter PIN number

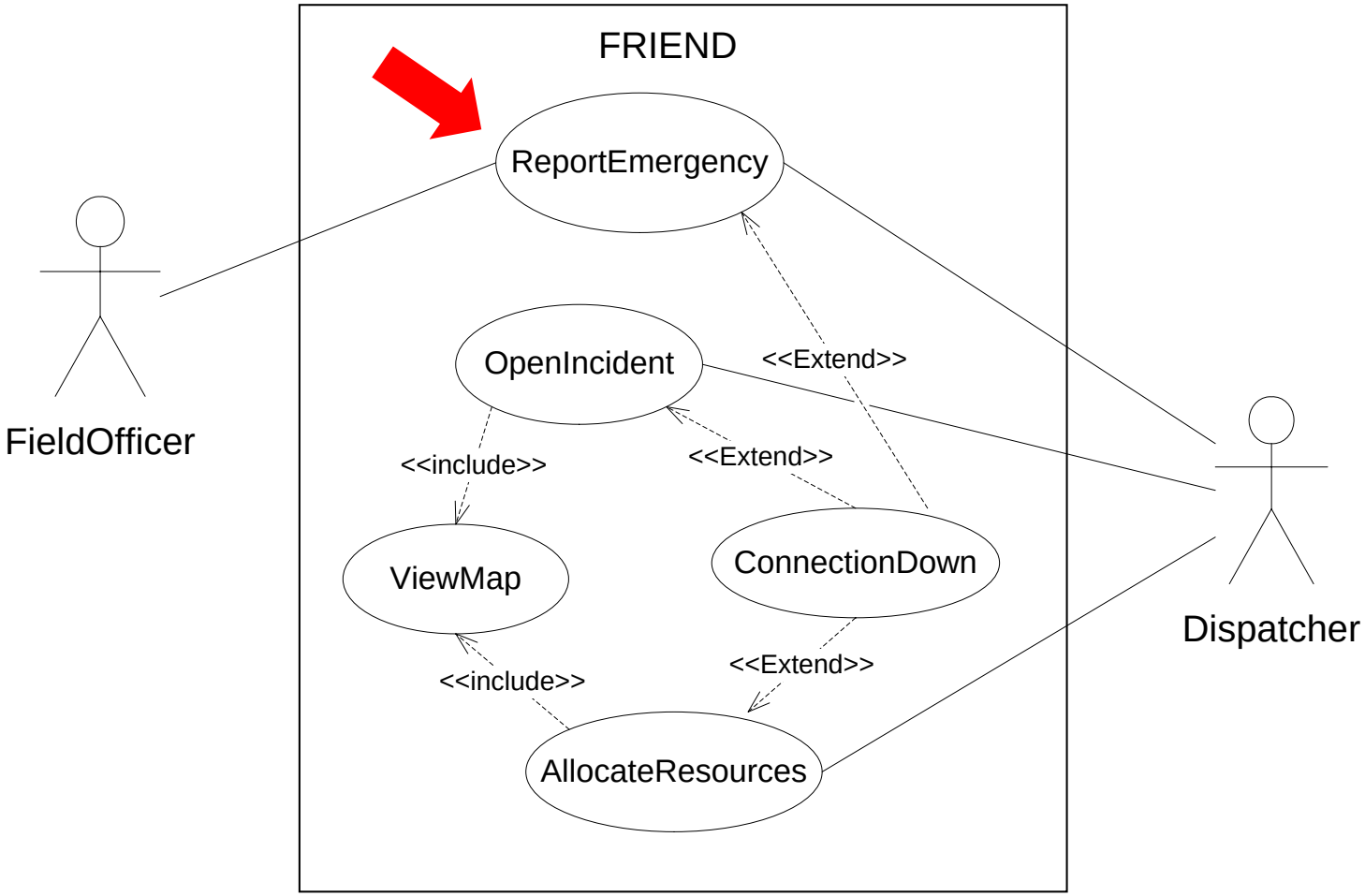
CheckAccountBalance (PIN)

GetCustBalance (PIN)

Show Customer balance

From use case to objects with Sequence Diagram

- Let us focus only on **ReportEmergency** use case and map it to objects with **Sequence Diagram**.



The Identified ReportEmergency Classes

FieldOfficer

EmergencyReport

ReportEmergencyButton

FieldOfficerStation

ReportEmergencyForm

ReportEmergencyControl

Dispatcher

Incident

IncidentForm

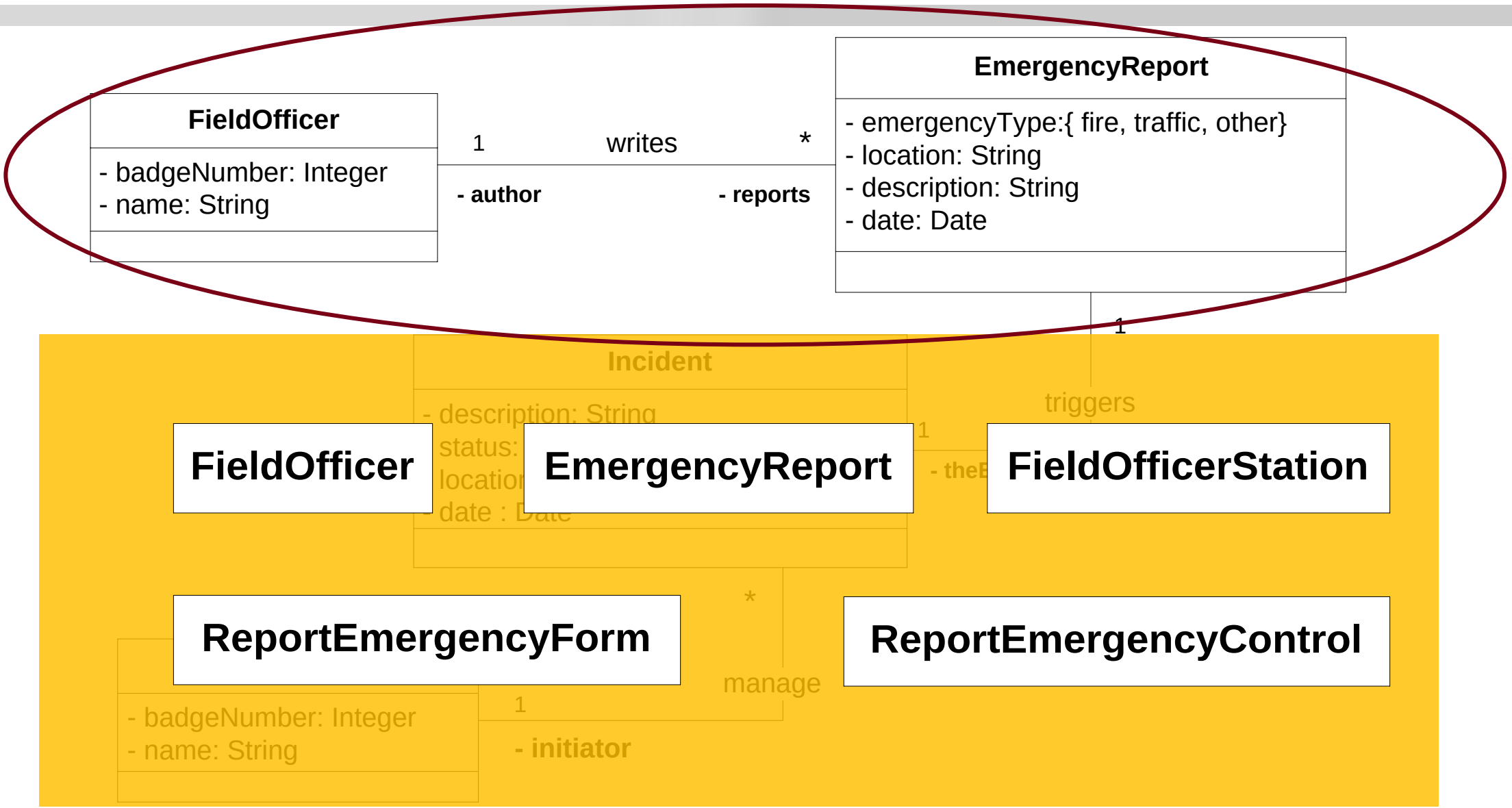
DispatcherStation

AcknowledgmentNotice

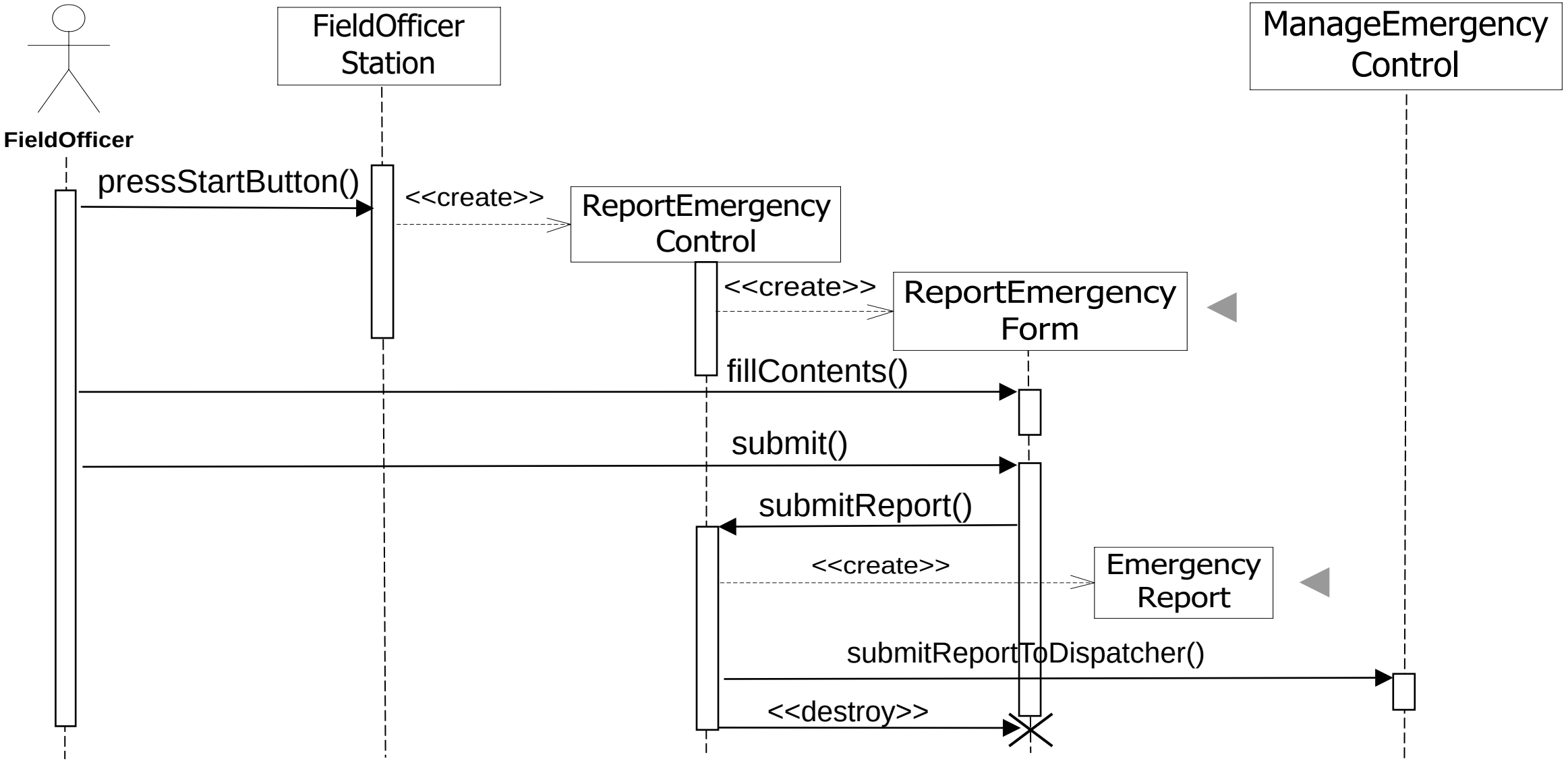
ManageEmergencyControl

- Other classes may be discovered during objects - sequence diagram mapping. Other classes may be eliminated as well.

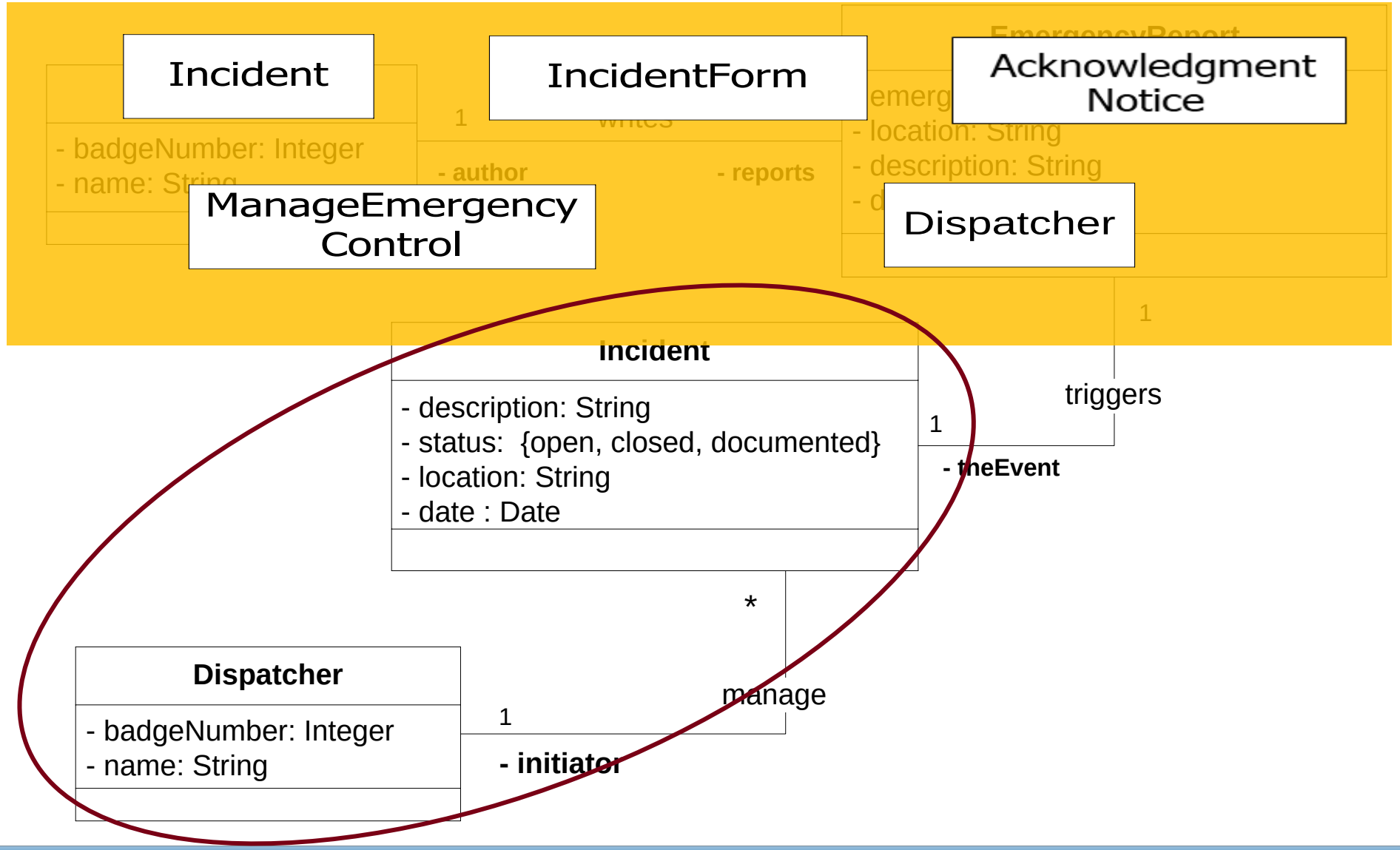
From Class diagram to Sequence Diagram



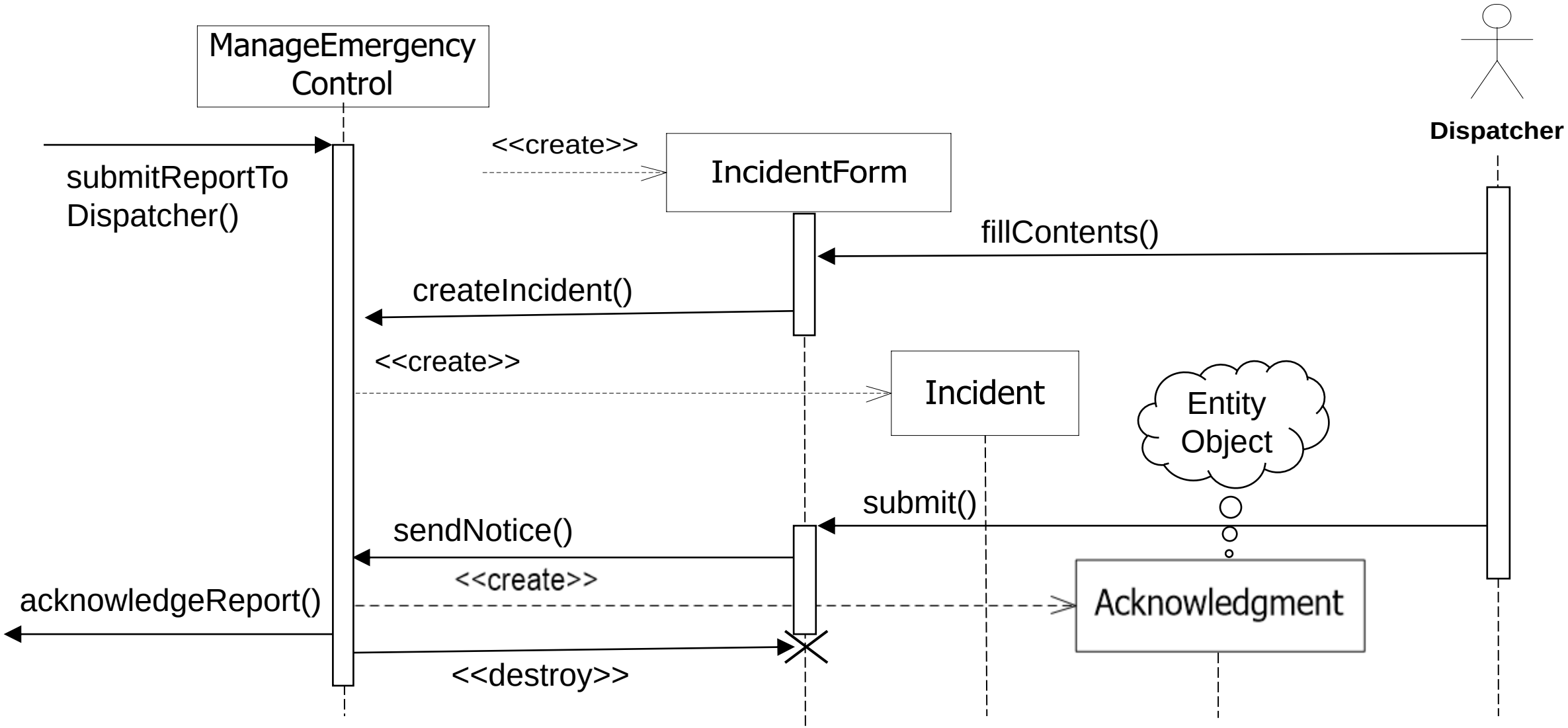
Sequence diagram for the ReportEmergency use case (1)



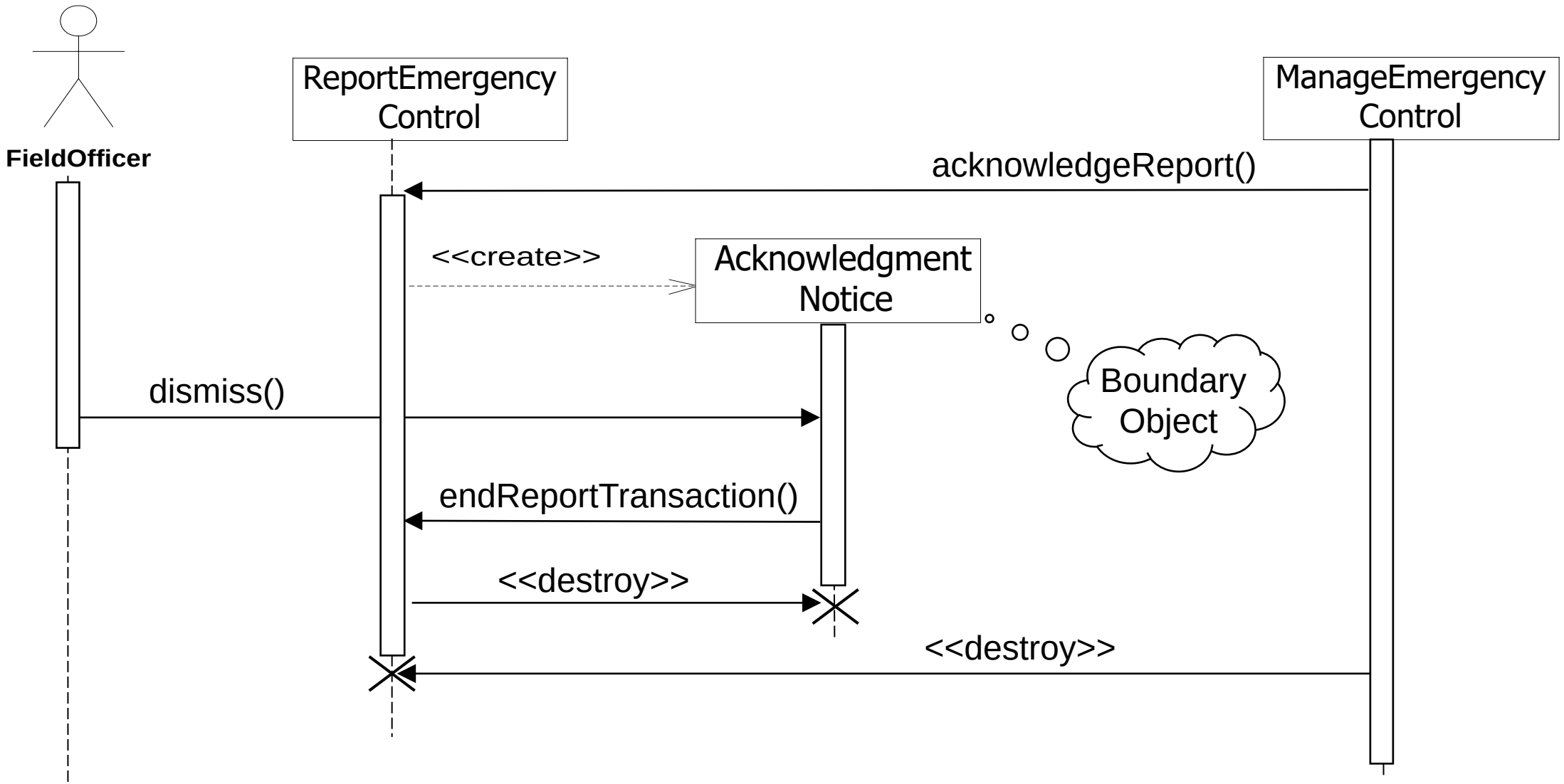
From Class diagram to Sequence Diagram



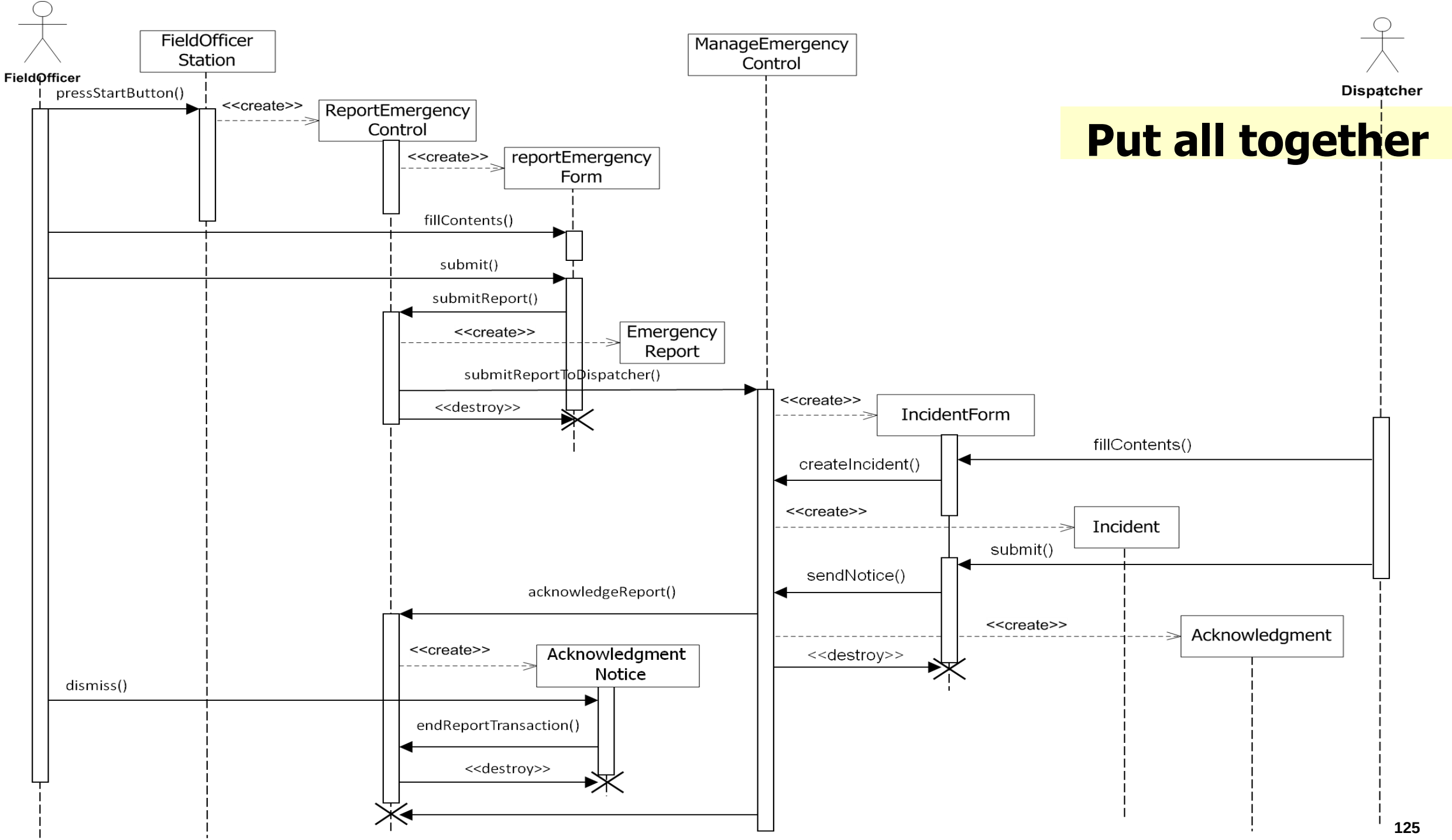
Sequence diagram for the ReportEmergency use case (2)



Sequence diagram for the ReportEmergency use case (3)



Put all together



Impact on FRIEND's Object Model

- At this modeling stage FRIEND's object model contains objects:

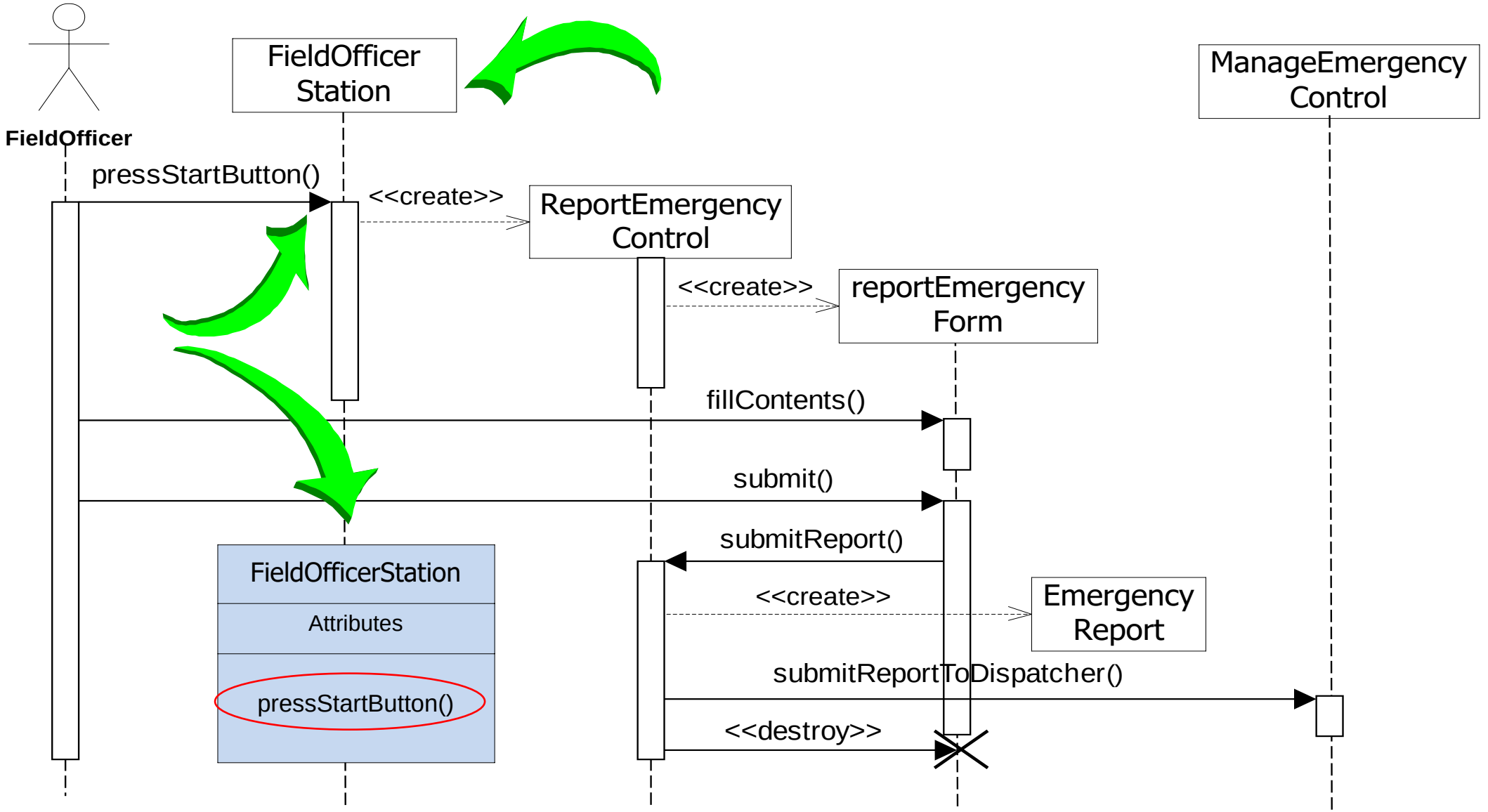
  Dispatcher , EmergencyReport , FieldOfficer , Incident , AcknowledgmentNotice , DispatcherStation , ReportEmergencyButton , EmergencyReportForm , FieldOfficerStation , IncidentForm , ReportEmergencyControl and ManageEmergencyControl

- The Sequence Diagram identifies one new entity class Acknowledgment , and dropped the ~~ReportEmergencyButton~~ class.

Impact on FRIEND's Object Model ...

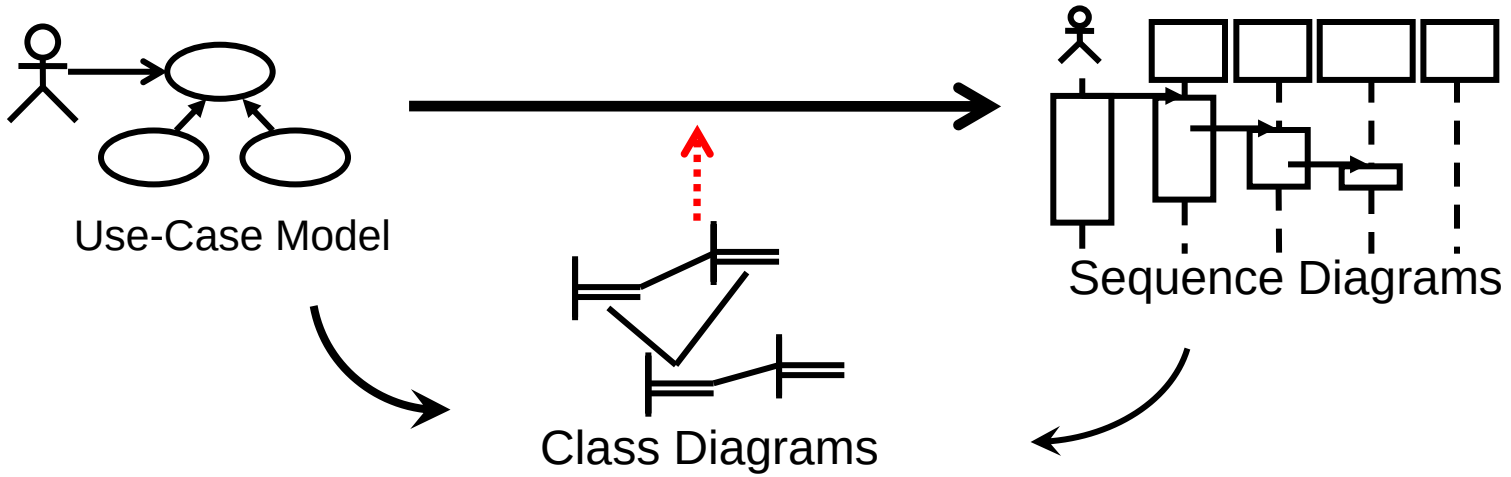
- The sequence diagram also supplies us with many new events
  - pressStartButton()
  - fillContents()
  - submitReport()
  - submitReportToDispatcher()
  - createIncident()
  - acknowledgeReport()
  - submit()
  - endReportTransaction()
  - dismiss()
- Question: Who owns these events?
- Answer:
  - For each object that receives an event there is a public operation in its associated class
  - The name of the operation is usually the name of the event.

Impact on FRIEND's Object Model



What else can we say about Sequence Diagrams?

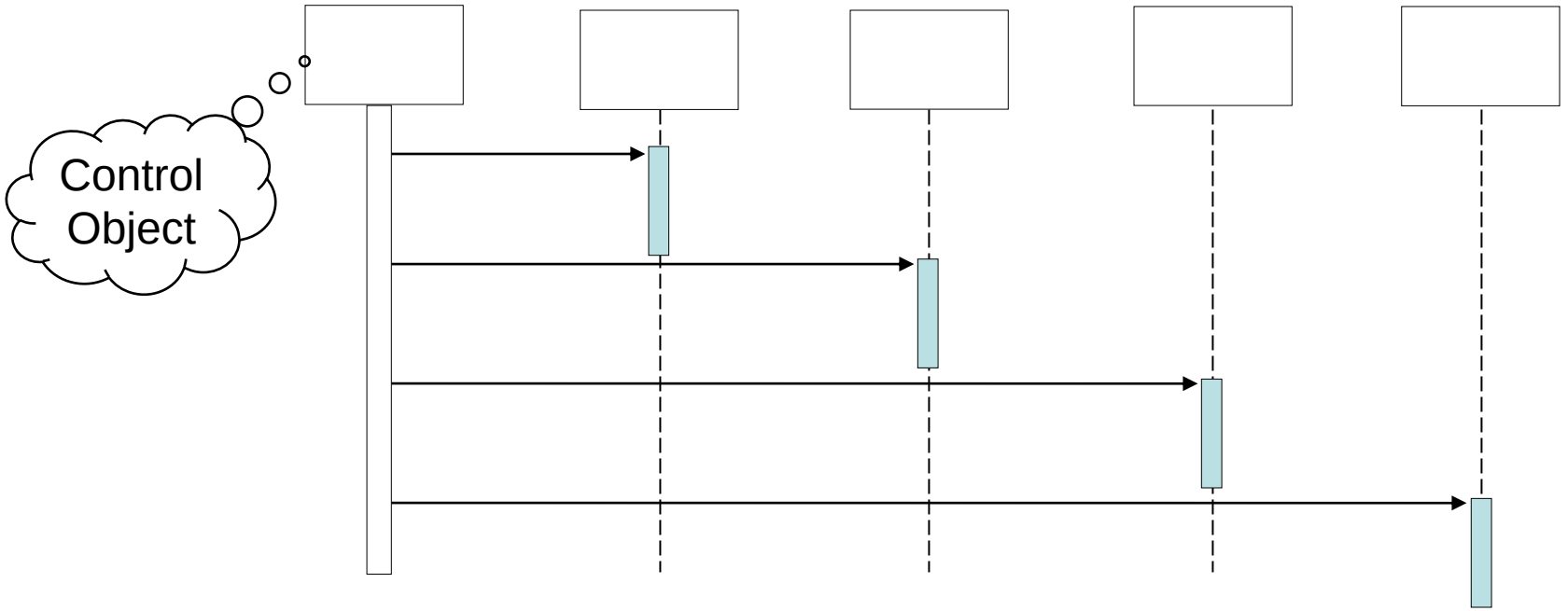
- Sequence diagrams are derived from use cases



- The structure of the sequence diagram helps us to determine how decentralized the system is.
- We distinguish two structures for sequence diagrams.
  - **Fork Diagrams** and **Stair Diagrams**

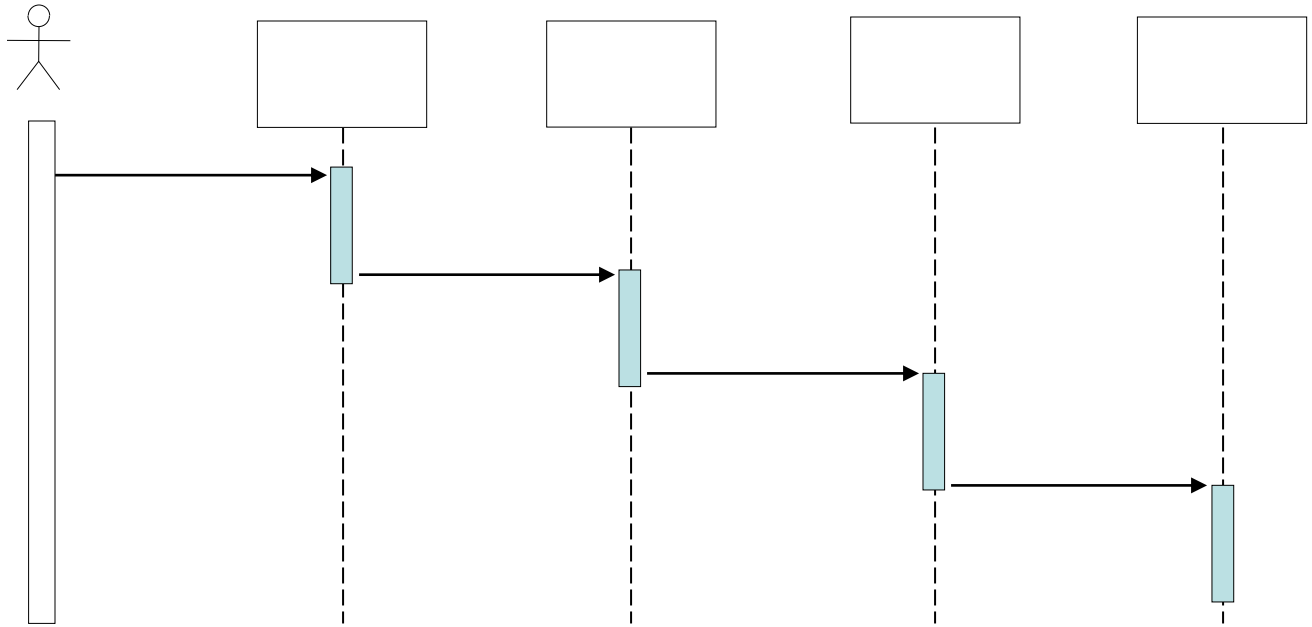
Fork Diagram

- The dynamic behavior is placed in a single object, usually a control object
  - It knows all the other objects and often uses them for direct questions and commands

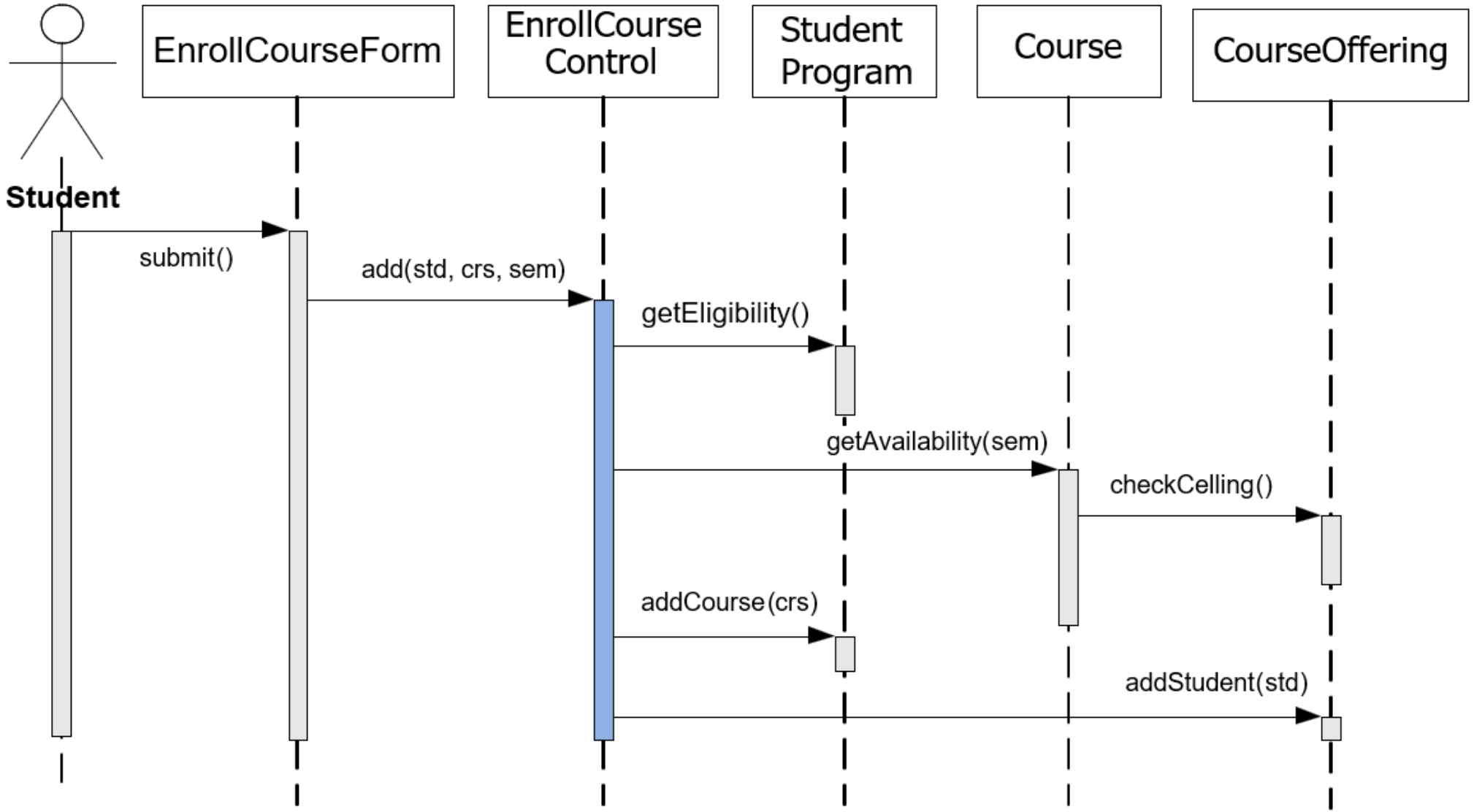


Stair Diagram

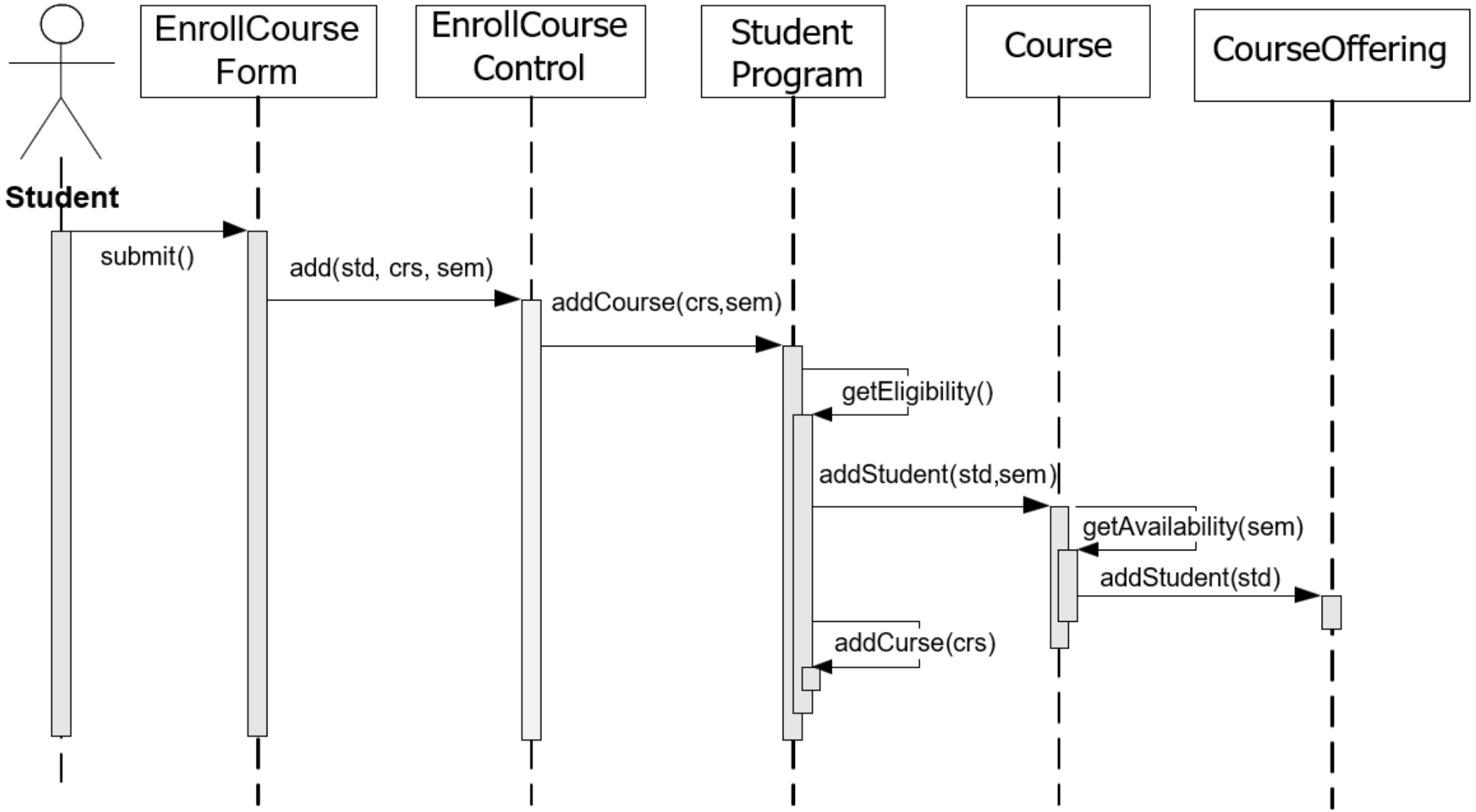
- The dynamic behavior is distributed. Each object delegates responsibility to other objects
  - Each object knows only a few of the other objects and knows which objects can help with a specific behavior



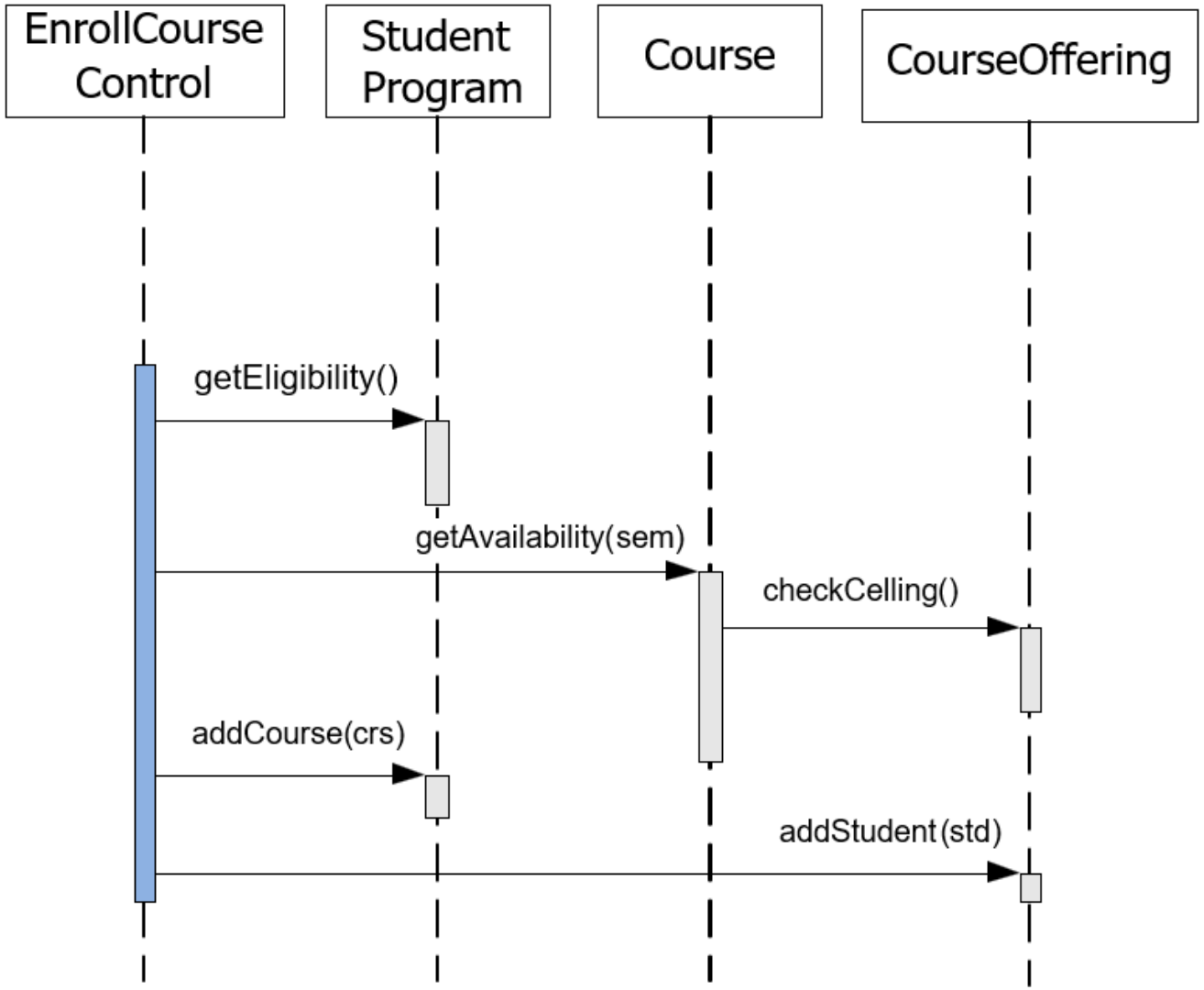
Fork Diagram - Example



Stair Diagram - Example



Fork Diagram - Example

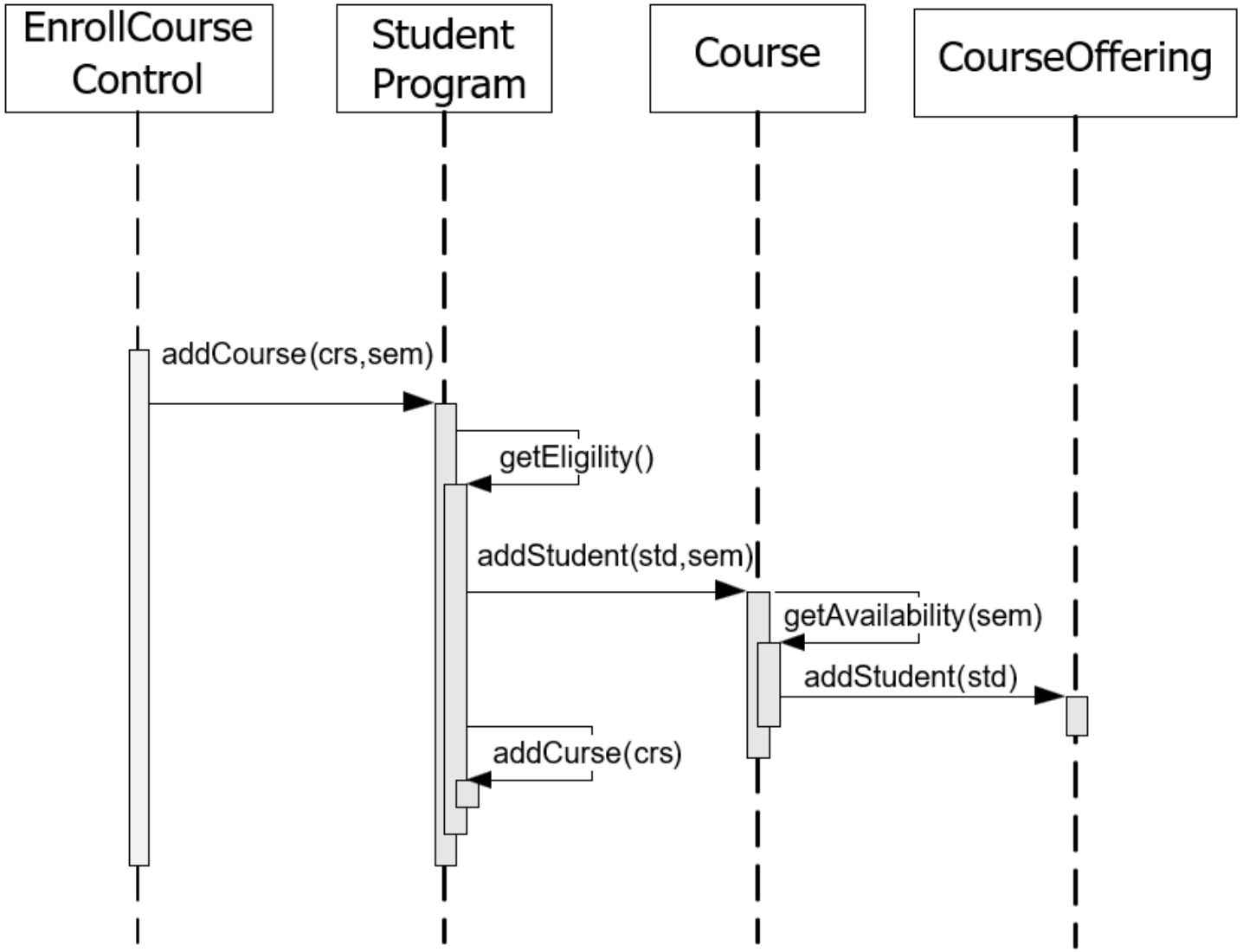


Student Program
getEligibility() : boolean addCourse(crs: CourseOffering) : void

Course
getAvailability() : CourseOffering

CourseOffering
checkCelling() : boolean addStudent(std : Student) : Void

Stair Diagram - Example



Student Program
getEligibility() : boolean addCourse(crs: CourseOffering, sem : String) : void addCourse(crs: CourseOffering) : void

Course
addStudent(std : Student, sem : String) : Void getAvailability(sem : String) : CourseOffering

CourseOffering
addStudent(std : Student) : Void

Fork or Stair?

- Choose the fork - a centralized control structure - if
  - The operations can change order
  - New operations are expected to be added as a result of new requirements.

- Choose the stair - a decentralized control structure - if
  - The operations have a strong connection
  - The operations will always be performed in the same order

Simple Chat System

- You are required to develop a distributed Chat system.
- This system will allow several people to communicate with each other simultaneously.
- It consists of a single server program that clients connect to.
- The client programs read input (message) from the keyboard and send the input to the server.
- The server then distributes that message to all the other clients
- Each client then displays the message sent to it by the server.

Let's focus on the client program only

Simple Chat System

- Let us apply the software engineering practices and start to identify the core functionality of the client program.
- What does the client do? (the client program functionality)

Get message from the user

Display message to the user

Connect to the server

Send data to the server

Receive data from the server

Simple Chat System

- Let us apply the software engineering practices and start to identify the core functionality of the client program.
- What does the client do? (the client program functionality)
- How does the client do it? (the logic that the client is using)

Get message from the user

Display message to the user

When program start, the client connects to the server

Once the user enter a new message, the program send it to the server

When the server respond, the message is displayed

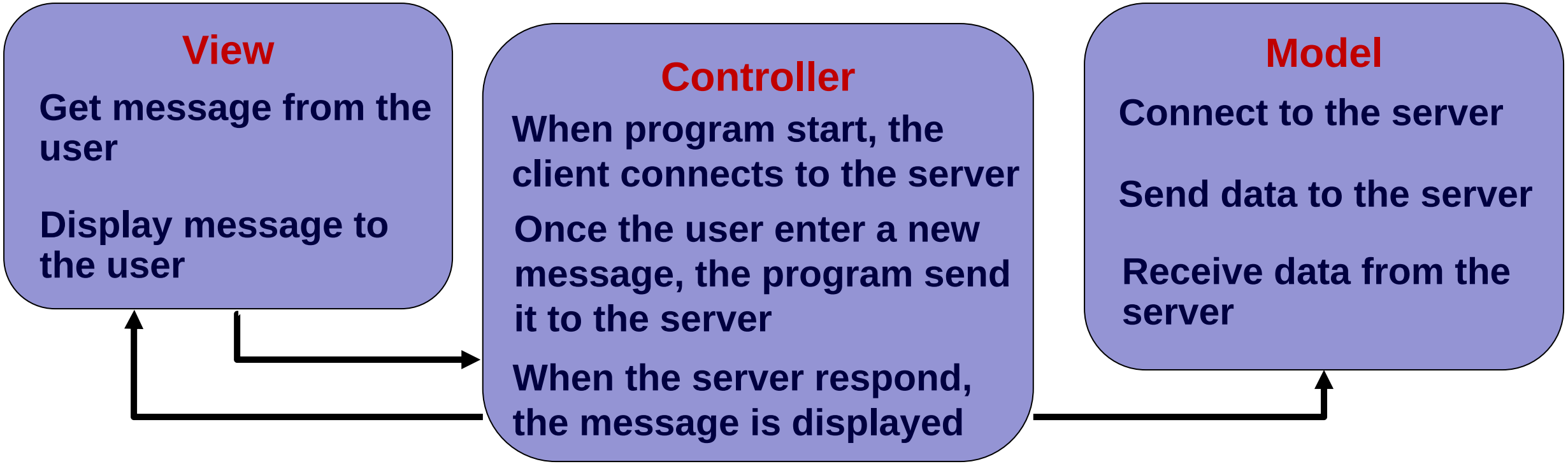
Connect to the server

Send data to the server

Receive data from the server

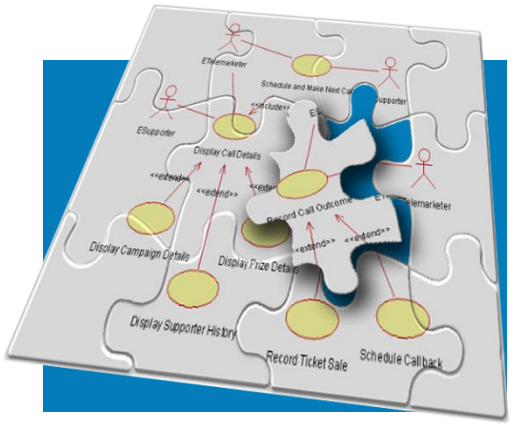
Simple Chat System

- Let us apply the software engineering practices and start to identify the core functionality of the client program.
- What does the client do? (the client program functionality)
- How does the client do it? (the logic that the client is using)



Simple Chat System

DEMO



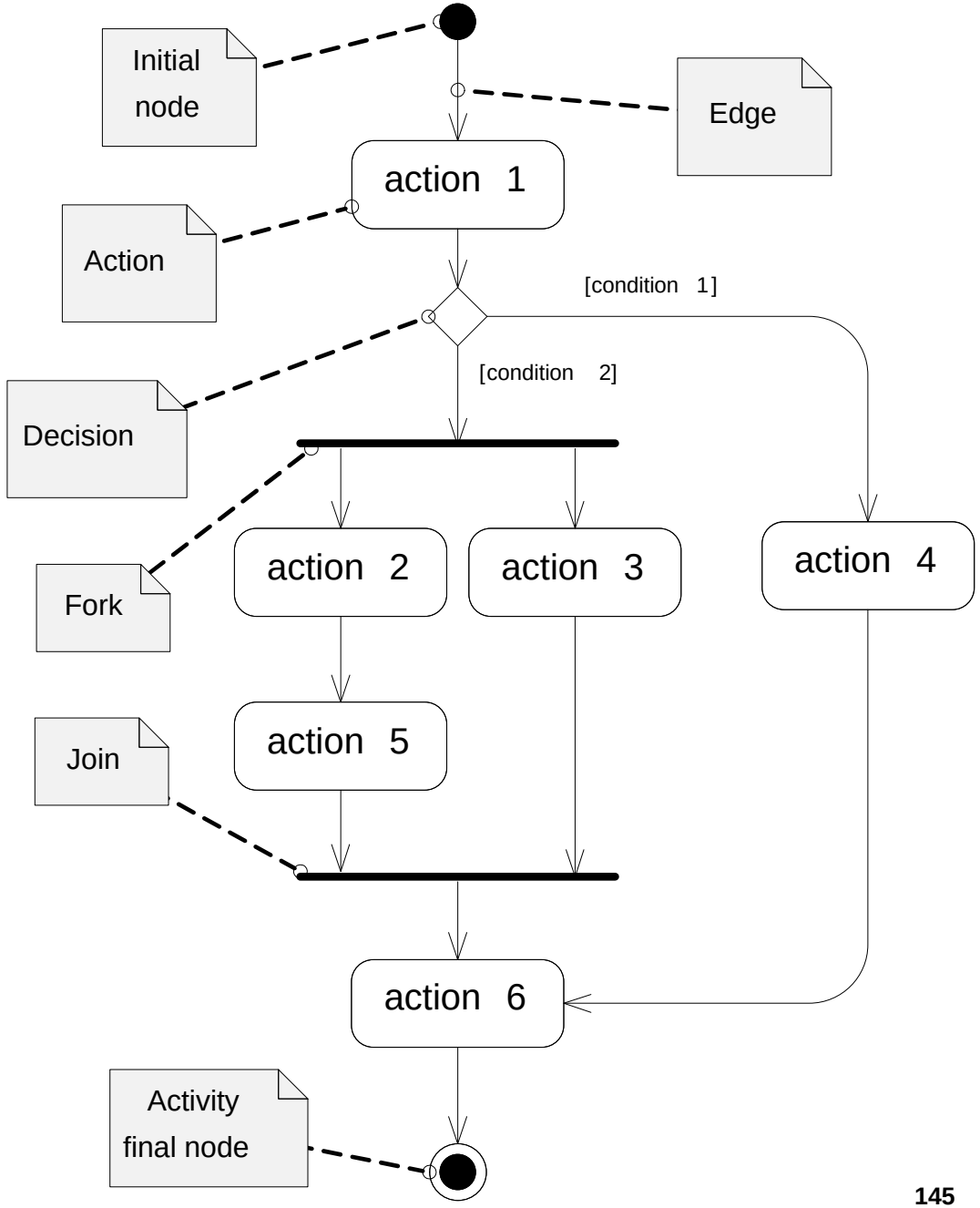
Modeling System Workflows: Activity Diagrams

Activity Diagrams

- **Use cases** show what your system **should do**.
- **Activity diagrams** allow you to specify **how** your system will accomplish its goals.
  - For example, you can use an activity diagram to model the steps involved with report emergency.
- Activity diagrams is used for modeling business processes, in which several object participate.
- **Activity diagrams are "OO flow charts"**

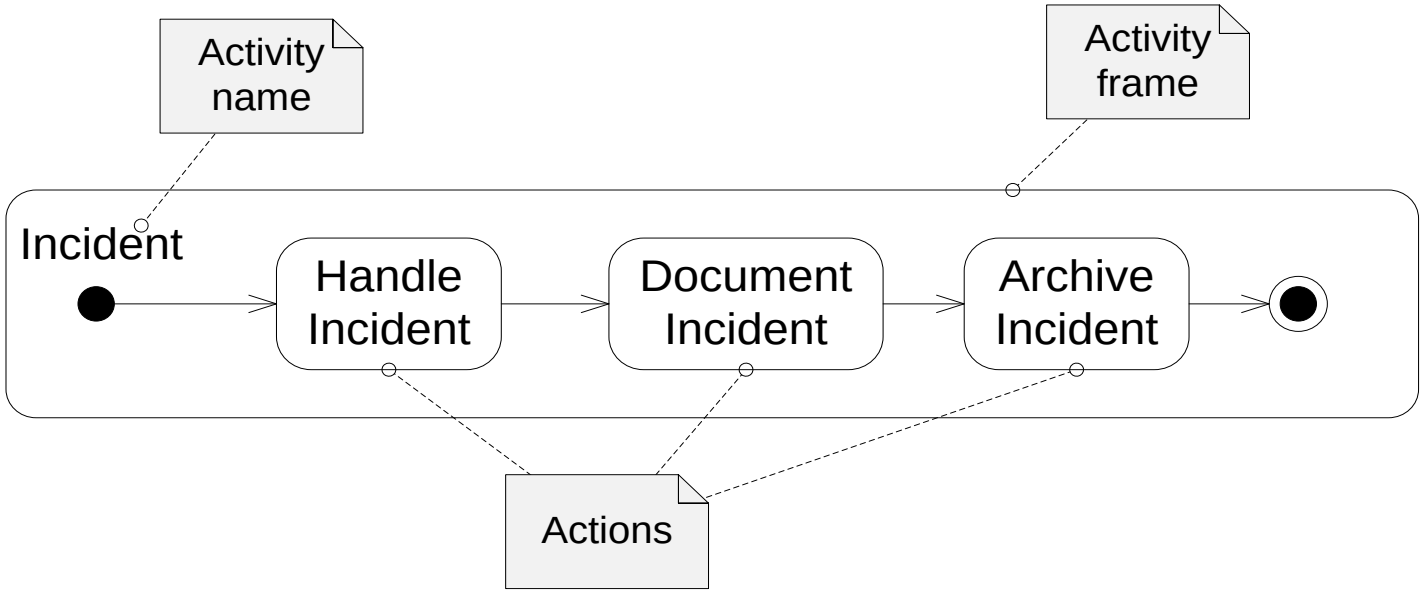
Activity Diagram Essentials

- The activity is launched by the **initial node** , which is drawn as a filled circle.
- At the other end of the diagram, the activity **final node**, drawn as two concentric circles with a filled inner circle, marks the end of the activity
- In between the initial node and the activity final node are **actions** , which are drawn as rounded rectangles.
- The flow of the activity is shown using arrowed lines called **edges** or **paths**.
- The diamond-shaped node is called a **decision**, analogous to an if-else statement.



Activities and Actions

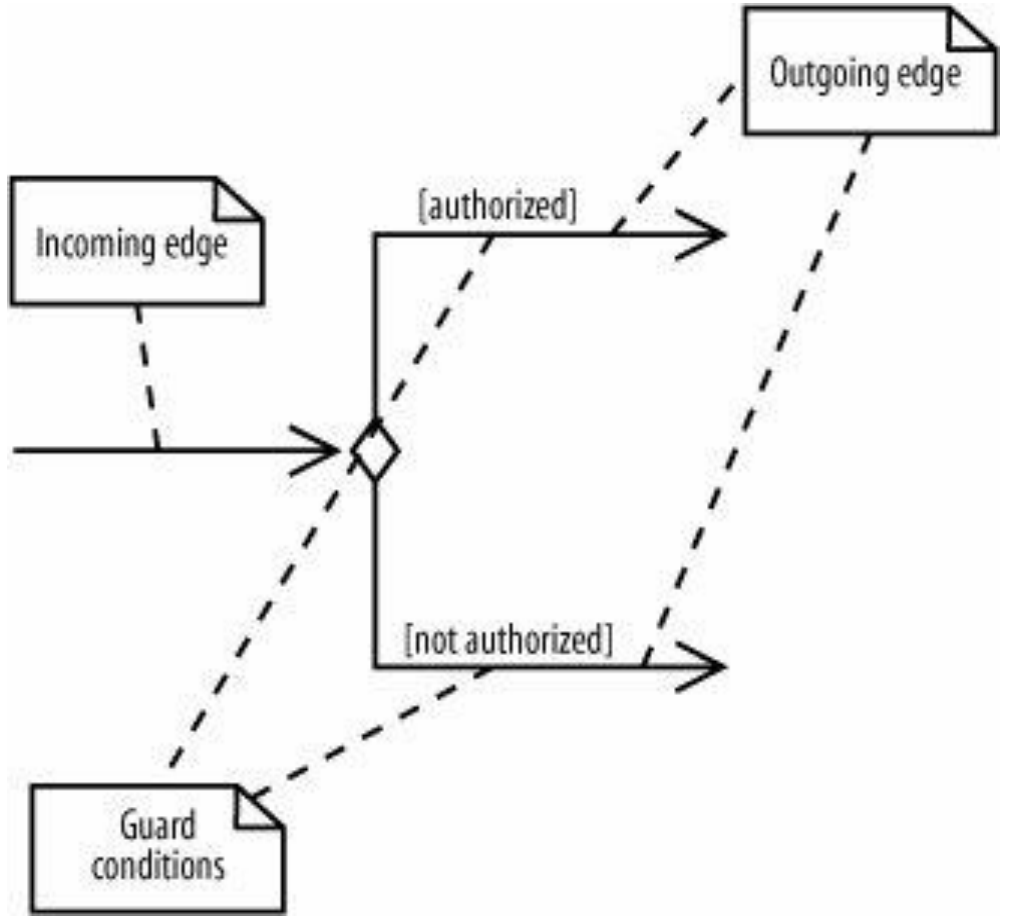
- Actions are active steps in the completion of a process.
- The word "activity" is often mistakenly used instead of "action" to describe a step in an activity diagram, but they are not the same.
  - An activity is the process being modeled, such as Incident. An action is a step in the overall activity, such as Handle Incident, Document Incident, and Archive Incident.



  - The activity frame is optional and is often left out of an activity diagram

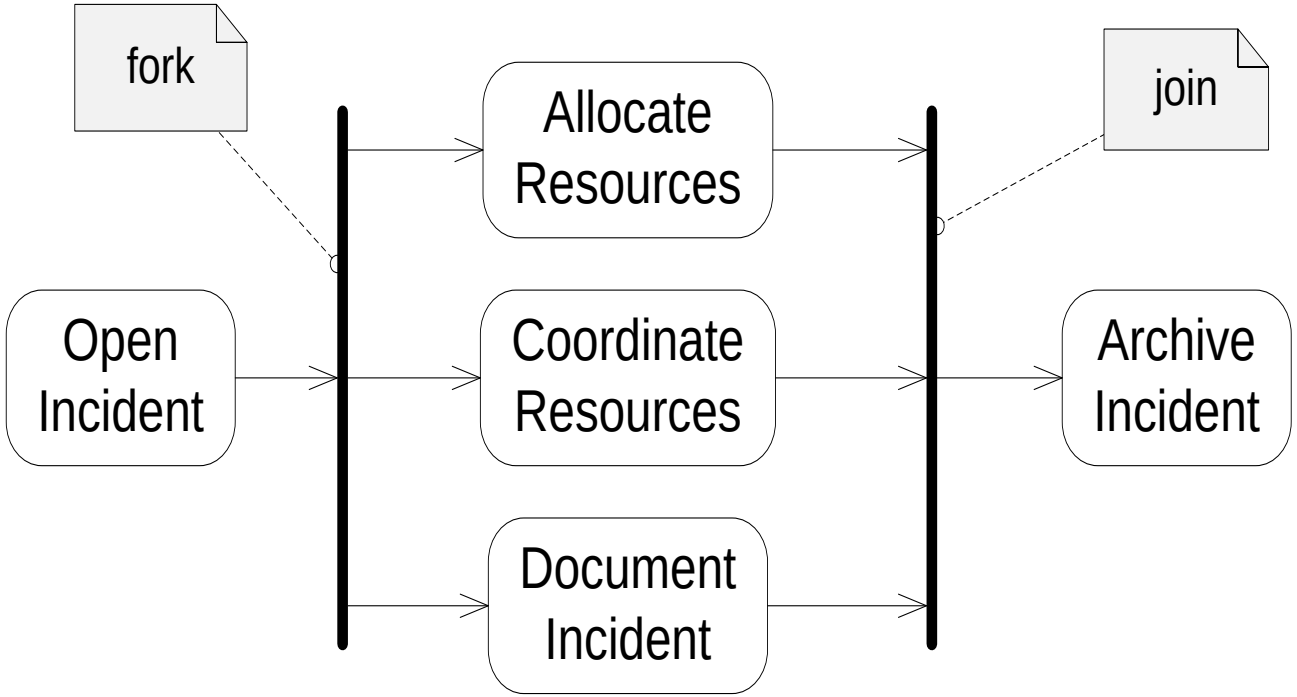
Decisions and Merges

- **Decisions** are used when you want to execute a different sequence of actions depending on a condition.
- Decisions are drawn as diamond-shaped nodes with one incoming edge and multiple outgoing edges, as shown
- Each branched edge contains a **guard condition** written in brackets. Guard conditions determine which edge is taken after a decision node.



Doing Multiple Tasks at the Same Time

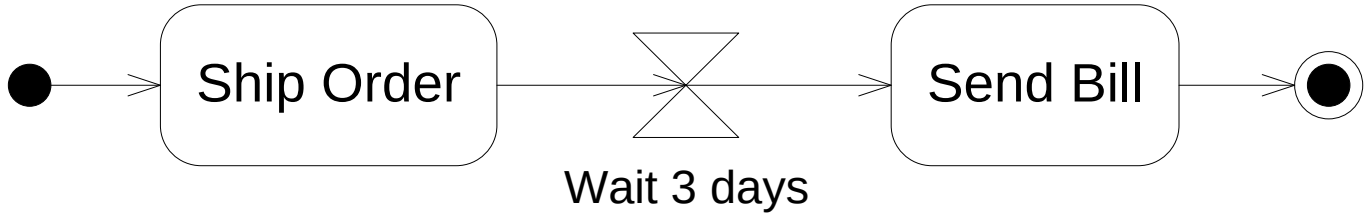
- We represent parallel actions in activity diagrams by using **forks** and **joins**, as shown in activity diagram fragment of FRIEND
- After a fork, the flow is broken up three simultaneous flows, and the actions along all forked flows execute.



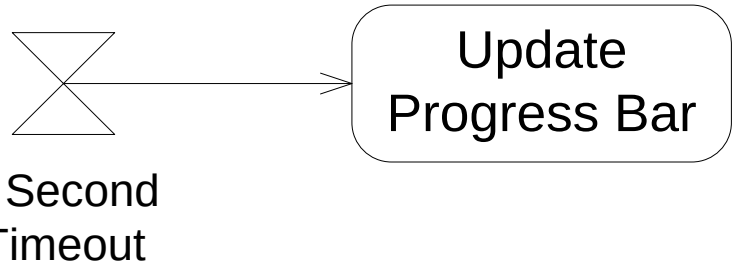
- In this example, Allocate Resources, Coordinate Resources and Document Resources begin executing at the same time.
- The join means that all incoming actions **must finish** before the flow can proceed past the join.

Time Events

- Sometimes time is a factor in your activity. You may want to model a wait period, such as waiting three days after shipping an order to send a bill. You may also need to model processes that kick off at a regular time interval, such as a system backup that happens every week.
- **Time events** are drawn with an hourglass symbol.

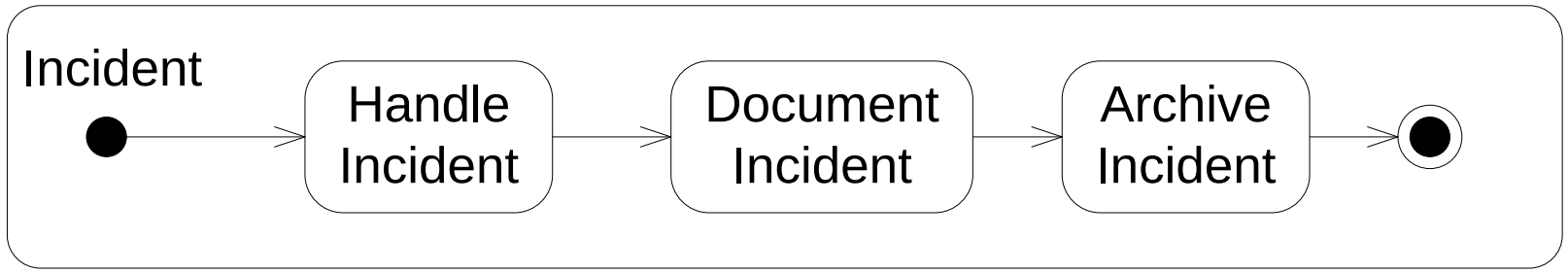


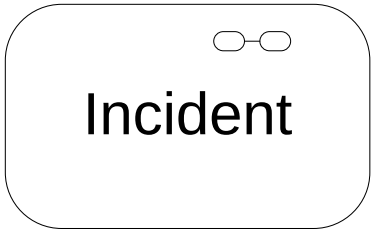
- **A time event with no incoming flows is a recurring** time event, meaning it's activated with the frequency in the text next to the hourglass



Subactivity

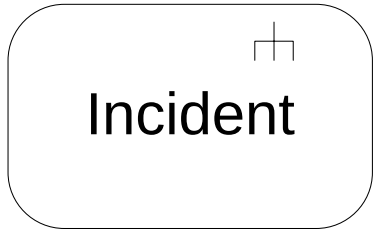
- **Subactivity**
  - Sub Activities hide the complexity of a model.





For composition,

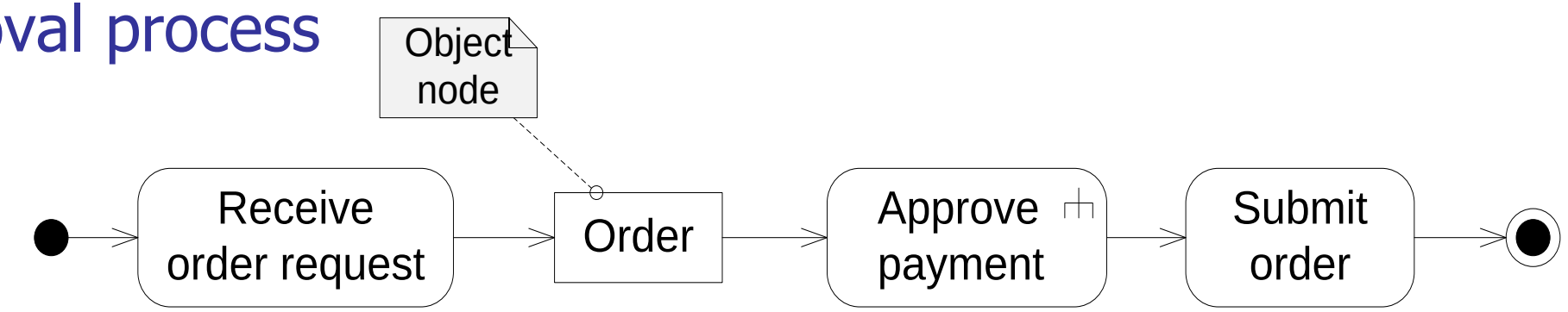
Or



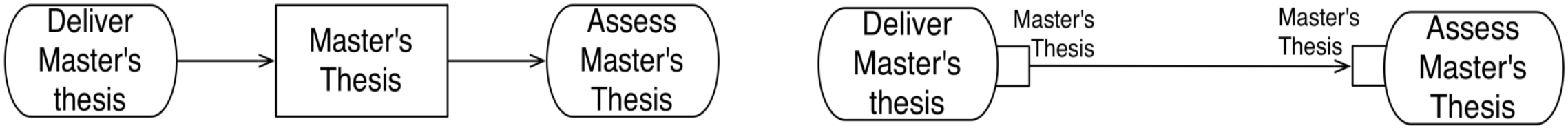
For call behaviour

Objects & Pins

- **Object nodes**
  - To represent objects and data as they flow in and out of invoked behaviors.
  - An object node is drawn with a rectangle, as shown in the order approval process

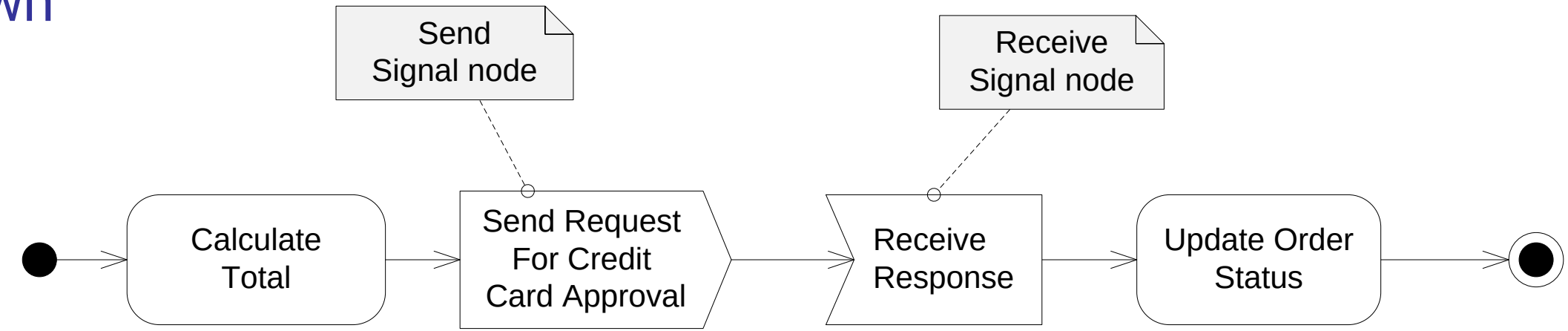


- **Pins**
  - Pin: Abbreviated notation for an object node. Object and Pin notations define object flow in an activity



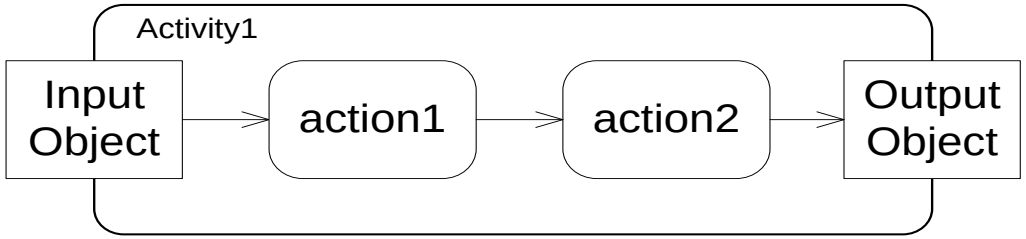
Sending and Receiving Signals

- Activities may involve interactions with external people, systems, or processes.
  - For example, when authorizing a credit card payment, you need to verify the card by interacting with an approval service provided by the credit card company
- In activity diagrams, **signals** represent interactions with external participants. Signals are messages that can be sent or received, as shown

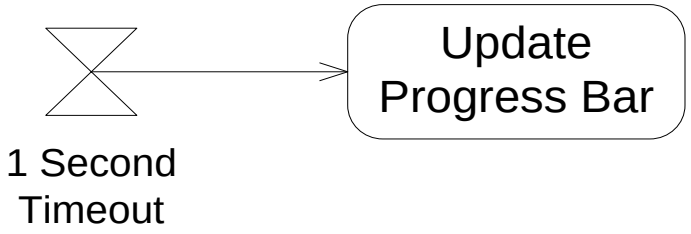


Starting, Ending Activities and Flows

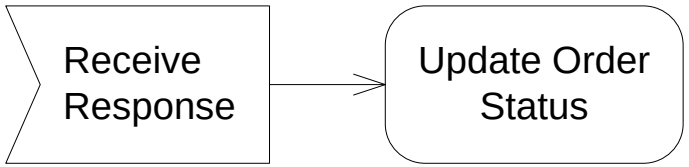
- The simplest and most common way to start an activity is with a single initial node.
- There are other ways to represent the start of an activity that have special meanings:
  - The activity starts by receiving input data (using object node).



  - The activity starts in response to a time event.

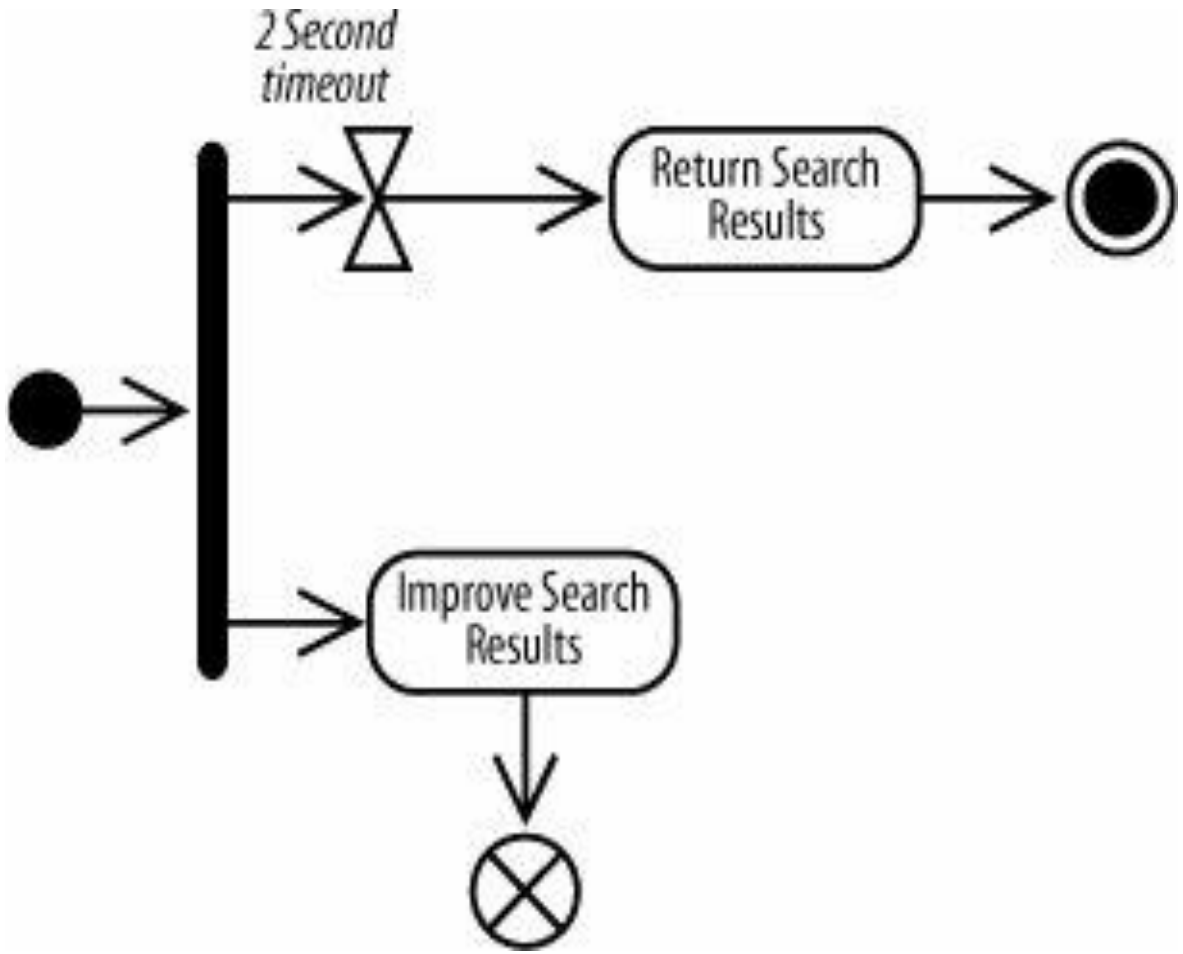


  - The activity starts as a result of being woken up by a signal.



Ending Activities and Flows

- The **activity final node** stops all flows within an activity. An activity diagram may have many activity final nodes, the first one to be activated terminate all other flows and the activity itself.
- The **flow final node** simply stops one of the flows within the activity, the other flows continue.

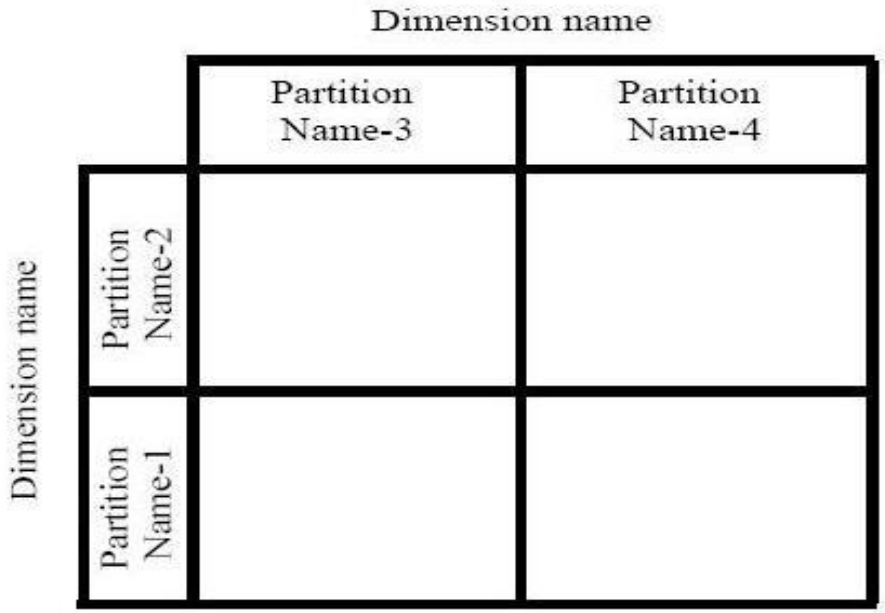


Partitions (or Swimlanes)

Partition: Partitions divide the nodes and edges to constrain and show a view of the contained nodes. They often correspond to organizational units in a business model.

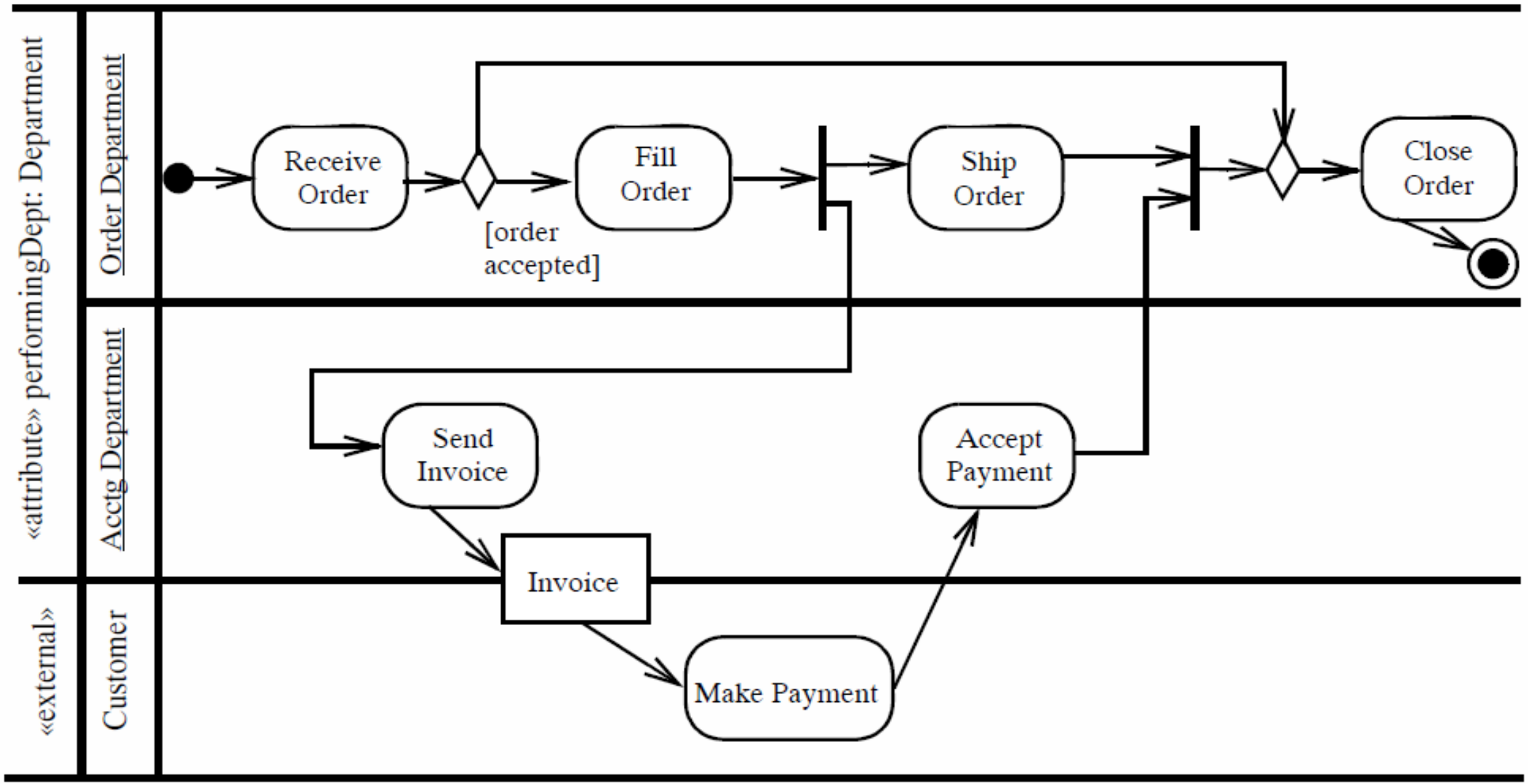
a) Partition using a swimlane notation

b) Partition using a hierarchical swimlane notation

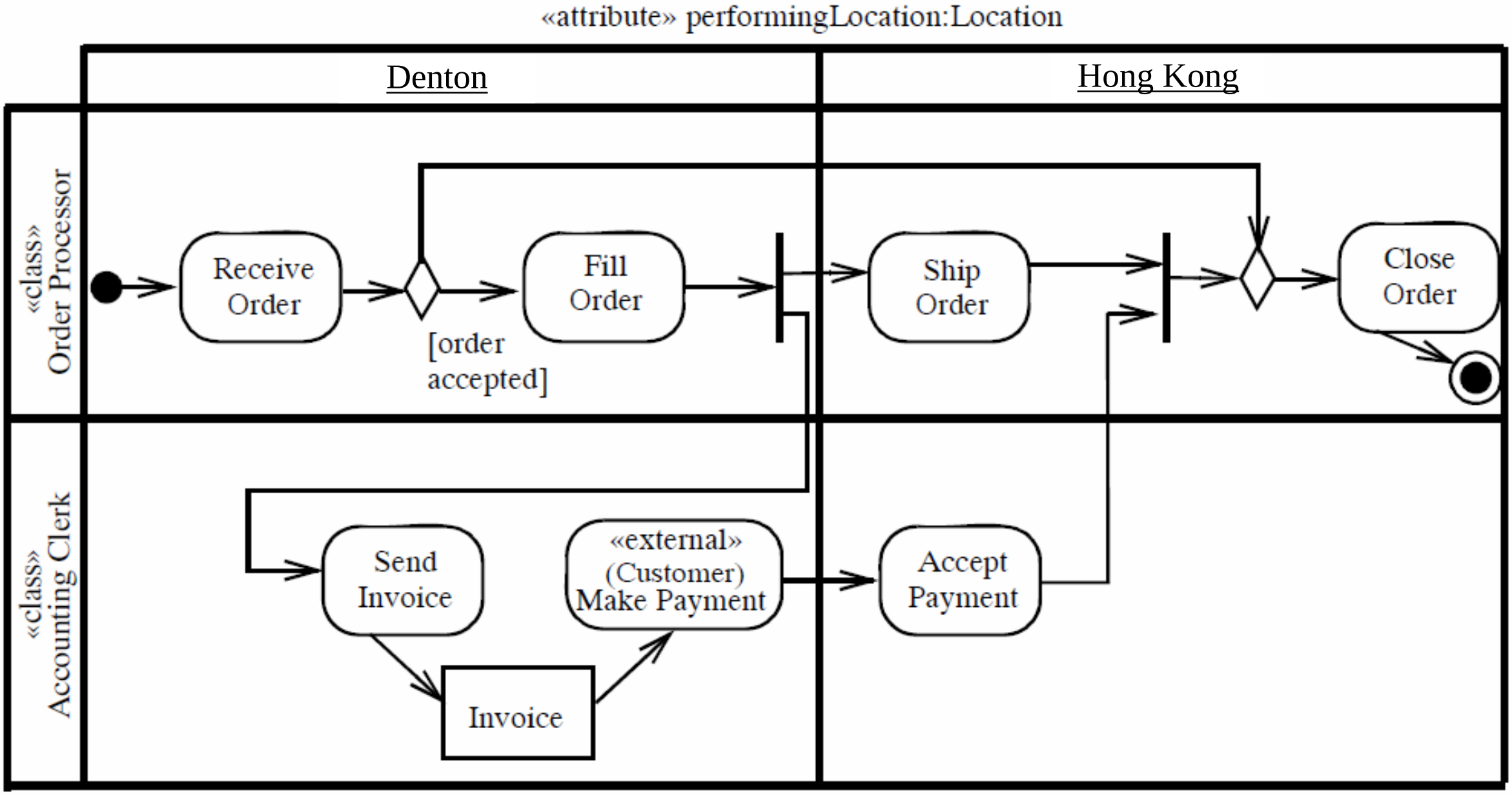


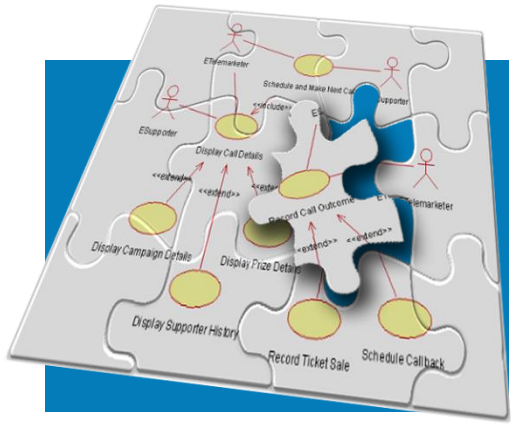
c) Partition using a multidimensional hierarchical swimlane notation

Activity Partition using Swimlane Example



Activity Partition using Multidimensional Swimlane Example





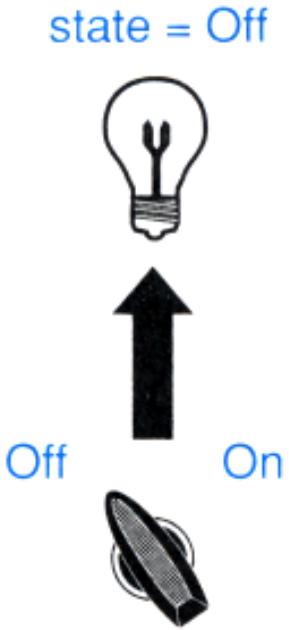
Modeling an Object's State: State Machine Diagrams

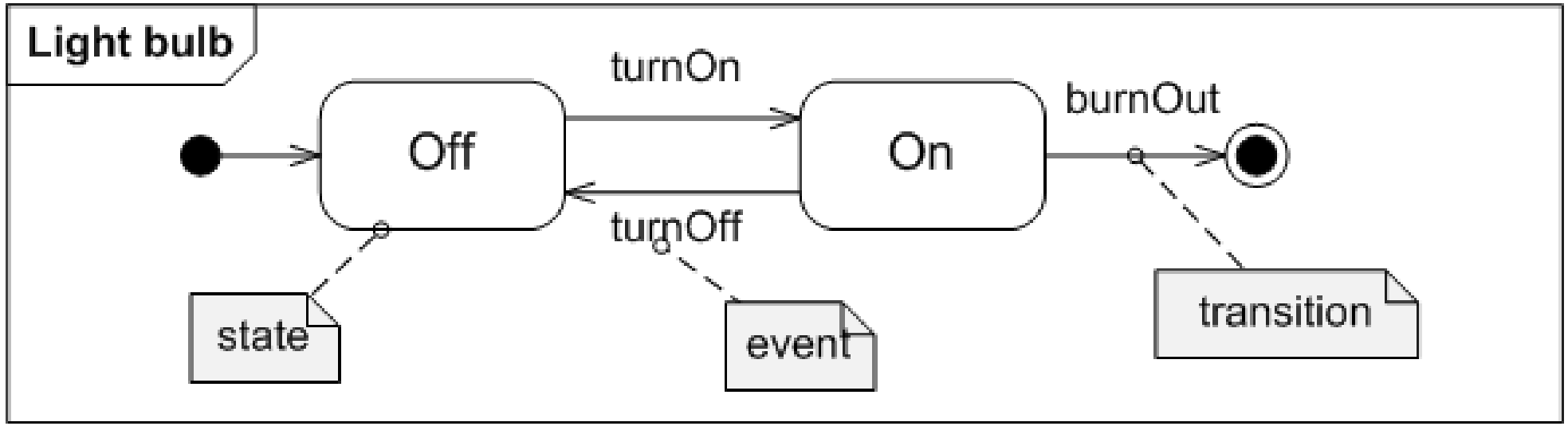
UML State Machine

- While Activity diagrams are used for modeling business processes in which several objects participate.
- UML **state machines** are used for modeling the life cycle history of a single reactive object as a finite state machine
  - a machine that can exist in a finite number of **states**. The machine makes **transitions** between these states in response to **events.**
- The three key elements of state machines are states, events, and transitions:
  - state – a situation of an object during which it satisfies some condition, performs some activity, or waits for some event
  - event – a specific occurrence (in time and space) of a stimulus that can trigger a state transition
  - transition – the movement from one state to another in response to an event.

State Machine Diagrams

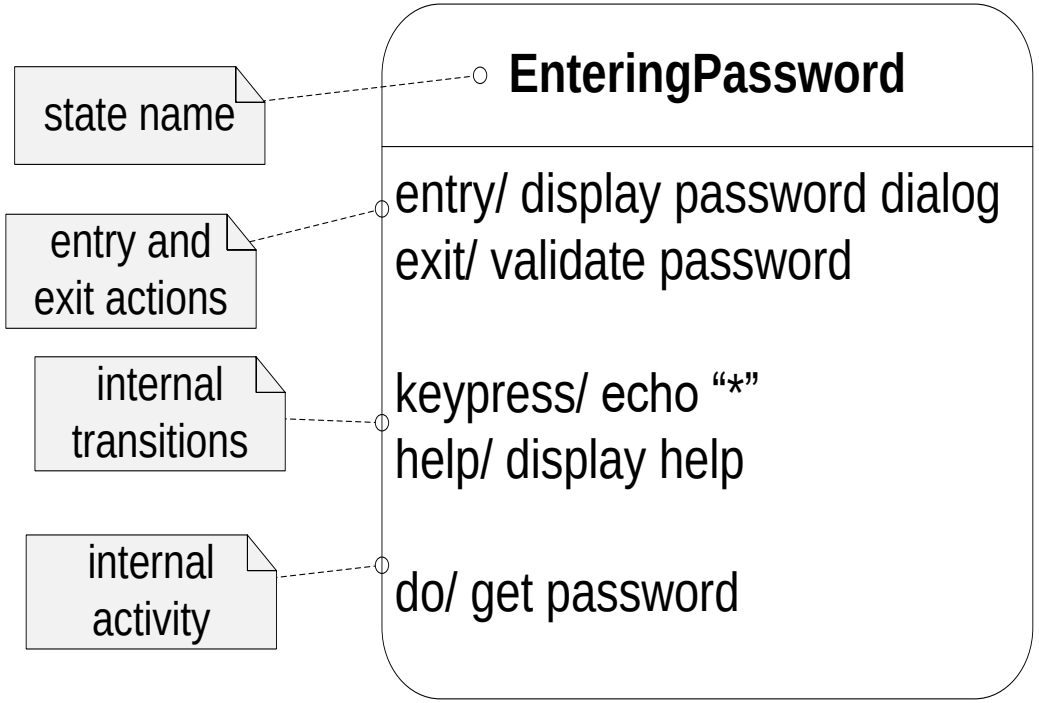
- Let us consider a simple real-world example. A light bulb state machine
- The Figure shows how you can send events to a light bulb by using a switch. The two events you can send are **turnOn** and **turnOff**.
- A state machine diagram contains **exactly one** state machine for **a single reactive object**. In this case, the reactive object is in a system comprising the light bulb, the switch, and the electricity supply.





State Syntax

- UML state syntax is summarized in Figure:
- Each state in a state machine may contain zero or more actions and activities.
- Action is an atomic execution, whereas activity takes a finite amount of time and can be interrupted.
- Each action in a state is associated with an internal transition that is triggered by an event.
- An internal transition doesn't cause a transition to a new state.



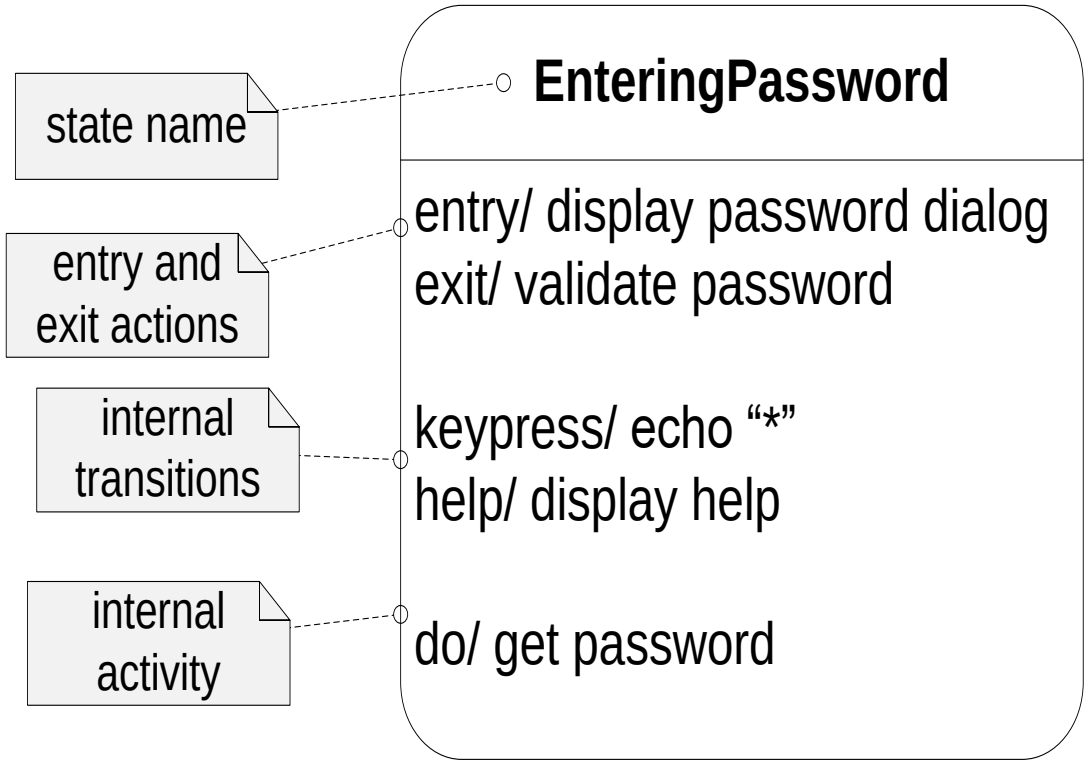
action syntax: **eventName/ someAction**

activity syntax: **do/ someActivity**

- For example, in Figure, pressing one of the keys on the keyboard is an event, but it doesn't cause a transition out of the state Entering-Password. This causes an internal transition that triggers the action echo "*".

State Syntax ...

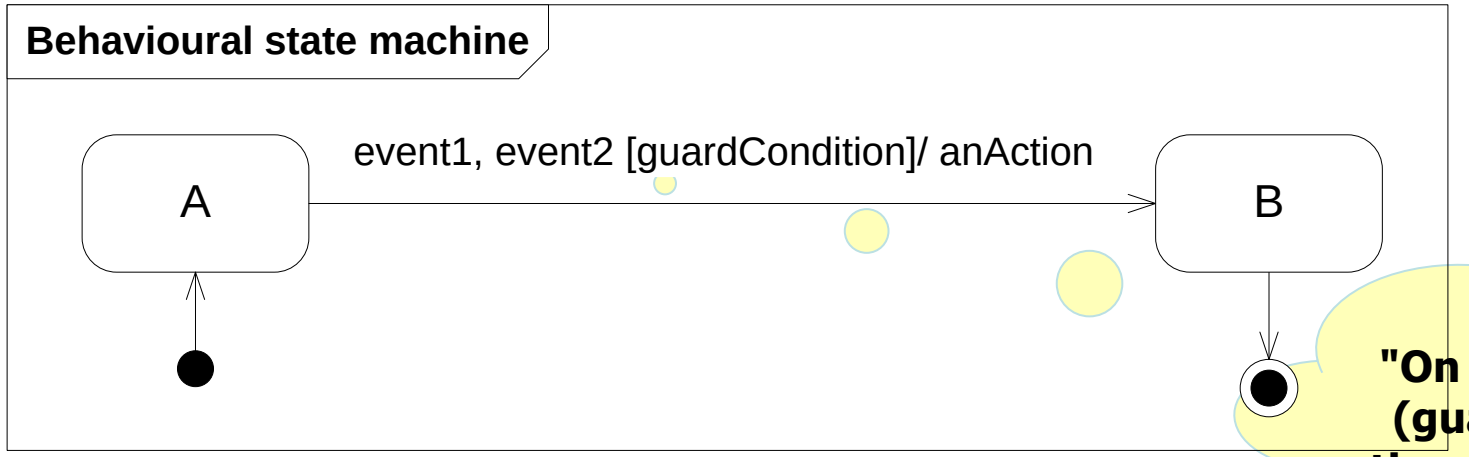
- Two special actions—the entry action and the exit action.
- The entry event occurs automatically on entry to the state—it is the first thing that happens when the state is entered, and it causes the associated entry action to execute.



- The exit event is the very last thing that happens automatically on exit from the state, and it causes the associated exit action to execute.
- Activities, on the other hand, take a finite amount of time and may be interrupted by the receipt of an event. The keyword do indicate an activity.
- Whereas actions always finish because they are atomic, it is possible to interrupt an activity before it has finished processing.

Transitions

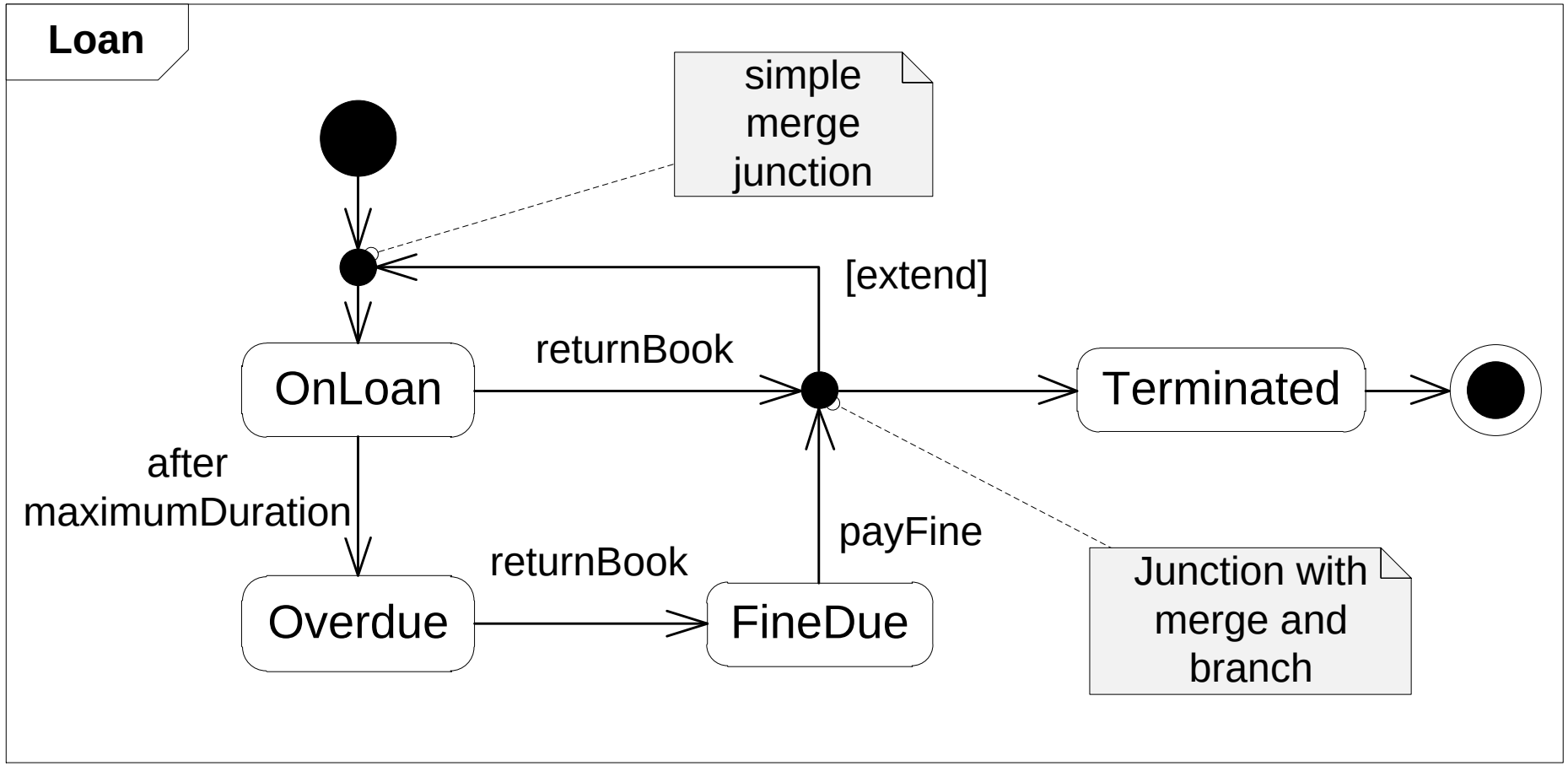
- UML transition syntax for behavioral state machines is summarized in Figure



- Every transition has three optional elements.
  - Zero or more events — these specify external or internal occurrences that can trigger the transition.
  - Zero or one guard condition — this is a Boolean expression that must evaluate to true before the transition can occur. It is placed after the events.
  - Zero or more actions — this is a piece of work associated with the transition, and occurs when the transition fires.

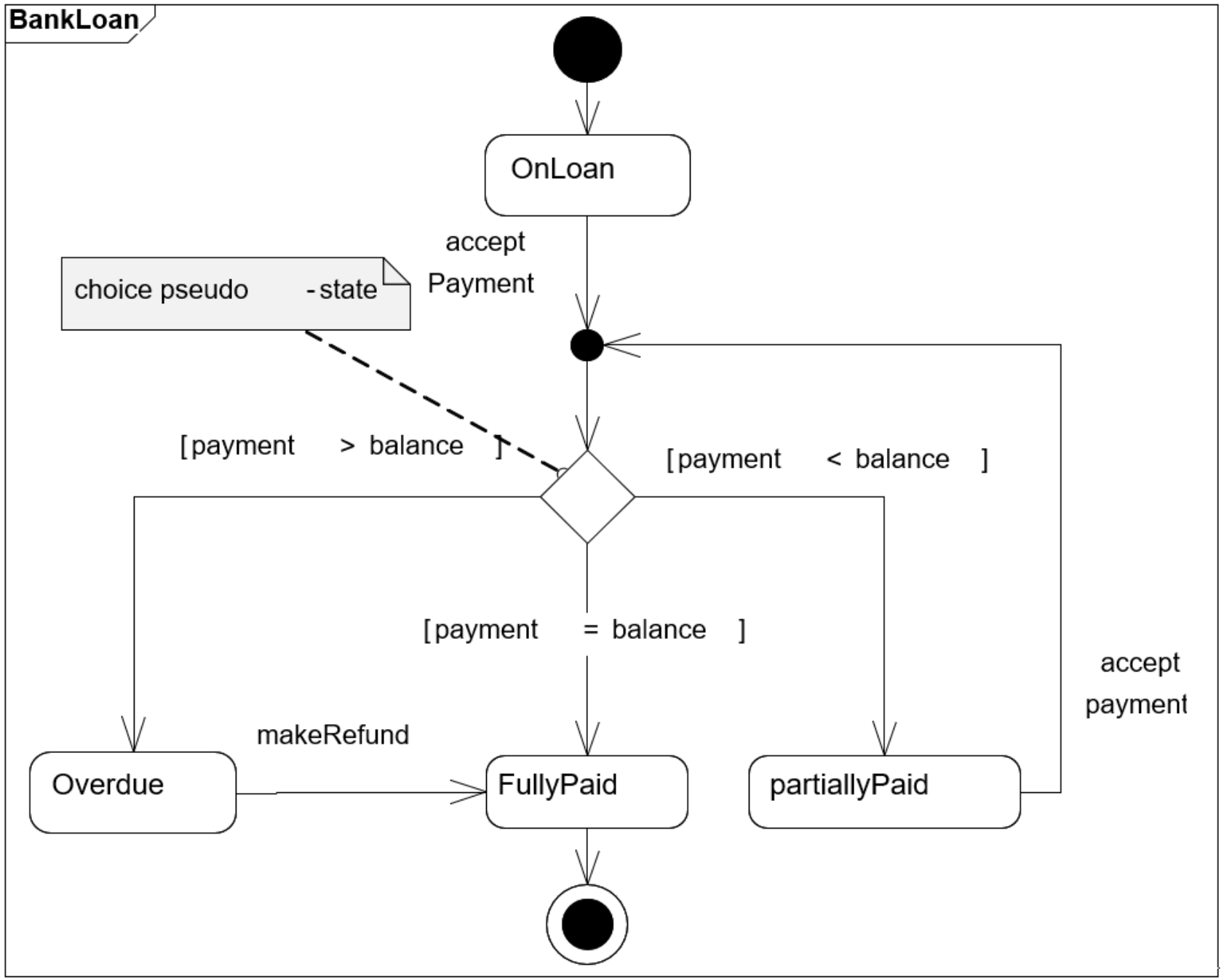
Connecting transitions – the junction pseudo-state

- Transitions can be connected by junction pseudo-states.
- These represent points where transitions merge or branch.
- They are represented as filled circles that have one or more input transitions and one or more output transitions.



Branching transitions – the choice pseudo-state

- If you want to show a simple branch without a merge, you should use a choice pseudo-state, as shown.

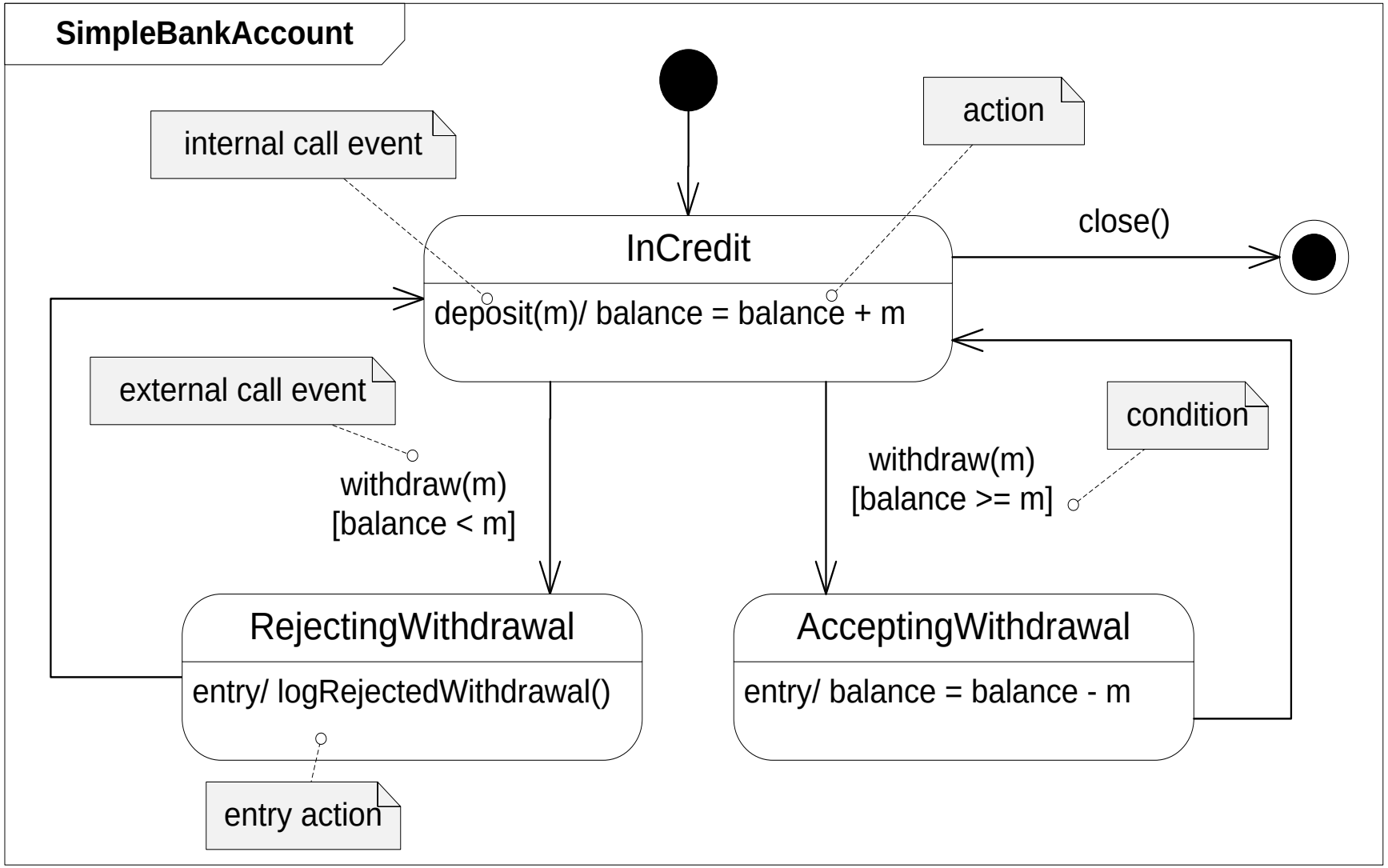


Events

- UML defines an event as a specific occurrence (in time and space) of a stimulus that can trigger transitions in state machines.
- There are four types of event, each of which has different semantics:
  - **call event;**
  - **signal event;**
  - **change event;**
  - **time event.**

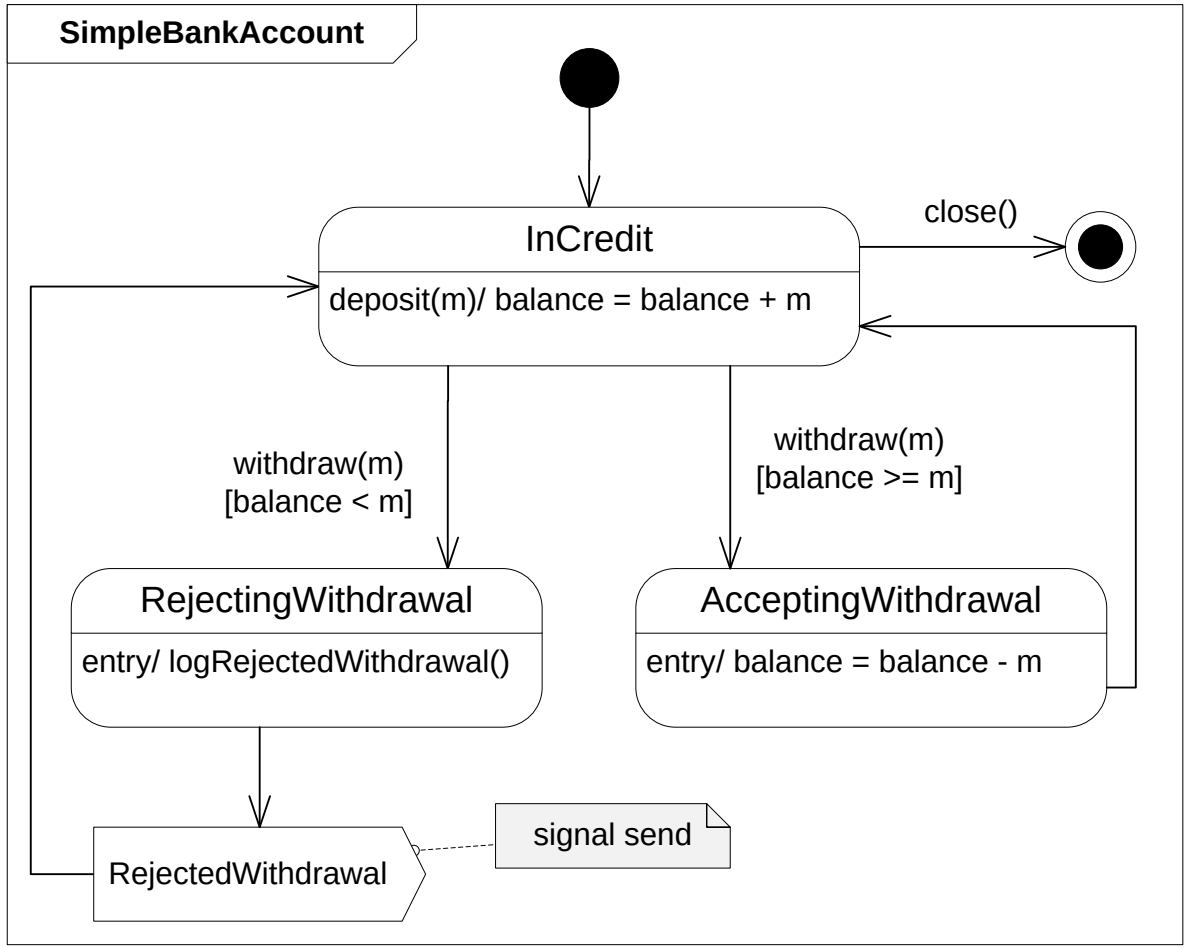
Call Events

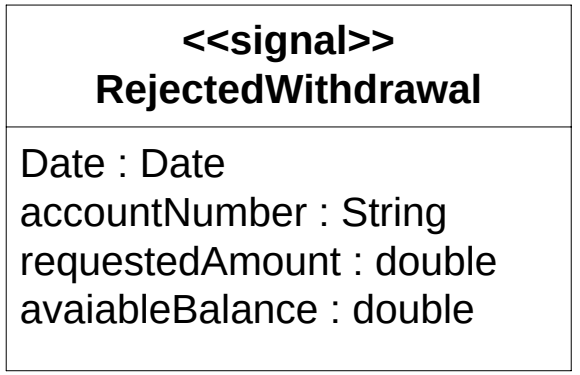
- A call event is a request for a specific operation to be invoked on an instance of the context class.
- Receipt of a call event is a trigger for the operation to execute.



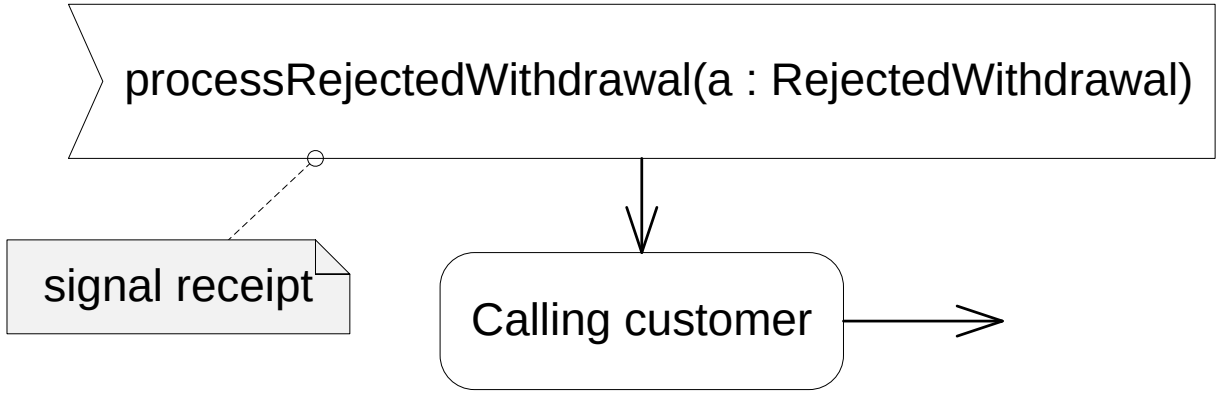
Signal Events

- A signal is a package of information that is sent asynchronously between objects.
- A signal usually doesn't have any operations because its sole purpose is to carry information.



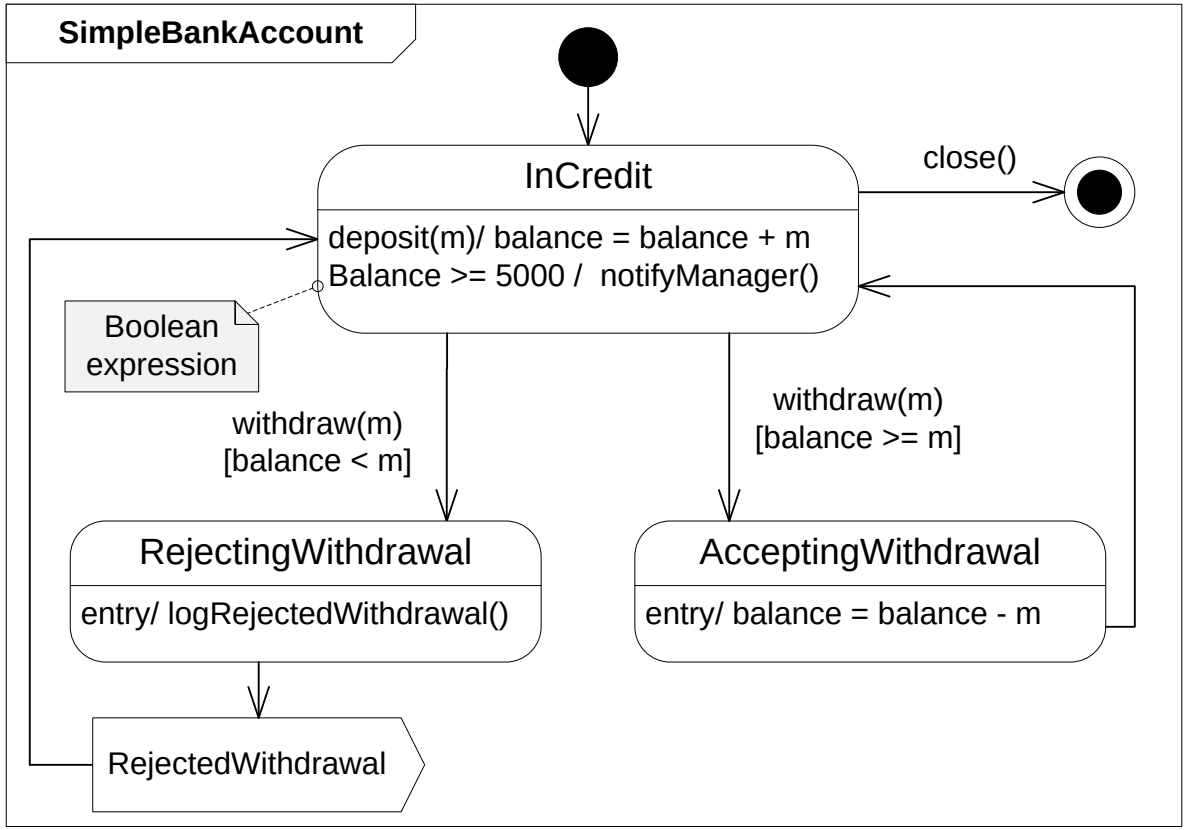


Context: Bank class



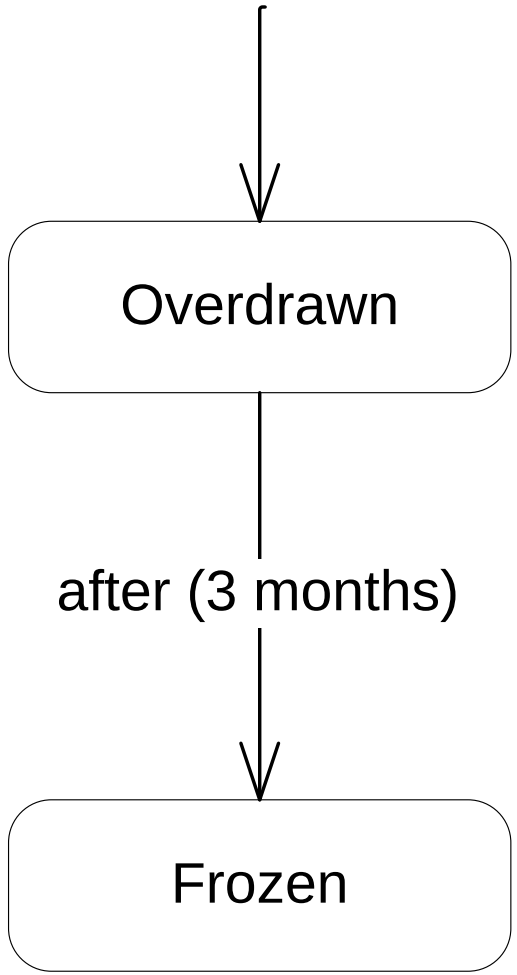
Change Events

- A change event is specified as a **Boolean expression**.
- The action associated with the event is performed when the value of the Boolean expression transitions from false to true.
- From the implementation perspective, a change event implies continually testing the Boolean condition while in the state.

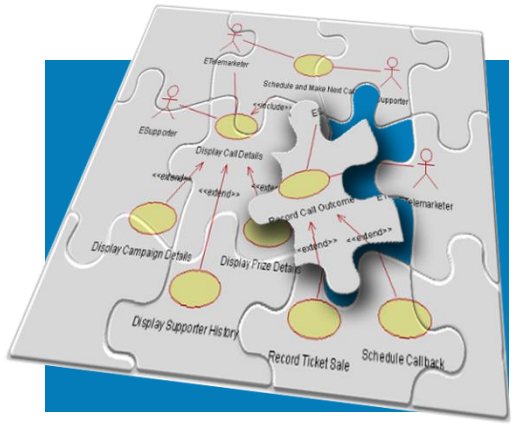


Time Events

- Time events are usually indicated by the keywords **when** and **after**.
- The keyword when specifies a particular time at which the event is triggered; after specifies a threshold time after which the event is triggered.
  - For examples, when(date = 24/01/2011) and after(3 months).



context: CreditAccount class



Modeling State-Dependent Behavior of Individual Objects

Define States

- After we have a pretty clear understanding of the functionality provided by each class in our analysis object model,
  - we can determine which of these classes have **significant state behavior** and model that behavior.
- **What is a state?**
  - Can be defined by the input of the object and the operations that can be performed as a result.
- **Why do we define states?**
  - The state of an object is one of the possible conditions in which an object may exist. Also help identify more class operations.
- **Things to consider:**
  - Which objects have significant state?
  - How to determine an object's possible states?

UML State Machine

RECALL

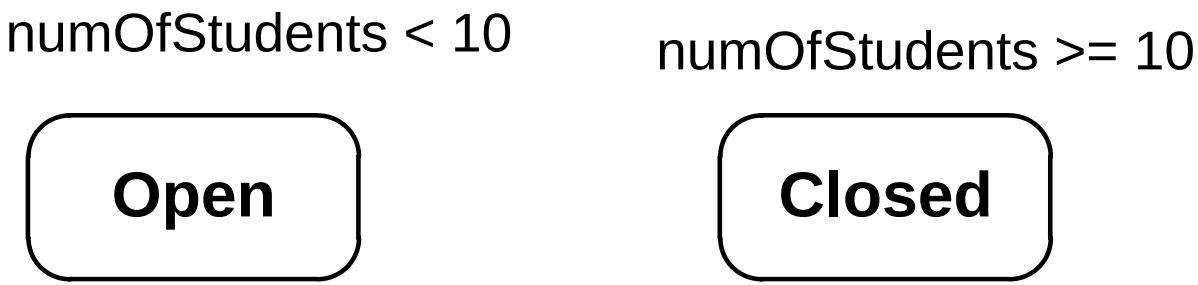
- UML **state machines** are used for modeling the life cycle history of a single reactive object as a finite state machine
  - a machine that can exist in a finite number of **states**. The machine makes **transitions** between these states in response to **events.**
- The three key elements of state machines are states, events, and transitions:
  - state – a situation of an object during which it satisfies some condition, performs some activity, or waits for some event
  - event – a specific occurrence (in time and space) of a stimulus that can trigger a state transition
  - transition – the movement from one state to another in response to an event.

Identify and Define the States (1)

- The states of an object can be found by looking at its class attributes and relationships with other classes.
- For example: consider the **CourseOffering** class.

CourseOffering
- numOfStudents : Integer

If the maximum number of students per course (ceiling) is 10, then the attribute **numOfStudents** can define the state of CourseOffering class whether it is open or closed.



Identify and Define the States (2)

- For example: consider the **link** between the CourseOffering class and Professor Class.

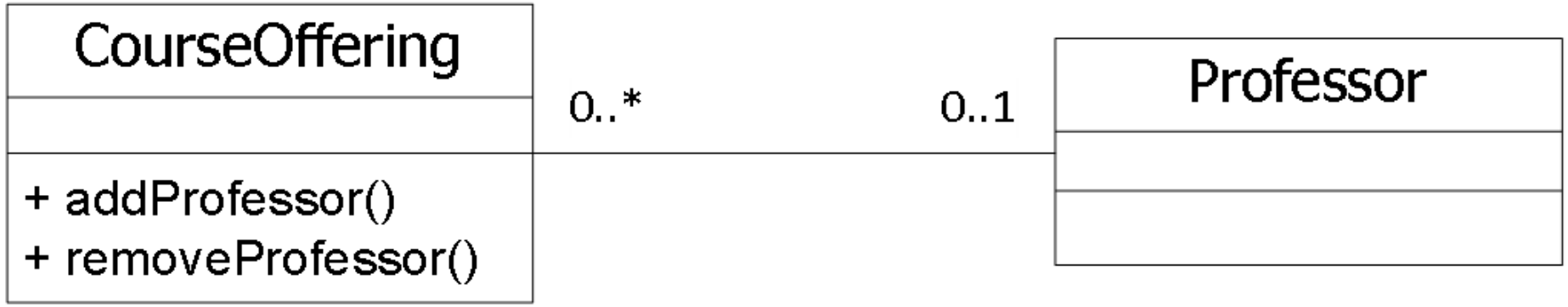
Based on the defined association, a CourseOffering instance may have a Professor **assigned** to teach it or not. If a Professor has been assigned to the CourseOffering instance, the state of the CourseOffering instance is "Assigned." If a Professor has not been assigned to the CourseOffering instance, the state of the CourseOffering instance is "Unassigned."

Link to Professor exists

Link to Professor doesn't exist

Identify the Events

- Look at the class operations

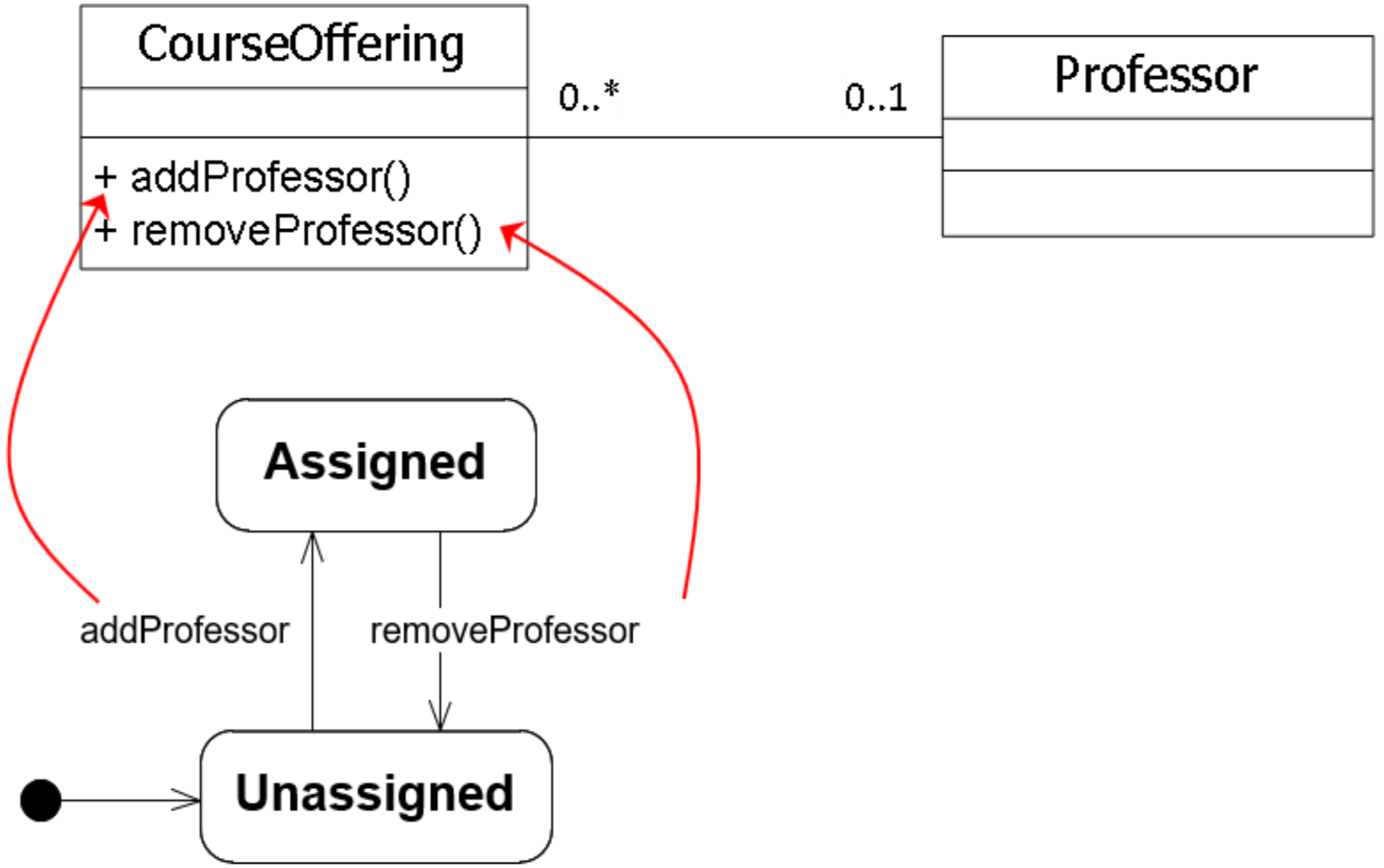


Events: addProfessor, removeProfessor

Note: Events are not operations, although they often map 1 to 1.

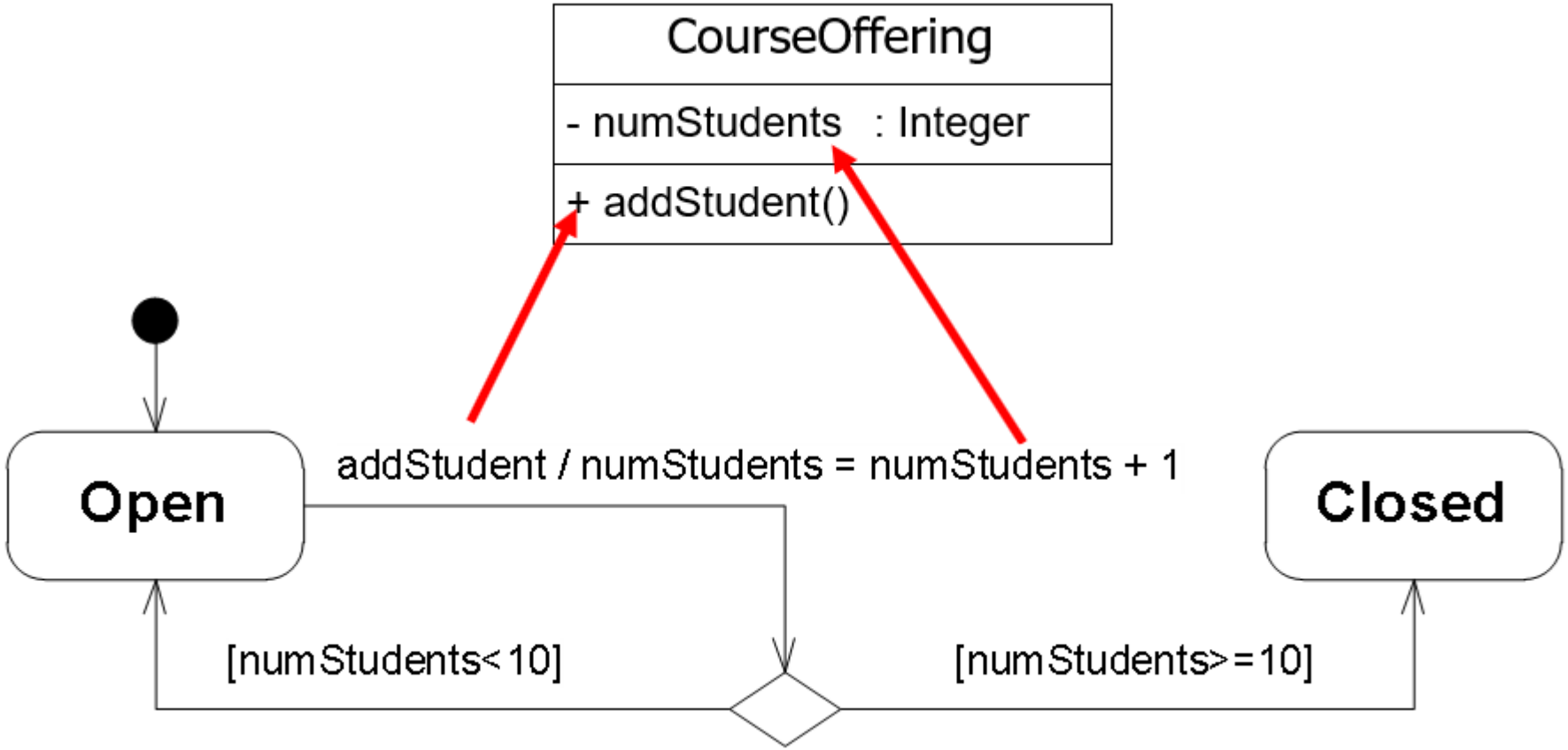
Identify the Transitions (1)

- For each state, determine what events cause transitions to what states, including guard conditions, when needed
- Transitions describe what happens in response to the receipt of an event

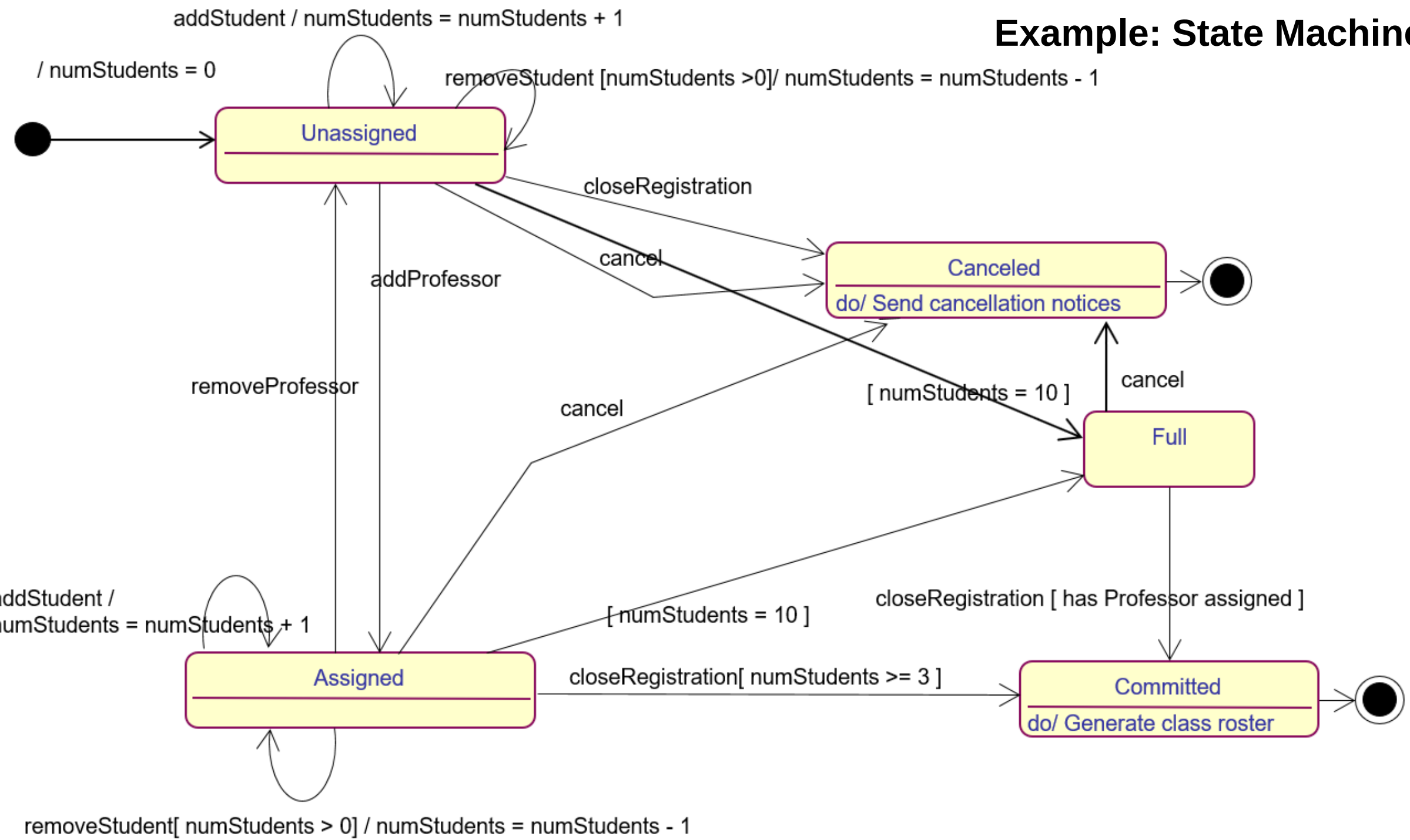


Identify the Transitions (2)

- Some state transition events can be associated with an operation.
- Depending on the object's state, the operation may have a different behavio



Example: State Machine



Questions?